Pruebas JavaScript

© JMA 2020. All rights reserved

INTRODUCCIÓN

© JMA 2020. All rights reserved

Calidad de Software

- El JavaScript es un lenguaje muy poco apropiado para trabajar en un entorno de calidad de software.
- En descargo del lenguaje JavaScript y de su autor, Brendan Eich, hay que decir que los problemas que han forzado esta evolución del lenguaje (así como las críticas ancestrales de la comunidad de desarrolladores) vienen dados por lo que habitualmente se llama "morir de éxito".
- Jamás se pensó que un lenguaje que Eich tuvo que montar en 12 días como una especie de "demo" para Mozilla, pasase a ser omnipresente en miles de millones de páginas Web. O como el propio Hejlsberg comenta:

  "JavaScript se creó –como mucho- para escribir cien o doscientas líneas de código, y no los cientos de miles necesarias para algunas aplicaciones actuales."
- JavaScript y el resto de los lenguajes web (HTML, CSS, …) son interpretados directamente por el navegador, por lo que hasta los errores sintácticos pueden llegar a producción, agravando los problemas de calidad.

© JMA 2020. All rights reserved

Gestión de la calidad

La gestión de la calidad (QM Quality Management) es el conjunto de actividades y procesos que se llevan a cabo en una organización con el objetivo de garantizar y mejorar la calidad de los productos o servicios que ofrece. Considera de forma completa la cadena de valor de un proyecto, proceso o producto. Es un concepto global, dentro del cual se incluyen otros conceptos anidados:

- QA (Quality Assurance) o Aseguramiento de la Calidad: Se centra en proporcionar confianza en que se cumplirán los requisitos de calidad. Basada en metodologías y buenas practicas, se enfoca de manera proactiva en los procesos y sistemas.
- QC (Quality Control) o Control de la Calidad: Se centra en el cumplimiento de los requisitos de calidad. Se enfoca de manera reactiva en las partes del sistema y los productos.
- Testing o Pruebas: Es el proceso de detección de errores en un sistema o producto para obtener las métricas que permiten evaluar los procesos y la calidad. Ayuda a reducir riesgos e incrementar la confianza.

© JMA 2020. All rights reserved

Pruebas

- Las pruebas consisten en actividades de verificación, validación y exploración que brindan información sobre la calidad y los riesgos relacionados, para establecer el nivel de confianza de que el objeto de prueba podrá entregar el valor comercial buscado en la base de la prueba.
- La verificación es la confirmación mediante examen y mediante la provisión de evidencia objetiva de que se han cumplido los requisitos especificados. Responde a la pregunta: ¿Estamos construyendo correctamente el sistema de TI?
- La validación es la confirmación mediante examen y mediante la provisión de evidencia objetiva de que se han cumplido las demandas para un uso específico previsto. Responde a la pregunta: ¿Estamos construyendo el sistema de TI adecuado?
- La exploración es la actividad de investigar y establecer la calidad y los riesgos del uso de un sistema de TI mediante examen, indagación y análisis. Responde a la pregunta: ¿Cómo se podría (mal)utilizar el sistema de TI?
- Las pruebas deben proporcionar diferentes niveles de información:
  - Detallado: para conocer la calidad de los objetos nuevos y modificados que entregaron e investigar anomalías en el sistema de TI y tomar medidas correctivas cuando sea necesario.
  - Intermedio: para realizar un seguimiento del estado y el progreso.
  - General: para respaldar las decisiones de seguir o no.

© JMA 2020. All rights reserved

Criterios de evaluación

- Las pruebas consisten en evaluar la calidad basándose en criterios. En las pruebas utilizamos criterios para determinar si el objeto de prueba cumple con las expectativas sobre calidad y riesgos.
- Los **criterios de entrada** son los criterios que un objeto (por ejemplo, un documento base de prueba o un objeto de prueba) debe satisfacer para estar listo para usarse en una actividad específica.
- Los **criterios de salida** son los criterios que un objeto (por ejemplo, un documento base de prueba o un objeto de prueba) debe satisfacer para estar listo al final de una actividad o etapa específica del proyecto.
- Los **criterios de aceptación** son los criterios que debe cumplir un objeto de prueba para ser aceptado por un usuario, cliente u otra parte interesada.
- Los **criterios de finalización** son los criterios que un equipo debe satisfacer para haber completado una (grupo de) actividad(es).

© JMA 2020. All rights reserved

Error, defecto o fallo

- En el área del aseguramiento de la calidad del software, debemos tener claros los conceptos de Error, Defecto y Fallo. En muchos casos se utilizan indistintamente pero representan conceptos diferentes:
  - Error: Es una acción humana, una idea equivocada de algo, que produce un resultado incorrecto. Es una equivocación por parte del desarrollador o analista.
  - Defecto: Es una imperfección de un componente causado por un error. El defecto se encuentra en algún componente del sistema. El analista de pruebas es quien debe encontrar el defecto ya que es el encargado de elaborar y ejecutar los casos de prueba.
  - Fallo: Es la manifestación visible de un defecto. Si un defecto es encontrado durante la ejecución de una aplicación entonces va a producir un fallo.
- Un error puede generar uno o más defectos y un defecto puede causar un fallo.

© JMA 2020. All rights reserved

V & V

- Validación: ¿Estamos construyendo el sistema correcto?
  - Proceso de evaluación de un sistema o componente durante o al final del proceso de desarrollo para comprobar si se satisfacen los requisitos especificados (IEEE Std610.12-1990)
- Verificación: ¿Estamos construyendo correctamente el sistema?
  - Proceso de evaluar un sistema o componente para determinar si los productos obtenidos en una determinada fase de desarrollo satisfacen las condiciones impuestas al comienzo de dicha fase (IEEE Std610.12-1990)

© JMA 2020. All rights reserved

Depuración y Pruebas

- Cuando se han encontrado fallos en un programa, éstos deben ser localizados y eliminados. A este proceso se le denomina depuración.
- La prueba de defectos y la depuración son consideradas a veces como parte del mismo proceso. En realidad, son muy diferentes, puesto que la prueba establece la existencia de fallos, mientras que la depuración se refiere a la localización los defectos/errores que se han manifestado en los fallos y corrección de los mismos.
- El proceso de depuración suele requerir los siguientes pasos:
  - Identificación de errores
  - Análisis de errores
  - Corrección y validación

© JMA 2020. All rights reserved

Testing vs QA

- El aseguramiento de calidad o QA (Quality Assurance), esta orientado al proceso de obtener un software de calidad, que viene determinado por las metodologías y buenas practicas empleadas en el desarrollo del mismo, y se inicia incluso antes que el propio proyecto.
- Las pruebas son parte del proceso de QA, validan y verifican la corrección del producto obtenido, destinadas a revelar los errores inherentes a cualquier actividad humana que ni las mejores practicas o metodologías pueden evitar.

© JMA 2020. All rights reserved

Principios fundamentales

- Hay 7 principios fundamentales respecto a las metodologías de pruebas que deben quedar claros desde el primer momento aunque volveremos a ellos continuamente:
  - Las pruebas exhaustivas no son viables
  - El proceso de pruebas no puede demostrar la ausencia de defectos
  - La mayoría de defectos relevantes suelen concentrarse en partes muy concretas.
  - Las pruebas se deben ejecutar bajo diferentes condiciones
  - Las pruebas no garantizan ni mejoran la calidad del software
  - Las pruebas tienen un coste
  - El inicio temprano de pruebas ahorran tiempo y dinero

© JMA 2020. All rights reserved

Las pruebas exhaustivas no son viables

- Es inviable (tiempo, coste o recursos) o imposible (cardinalidad) crear casos de prueba que cubran todas las posibles combinaciones de entrada y salida que pueden llegar a tener las funcionalidades (salvo que sean triviales).
- Por otro lado, en proyectos cuyo número de casos de uso o historias de usuario desarrollados sea considerable, se requeriría de una inversión muy alta en cuanto a recursos y tiempo necesarios para cubrir con pruebas todas las funcionalidades del sistema.
- Por lo tanto es conveniente realizar un análisis de riesgos de todas las funcionalidades y determinar en este punto cuales serán objeto de prueba y cuales no, creando pruebas que cubran el mayor número de casos de prueba posibles.

© JMA 2020. All rights reserved

El proceso no puede demostrar la ausencia de defectos

- Independientemente de la rigurosidad con la que se haya planeado el proceso de pruebas de un producto, nunca será posible garantizar al ejecutar este proceso, la ausencia total de defectos (es inviable una cobertura del 100% de los casos de uso).
- Una prueba se debe considerar un éxito si detecta un error. Si se supera la prueba, no detecta un error, no significa que no haya error, significa que no se ha detectado. Una prueba fallida es concluyente, una prueba superada no lo es.
- Un proceso de pruebas riguroso puede garantizar una reducción significativa de los posibles fallos y/o defectos del software, sobre todo en las áreas criticas, pero nunca podrá garantizar que no fallará en producción (la falacia de la ausencia de errores).
- Las pruebas reducen la probabilidad de la presencia de defectos que permanezcan sin ser detectados. La ausencia de fallos no demuestran la corrección de un producto software

© JMA 2020. All rights reserved

Agrupación de defectos

- Generalmente, hay ciertos módulos y funcionalidades que son más proclives a presentar incidencias (de mayor prioridad) en comparación al resto de las partes que conforman un producto.
- Encuentra un defecto y encontrarás más defectos cerca. Los defectos aparecen agrupados como hongos o cucarachas, vale la pena investigar un mismo módulo donde se ha detectado un defecto: fallo → defecto → error → defectos → fallos.
- Este es el Principio de Pareto aplicado a las pruebas de software, donde el 80% de los problemas se encuentran en el 20% del código. Existen múltiples razones (complejidad, especificidad, novedad, organizativas, …) que explican esta concentración.
- El Principio de Pareto también se puede aplicar a la funcionalidad, el 80% de los usuarios solo usa el 20% de la funcionalidad.
- La coincidencias de ambos 20% puede llevar a una situación critica.

© JMA 2020. All rights reserved

Ejecución de pruebas bajo diferentes condiciones

- El plan de pruebas determina la condiciones y el número de ciclos de prueba que se ejecutarán sobre las funcionalidades del negocio.
- Por cada ciclo de prueba, se generan diferentes tipos de condiciones, una combinación única, basados principalmente en la variabilidad de los datos de entrada y en los conjuntos de datos utilizados.
- No es conveniente, ejecutar en cada ciclo, los casos de prueba basados en los mismos datos del ciclo anterior, dado que con mucha probabilidad, se obtendrán los mismos resultados. Cuando se ejecutan los mismos casos de pruebas una y otra vez y sin ningún cambio, eventualmente estos dejarán de encontrar defectos nuevos (paradoja del pesticida). La paradoja del pesticida solo resulta beneficiosa en las pruebas de regresión.
- Ejecutar ciclos bajo diferentes tipos de condiciones, permitirá identificar posibles fallos en el sistema que antes no se detectaron y no son fácilmente reproducibles.

© JMA 2020. All rights reserved

Las pruebas no garantizan ni mejoran la calidad del software

- Las pruebas ayudan a **mejorar la percepción** de la calidad permitiendo la eliminación de los defectos detectados. Una prueba fallida no beneficia directamente al usuario. ¡El valor solo se crea cuando se soluciona el error!
- La calidad del software viene determinada por las metodologías y buenas practicas empleadas en el desarrollo del mismo. Actualmente hay metodologías que ponen el foco en las pruebas.
- Las pruebas **permiten medir la calidad** del software, lo que permite, a su vez, mejorar los procesos de desarrollo que son los que conllevan la mejora de la calidad y permiten garantizar un nivel determinado de calidad.
- Las pruebas dependen de su contexto: la estrategia y el tipo de pruebas serán seleccionados en función del sistema y a los entornos que se pretenden verificar.

© JMA 2020. All rights reserved

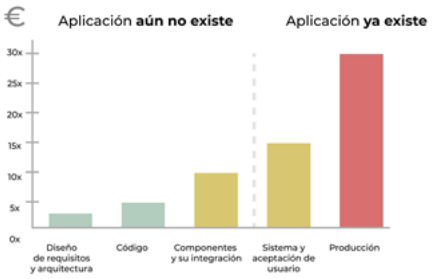
Las pruebas tienen un coste

- Aunque exige dedicar esfuerzo para crear y mantener los test (coste para las empresas), los beneficios obtenidos son mayores que la inversión realizada.
- El proceso de prueba implica más que sólo la ejecución de la prueba.
- La creciente inclusión del software como un elemento más de muchos sistemas productivos y la importancia de los "costes" asociados a un fallo del mismo están motivando la creación de pruebas minuciosas y bien planificadas.
- No es raro que una organización de desarrollo de software gaste el 40 por 100 del esfuerzo total de un proyecto en la prueba. En casos extremos, la prueba del software para actividades críticas (por ejemplo, hay vidas o mucho dinero en juego, como control de tráfico aéreo o de reactores nucleares, …) puede costar ¡de 3 a 5 veces más que el resto de los pasos de la ingeniería del software juntos!
- El coste de hacer las pruebas es siempre muy inferior al coste de no hacer las pruebas (deuda técnica).

© JMA 2020. All rights reserved

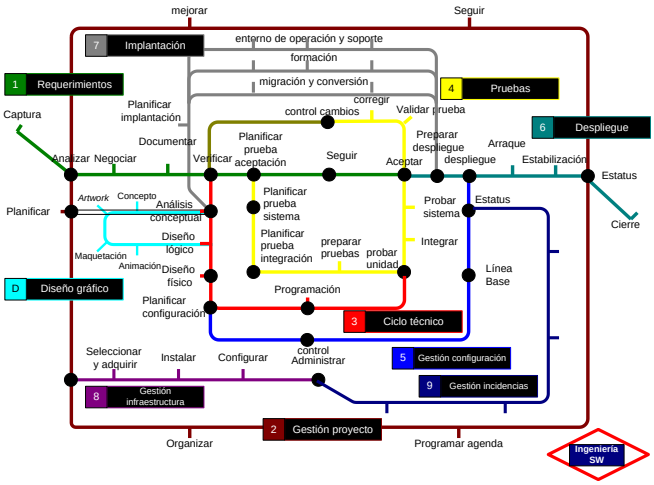
Inicio temprano de pruebas

- Si bien la fase de pruebas es la última del ciclo de vida, las actividades del proceso de pruebas deben ser incorporadas desde la fase de especificación, incluso antes de que se ejecutan las etapas de análisis y diseño.
- De esta forma, los documentos de especificación y de diseño deben ser sometidos a revisiones y validaciones (pruebas estáticas), lo que ayudará a detectar problemas en la lógica del negocio mucho antes de que se escriba una sola línea de código.
- Cuanto mas temprano se detecte un defecto, ya sea sobre los entregables de especificación, diseño o sobre el producto, menor impacto tendrá en el desarrollo y menor será el costo de dar solución a dichos defectos.



© JMA 2020. All rights reserved

Integración en el ciclo de vida
Símil del mapa del Metro



© JMA 2020. All rights reserved

Conflicto de intereses

- En cualquier proyecto software existe un **conflicto de intereses** inherente que aparece cuando comienza la prueba:
  - Desde un punto de vista psicológico, el análisis, diseño y codificación del software son tareas **constructivas**.
    - El ingeniero de software crea algo de lo que está orgulloso, y se enfrentará a cualquiera que intente sacarle defectos.
    - La prueba, entonces, puede aparecer como un intento de "romper" lo que ha construido el ingeniero de software.
  - Desde el punto de vista del constructor, la prueba se puede considerar (psicológicamente) **destructiva**.
    - Por tanto, el constructor anda con cuidado, diseñando y ejecutando pruebas que demuestren que el programa funciona (Happy Path), en lugar de detectar errores.
    - Desgraciadamente los errores seguirán estando, y si el ingeniero de software no los encuentra, lo hará el cliente o el usuario.

© JMA 2020. All rights reserved

Desarrollador como probador

- A menudo, existen ciertos **malentendidos** que se pueden deducir equivocadamente de la anterior discusión:
  1. El desarrollador del software no debe entrar en el proceso de prueba.
  2. El software debe ser “puesto a salvo” de extraños que puedan probarlo de forma despiadada.
  3. Los encargados de la prueba sólo aparecen en el proyecto cuando comienzan las etapas de prueba.
- Cada una de estas frases es incorrecta.
- El **desarrollador** siempre es responsable de probar las unidades individuales (módulos) del programa, asegurándose de que cada una lleva a cabo la función para la que fue diseñada.
- En muchos casos, también se encargará de la prueba de integración de todos los elementos en la estructura total del sistema.

© JMA 2020. All rights reserved

Grupo independiente de prueba

- Sólo una vez que la arquitectura del software esté completa, entra en juego un **grupo independiente de prueba**, que debe eliminar los problemas inherentes asociados con el hecho de permitir al constructor que pruebe lo que ha construido. Una prueba independiente elimina el conflicto de intereses que, de otro modo, estaría presente.
- El grupo independiente de prueba está formado por especialista en técnicas de pruebas que basan los casos de prueba en las técnicas, la intuición y la experiencia: ¿Qué casos tienen mas probabilidad de encontrar fallos?¿Dónde se han acumulado errores en el pasado?¿Dónde falla normalmente el software?
- En cualquier caso, el desarrollador y el grupo independiente deben trabajar estrechamente a lo largo del proyecto de software para asegurar que se realizan pruebas exhaustivas.
- Mientras se dirige la prueba, el desarrollador debe estar disponible para corregir los errores que se van descubriendo.

© JMA 2020. All rights reserved

Variedad de pruebas

- Las pruebas cubren una gran cantidad de escenarios por lo que tienen una gran variedad de clasificaciones en función de la metodología, enfoque, objetivos, criterios, …
  - Tipo: Estáticas y Dinámicas
  - Información: Pruebas de defectos y Pruebas estadísticas
  - Nivel: Unitarias, Integración, Sistema, Aceptación
  - Proceso: Pruebas manuales o automatizadas
  - Características de calidad: Pruebas funcionales y no funcionales (rendimiento, estructurales, de mantenimiento, seguridad, …)
  - Enfoque: Basadas en la especificación, en la estructura o en la experiencia.
  - Estrategias: Pruebas de humo, progresión, regresión, aprendizaje, exploratorias, …

© JMA 2020. All rights reserved

Pruebas estáticas

- Las pruebas estáticas, también conocidas como revisión, son pruebas que examinan productos (como especificaciones de requisitos, manuales o código fuente) sin que se ejecuten programas.
- Esto se puede hacer sin que el objeto de prueba se esté ejecutando, por ejemplo, revisando requisitos o documentos de diseño, código fuente, manuales de usuario, planes de proyecto y prueba y casos de prueba.
- En los modelos de entrega de TI de alto rendimiento, como Scrum y DevOps, una parte importante de las actividades de prueba estáticas se realiza durante el refinamiento de las historias de los usuarios, por lo que no se implementa como una actividad separada, pero aún requiere un enfoque específico. Además, se realiza un análisis estático del código para cumplir con los estándares de codificación.
- Las pruebas estáticas estarán respaldadas por herramientas tanto como sea posible. De hecho, hoy en día existen herramientas que no sólo hacen análisis estático sino que también son capaces de refactorizar el código basándose en patrones de refactorización estándar. Aún así, en general, las pruebas estáticas son una combinación de inteligencia humana y poder de herramientas.

© JMA 2020. All rights reserved

Pruebas dinámicas

- Las pruebas dinámicas son pruebas mediante la ejecución del objeto de prueba, es decir, la ejecución de una aplicación.
- Cuando ejecutamos una prueba, en otras palabras, ejecutamos el objeto de la prueba (que puede variar desde un único módulo de programa hasta un proceso de negocio completo de extremo a extremo en múltiples organizaciones), eso es lo que llamamos prueba dinámica.
- Las pruebas dinámicas se pueden realizar manualmente, pero para muchas variedades de pruebas, se recomienda encarecidamente el soporte de herramientas de automatización de pruebas.

© JMA 2020. All rights reserved

Prueba de defectos

- La **prueba de defectos** intenta incluir áreas donde el programa no está de acuerdo con sus especificaciones.
- Las pruebas se diseñan para revelar la presencia de defectos en el sistema que se manifiestan en forma de fallos.
- Los fallos demuestran la existencia de defectos que han sido ocasionados por errores.
- Cuando se han encontrado defectos en un programa, éstos deben ser localizados y eliminados. A este proceso se le denomina **depuración**.

© JMA 2020. All rights reserved

Prueba estadística

- La **prueba estadística** se puede utilizar para probar el rendimiento del programa y su confiabilidad.
- Las pruebas se diseñan para reflejar la frecuencia de entradas reales de usuario.
- Después de ejecutar las pruebas, se puede hacer una estimación de la confiabilidad operacional del sistema.
- El rendimiento del programa se puede juzgar midiendo la ejecución de las pruebas estadísticas.

© JMA 2020. All rights reserved

Niveles de pruebas

- **Pruebas Unitarias o de Componentes**: verifican la funcionalidad y estructura de cada componente individualmente, una vez que ha sido codificado.
- **Pruebas de Integración**: verifican el correcto ensamblaje entre los distintos componentes una vez que han sido probados unitariamente, con el fin de comprobar que interactúan correctamente a través de sus interfaces, cubren la funcionalidad establecida y se ajustan a los requisitos no funcionales especificados en las verificaciones correspondientes.
- **Pruebas del Sistema**: ejercitan profundamente el sistema comprobando la integración del sistema de información globalmente, verificando el funcionamiento correcto de las interfaces entre los distintos subsistemas que lo componen y con el resto de sistemas de información con los que se comunica.
- **Pruebas de Aceptación**: validan que un sistema cumple con el funcionamiento esperado y permitir al usuario de dicho sistema, que determine su aceptación desde el punto de vista de su funcionalidad y rendimiento.

© JMA 2020. All rights reserved

Pruebas Unitarias

- Las pruebas unitarias tienen como objetivo verificar la funcionalidad y estructura de cada componente individualmente, una vez que ha sido codificado.
- Con las pruebas unitarias verificas el diseño de los programas, vigilando que no se producen errores y que el resultado de los programas es el esperado.
- Estas pruebas las efectúa normalmente la misma persona que codifica o modifica el componente y que, también normalmente, genera un juego de ensayo para probar y depurar las condiciones de prueba.
- Las pruebas unitarias constituyen la prueba inicial de un sistema y las demás pruebas deben apoyarse sobre ellas.

© JMA 2020. All rights reserved

Pruebas de Integración

- Las pruebas de integración te permiten verificar el correcto ensamblaje entre los distintos componentes una vez que han sido probados unitariamente, con el fin de comprobar que interactúan correctamente a través de sus interfaces, tanto internas como externas, cubren la funcionalidad establecida y se ajustan a los requisitos no funcionales especificados en las verificaciones correspondientes.
- Se trata de probar los caminos lógicos del flujo de los datos y mensajes a través de un conjunto de componentes relacionados que definen una cierta funcionalidad.
- En las pruebas de integración examinas las interfaces entre grupos de componentes o subsistemas para asegurar que son llamados cuando es necesario y que los datos o mensajes que se transmiten son los requeridos.
- Debido a que en las pruebas unitarias es necesario crear módulos auxiliares que simulen las acciones de los componentes invocados por el que se está probando, y a que se han de crear componentes "conductores" para establecer las precondiciones necesarias, llamar al componente objeto de la prueba y examinar los resultados de la prueba, a menudo se combinan los tipos de prueba unitarias y de integración.

© JMA 2020. All rights reserved

Pruebas del Sistema

- **Pruebas funcionales**: dirigidas a asegurar que el sistema de información realiza correctamente todas las funciones que se han detallado en las especificaciones dadas por el usuario del sistema.
- **Pruebas de humo:** son un conjunto de pruebas aplicadas a cada nueva versión, su objetivo es validar que las funcionalidades básicas de la versión se cumplen según lo especificado. Impiden la ejecución el plan de pruebas si detectan grandes inestabilidades o si elementos clave faltan o son defectuosos.
- **Pruebas de comunicaciones:** determinan que las interfaces entre los componentes del sistema funcionan adecuadamente, tanto a través de dispositivos remotos, como locales. Asimismo, se han de probar las interfaces hombre-máquina.
- **Pruebas de rendimiento:** consisten en determinar que los tiempos de respuesta están dentro de los intervalos establecidos en las especificaciones del sistema.
- **Pruebas de volumen:** consisten en examinar el funcionamiento del sistema cuando está trabajando con grandes volúmenes de datos, simulando las cargas de trabajo esperadas.
- **Pruebas de sobrecarga o estrés:** consisten en comprobar el funcionamiento del sistema en el umbral límite de los recursos, sometiéndole a cargas masivas. El objetivo es establecer los puntos extremos en los cuales el sistema empieza a operar por debajo de los requisitos establecidos.

© JMA 2020. All rights reserved

Pruebas del Sistema

- **Pruebas de disponibilidad de datos:** consisten en demostrar que el sistema puede recuperarse ante fallos, tanto de equipo físico como lógico, sin comprometer la integridad de los datos.
- **Pruebas de usabilidad:** consisten en comprobar la adaptabilidad del sistema a las necesidades de los usuarios, tanto para asegurar que se acomoda a su modo habitual de trabajo, como para determinar las facilidades que aporta al introducir datos en el sistema y obtener los resultados.
- **Pruebas extremo a extremo (e2e):** consisten en interactuar con la aplicación como un usuario regular lo haría, cliente-servidor, y evaluando las respuestas para el comportamiento esperado.
- **Pruebas de configuración:** consisten en comprobar todos y cada uno de los dispositivos, en sus propiedades mínimo y máximo posibles.
- **Pruebas de operación:** consisten en comprobar la correcta implementación de los procedimientos de operación, incluyendo la planificación y control de trabajos, arranque y re-arranque del sistema, etc.
- **Pruebas de entorno:** consisten en verificar las interacciones del sistema con otros sistemas dentro del mismo entorno.
- **Pruebas de seguridad:** consisten en verificar los mecanismos de control de acceso al sistema para evitar alteraciones indebidas en los datos.

© JMA 2020. All rights reserved

Pruebas de Aceptación

- El objetivo de las pruebas de aceptación es validar que un sistema cumple con el funcionamiento esperado y permitir al usuario de dicho sistema, que determine su aceptación desde el punto de vista de su funcionalidad y rendimiento.
- Las pruebas de aceptación son preparadas por el usuario del sistema y el equipo de desarrollo, aunque la ejecución y aprobación final corresponde al usuario.
- Estas pruebas van dirigidas a comprobar que el sistema cumple los requisitos de funcionamiento esperado recogidos en el catálogo de requisitos y en los criterios de aceptación del sistema de información, y conseguir la aceptación final del sistema por parte del usuario.

© JMA 2020. All rights reserved

Pruebas de Aceptación

- Previamente a la realización de las pruebas, el responsable de usuarios revisa los criterios de aceptación que se especificaron previamente en el plan de pruebas del sistema y dirige las pruebas de aceptación final.
- La validación del sistema se consigue mediante la realización de pruebas de caja negra que demuestran la conformidad con los requisitos y que se recogen en el plan de pruebas, el cual define las verificaciones a realizar y los casos de prueba.
- Dicho plan está diseñado para asegurar que se satisfacen todos los requisitos funcionales especificados por el usuario teniendo en cuenta, a su vez, los requisitos no funcionales relacionados con el rendimiento, seguridad de acceso al sistema, a los datos y procesos, así como a los distintos recursos del sistema.
- La formalidad de estas pruebas dependerá en mayor o menor medida de cada organización, y vendrá dada por la criticidad del sistema, el número de usuarios implicados en las mismas y el tiempo del que se disponga para llevarlas cabo, entre otros.

© JMA 2020. All rights reserved

Pruebas alfa y beta

- Las pruebas alfa y beta son la alternativa a las pruebas de aceptación en el software comercial de distribución masiva (COTS). Las pruebas alfa y beta suelen ser utilizadas por los desarrolladores de COTS que desean obtener retroalimentación de los usuarios, clientes y/u operadores existentes o potenciales antes de que el producto de software sea puesto en el mercado. Uno de los objetivos de las pruebas alfa y beta es generar la confianza de que se pueden utilizar el sistema en condiciones normales y cotidianas, con la mínima dificultad, coste y riesgo.
- Las pruebas alfa son pruebas de software realizadas en las fases iniciales del proyecto cuando el sistema está en desarrollo y cuyo objetivo es asegurar que lo que estamos desarrollando es probablemente correcto y útil para el cliente. Si las pruebas alfa devuelven buenos comentarios positivos entonces podríamos seguir por esa vía. Si devolvieran resultados muy negativos tendríamos que replantear el problema para adaptarse mejor a los requerimientos.
- Las pruebas beta son las pruebas de software que se realizan cuando el sistema está completado y, en teoría, es correcto, antes de pasar a producción. Dado que no es viable realizar pruebas exhaustivas, siempre habrá fallos que no han sido descubiertos por los desarrolladores ni por el equipo de pruebas. Las pruebas beta son pruebas para localizar esos problemas no detectados y poder corregirlos antes de liberar la versión definitivas; la prueba debería ser realizada por usuarios finales.

© JMA 2020. All rights reserved

Pruebas de Regresión

- El objetivo de las pruebas de regresión es eliminar el efecto onda, es decir, comprobar que los cambios sobre un componente de un sistema de información, no introducen un comportamiento no deseado o errores adicionales en otros componentes no modificados.
- Las pruebas de regresión se deben llevar a cabo cada vez que se hace un cambio en el sistema, tanto para corregir un error como para realizar una mejora.
- No es suficiente probar sólo los componentes modificados o añadidos, o las funciones que en ellos se realizan (también conocidas como **pruebas de progresión o confirmación**), sino que también es necesario controlar que las modificaciones no produzcan efectos secundarios negativos sobre el mismo u otros componentes.
- Normalmente, este tipo de pruebas implica la repetición de las pruebas que ya se han realizado previamente, con el fin de asegurar que no se introducen errores que puedan comprometer el funcionamiento de otros componentes que no han sido modificados y confirmar que el sistema funciona correctamente una vez realizados los cambios.

© JMA 2020. All rights reserved

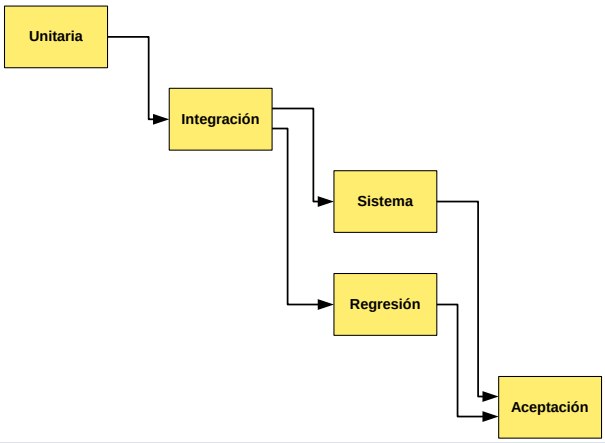
Pruebas de Regresión

- Las pruebas de regresión **pueden incluir**:
  - La repetición de los casos de pruebas que se han realizado anteriormente y están directamente relacionados con la parte del sistema modificada.
  - La revisión de los procedimientos manuales preparados antes del cambio, para asegurar que permanecen correctamente.
  - La obtención impresa del diccionario de datos de forma que se compruebe que los elementos de datos que han sufrido algún cambio son correctos.
- El **responsable** de realizar las pruebas de regresión será el equipo de desarrollo junto al técnico de mantenimiento, quién a su vez, será responsable de especificar el plan de pruebas de regresión y de evaluar los resultados de dichas pruebas.

© JMA 2020. All rights reserved

Niveles de pruebas y orden de ejecución.

- De tal forma que la secuencia de pruebas es:



© JMA 2020. All rights reserved

Smoke Testing y Sanity Testing

- Un smoke test (prueba de humo) es una prueba superficial realizada para asegurar de que una nueva distribución puede realizar la funcionalidad básica sin encontrar errores críticos. El objetivo principal de un smoke test, subconjunto de pruebas rápidas y automatizadas, es determinar si la compilación del software desplegado es suficientemente estable antes de realizar pruebas más exhaustivas y costosas.
- Un sanity test (prueba de cordura o sanidad) es una prueba superficial realizada para asegurar para comprobar si un cambio o nueva funcionalidad de una nueva distribución funciona correctamente. Un sanity test es un tipo de pruebas de regresión más enfocado y específico para validar los componentes mejorados en lugar del producto completo.
- Las pruebas de sanidad se ejecutan después de haber superado las pruebas de humo y antes de acometer las pruebas de regresión.
- Las pruebas de humo y sanidad son parte de las estrategias de pruebas y ahorran mucho tiempo y esfuerzo porque evitan que los equipos de control de calidad pierdan tiempo en pruebas más profundas antes de asegurarse de que las funciones básicas de la compilación del software funcionan como deberían.

© JMA 2020. All rights reserved

Pruebas de instantáneas y de regresión visual

- Las pruebas de instantáneas (snapshot test) es una poderosa técnica que permite comparar la salida del código con un valor almacenado previamente, también conocido como instantánea (snapshot) o, coloquialmente, una foto. De esta manera, de forma muy simple, se crea un bucle de retroalimentación rápida para detectar fácilmente cualquier cambio o regresión en el código sin tener que escribir afirmaciones complejas.
- Las pruebas de instantáneas generalmente almacenan una salida generada previamente como un archivo separado, donde se ha verificado manualmente que es le resultado esperado (approved). Cuando se vuelve a ejecutar el caso de prueba, compara la nueva salida con el archivo almacenado: si coincide con la salida existente, la prueba pasará, de lo contrario la prueba fallará. Se debe actualizar la instantánea si ha quedado obsoleta.
- Las pruebas de instantáneas y las pruebas de regresión visual son dos formas distintas de probar las IU y tienen diferentes propósitos. Las herramientas de pruebas de regresión visual toman capturas de pantalla de las páginas web y comparan las imágenes resultantes píxel por píxel. Con las pruebas de instantáneas, los valores se serializan, se almacenan en archivos de texto y se comparan utilizando un algoritmo de diferencias. Estas pruebas de instantáneas y de regresión visual son muy fáciles y rápidas de elaborar, detectan cambios inesperados y actúan como documentación visual. Aunque cubren un amplio espectro, al depender de la entrada, son pruebas frágiles y suelen ser poco concluyentes.

© JMA 2020. All rights reserved

Pruebas de extremo a extremo (e2e)

- Algunas pruebas deben tener una vista de pájaro de alto nivel de la aplicación. Simulan a un usuario interactuando con la aplicación: navegando a una dirección, leyendo texto, haciendo clic en un enlace o botón, llenando un formulario, moviendo el mouse o escribiendo en el teclado. Estas pruebas generan las expectativas sobre lo que el usuario ve y lee en el navegador.
- Desde la perspectiva del usuario, no importa que la aplicación como esté implementada. Los detalles técnicos como la estructura interna de su código no son relevantes. No hay distinción entre front-end y back-end, entre partes del código. Se prueba la experiencia completa.
- Estas pruebas se denominan pruebas de extremo a extremo (E2E) ya que integran todas las partes de la aplicación desde un extremo (el usuario) hasta el otro extremo (los rincones más oscuros del back-end). Las pruebas de extremo a extremo también forman la parte automatizada de las pruebas de aceptación, ya que indican si la aplicación funciona para el usuario.

© JMA 2020. All rights reserved

Prueba exploratoria

- Incluso los esfuerzos de automatización de pruebas más diligentes no son perfectos. A veces se pierden ciertos casos extremos en sus pruebas automatizadas. A veces es casi imposible detectar un error en particular escribiendo una prueba unitaria. Ciertos problemas de calidad ni siquiera se hacen evidentes en las pruebas automatizadas (como en el diseño o la usabilidad).
- Las limitaciones de la automatización de pruebas son:
  - No todas las pruebas manuales se pueden automatizar y no son un sustituto de las pruebas exploratorias.
  - La automatización sólo puede comprobar resultados interpretables por la máquina.
  - La automatización sólo puede comprobar los resultados reales que pueden ser verificados por un oráculo de prueba automatizado.
- Las pruebas exploratorias es un enfoque de prueba manual que enfatiza la libertad y creatividad de la persona que prueba para detectar problemas de calidad en un sistema en ejecución.
  - Simplemente tomate un tiempo en un horario regular, arremángate e intenta romper la aplicación.
  - Usa una mentalidad destructiva y encuentra formas de provocar problemas y errores en la aplicación.
  - Ten en cuenta los errores, los problemas de diseño, los tiempos de respuesta lentos, los mensajes de error faltantes o engañosos y, en general, todo lo que pueda molestarte como usuario de una aplicación.
  - Documenta todo lo que encuentre para más adelante.
- La buena noticia es que se puede automatizar la mayoría de los hallazgos con pruebas automatizadas. Escribir pruebas automatizadas para los errores que se detectan asegura que no habrá regresiones a ese error en el futuro. Además, ayuda a reducir la causa raíz de ese problema durante la corrección de errores.

© JMA 2020. All rights reserved

Automatización de pruebas

- Las pruebas exploratorias (manuales) son muy costosas y difícilmente repetibles, por lo que se impone una estrategia de automatización.
- Las pruebas funcionales de usuario final son caras de ejecutar que, por ejemplo, requieren abrir un navegador e interactuar con el. Además, normalmente requieren que una infraestructura considerable este disponible para estas ejecutarse de manera efectiva. Es una buena regla preguntarse siempre si lo que se quiere probar se puede hacer usando enfoques de prueba más livianos como las pruebas unitarias o con un enfoque de bajo nivel.
- Los analizadores de código son herramientas de fuerza bruta que realizan la lectura del código fuente y devuelve observaciones o puntos en los que tu código puede mejorarse desde la percepción de buenas prácticas de programación y código limpio.

© JMA 2020. All rights reserved

Aprender con pruebas unitarias

- La incorporación de código de tercero es complicado, hay que aprenderlo primero e integrarlo después. Hacer las dos cosa a la vez es el doble de complicado.
- Utilizar pruebas unitarias en el proceso de aprendizaje (pruebas de aprendizaje según Jim Newkirk) aporta importantes ventajas:
  - La inmediatez de las pruebas unitarias y sus entornos.
  - Realizar “pruebas de concepto” para comprobar si el comportamiento se corresponde con lo que hemos entendido, permitiéndonos clarificarlo.
  - Experimentar para encontrar los mejores escenarios de integración.
  - Permite saber si un fallo es nuestro, de la librería o del uso inadecuado de la librería.

© JMA 2020. All rights reserved

Pruebas de aprendizaje

- Las pruebas de aprendizaje no suponen un coste adicional, es parte del coste de aprendizaje que, en todo caso, lo minoran.
- Es mas, las pruebas de aprendizaje son rentables. Ante la aparición de nuevas versiones del código ajeno, ejecutar la batería de pruebas de aprendizaje valida el impacto de la adopción de la nueva versión: detecta cambios relevantes, efectos negativos en las integraciones, ...
- Las pruebas de aprendizaje no sustituyen al conjunto de pruebas que respaldan los limites establecidos.

© JMA 2020. All rights reserved

Cobertura

- La cobertura es la relación entre lo que se puede probar y lo que realmente se prueba con el equipo de prueba.
- La cobertura es un concepto útil que ayuda a los evaluadores con preguntas complejas, como:
  - Dado que es imposible probarlo todo y, por lo tanto, nos vemos obligados a probar solo un subconjunto de todas las posibilidades, ¿cuál es el mejor subconjunto?
  - ¿Cuál es la diferencia entre "pruebas elementales" y "pruebas exhaustivas"? ¿Qué significa concretamente para los casos de prueba que necesitamos para esto?
- La cobertura tiene mucho que ver con el deseo de encontrar la mayor cantidad de defectos posibles con el menor número de casos de prueba posibles.
- En lugar de simplemente probar "cualquier" subconjunto de opciones, nuestro objetivo es compilar un conjunto de casos de prueba que creen el mayor cambio posible para encontrar los defectos que existen.

© JMA 2020. All rights reserved

Métricas de código

- La mayor complejidad de las aplicaciones de software moderno también aumenta la dificultad de hacer que el código confiable y fácil de mantener.
- Las métricas de código son un conjunto de medidas de software que proporcionan a los programadores una mejor visión del código que están desarrollando.
- Aprovechando las ventajas de las métricas del código, los desarrolladores pueden entender qué tipos o métodos deberían rehacerse o hacer más pruebas.
- Los equipos de desarrollo pueden identificar los posibles riesgos, comprender el estado actual de un proyecto y realizar un seguimiento del progreso durante el desarrollo de software. Los desarrolladores pueden generar datos de métricas de código con que medir la complejidad y el mantenimiento del código administrado.
- Se insiste mucho en que la cobertura de código de test unitarios de los proyectos sea lo más alta posible, pero es evidente que cantidad (de test, en este caso) no siempre implica calidad, la calidad no se puede medir "al peso", y es la calidad lo que realmente importa. El criterio de la cobertura de código se basa en la suposición que a mayor cantidad de código ejecutada por las pruebas, menor la probabilidad de que el código presente defectos.

© JMA 2020. All rights reserved

Calidad de las pruebas

- La cobertura del código a menudo se considera la métrica estándar de oro para comprender la calidad de las pruebas, y eso es algo desafortunado.
- Un problema aún más insidioso con la cobertura del código es que, al igual que otras métricas, rápidamente se convierte en un objetivo en sí mismo.
- La cobertura de prueba tradicional (líneas, instrucciones, ramas, etc.) solo mide la proporción del código qué participa en las pruebas. No comprueba si las pruebas son realmente capaces de detectar fallos en el código ejecutado, solo pueden identificar el código que no se ha probado. Los ejemplos más extremos del problema son las pruebas sin afirmaciones (poco comunes en la mayoría de los casos). Mucho más común es el código que solo se prueba parcialmente, cubrir todo los caminos no implica ejercitar todas las clases de equivalencia y valores límite.
- La calidad de las pruebas también debe ser puesta a prueba: No serviría de tener una cobertura del 100% en test unitarios, si no son capaces de detectar y prevenir problemas en el código. La herramienta que testea la calidad de los test unitarios son los test de mutaciones: Es un test de los test.

© JMA 2020. All rights reserved

Pruebas de mutaciones

- Los pruebas de mutaciones son las pruebas de las pruebas unitarias y el objetivo es tener una idea de la calidad de las pruebas en cuanto a fiabilidad.
- Su funcionamiento es relativamente sencillo: la herramienta que se utilice debe generar pequeños cambios en el código fuente. A estos pequeños cambios se les conoce como mutaciones y crean mutantes.
- Una vez creados los mutantes, se lanzan todos los tests:
  - Si los test unitarios fallan, es que han sido capaces de detectar ese cambio de código. En este caso el mutante se considera eliminado.
  - Si, por el contrario, los test unitarios pasan, el mutante sobrevive y la fiabilidad (y calidad) de los tests unitarios queda en entredicho.
- Los test de mutaciones presentan informes del porcentaje de mutantes detectados: cuanto más se acerque este porcentaje al 100%, mayor será la calidad de nuestros test unitarios.

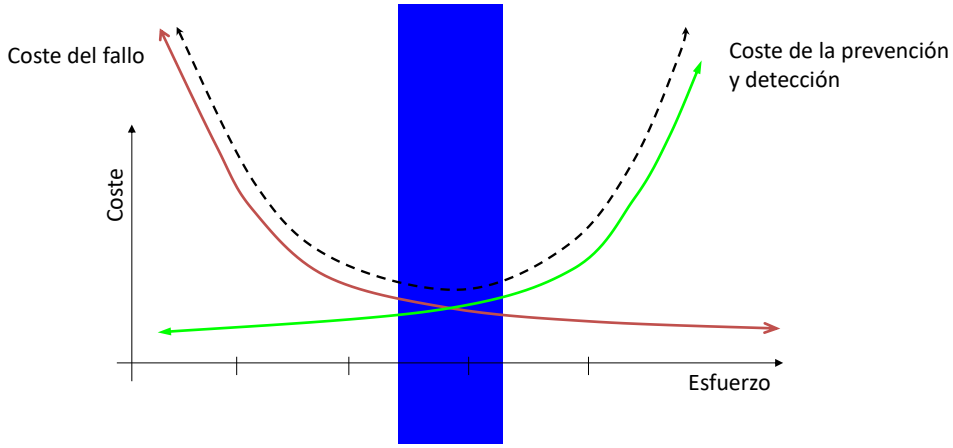
© JMA 2020. All rights reserved

Cantidad correcta de pruebas

- Hay un feroz debate que gira en torno a la cantidad correcta de pruebas. Muy pocas pruebas son un problema: las funciones no se especifican correctamente, los errores pasan desapercibidos, ocurren regresiones. Pero demasiadas pruebas consumen tiempo de desarrollo y recursos, no generan ganancias adicionales y ralentizan el desarrollo a largo plazo.
- No es cuestión de hacer muchas pruebas, de hecho, hay que hacer las imprescindibles pero seleccionando buenas pruebas: las que mayor probabilidad tengan de detectar un error y cubran el mayor numero de escenarios.
- Las pruebas difieren en su valor y calidad. Algunas pruebas son más significativas que otras. Los recursos son limitados.
- Esto significa que la calidad de las pruebas es más importante que su cantidad.

© JMA 2020. All rights reserved

Las pruebas tienen un coste



© JMA 2020. All rights reserved

Pirámide de pruebas

© JMA 2020. All rights reserved

https://martinfowler.com/bliki/TestPyramid.html

Comparativa

Nivel	Extremo a extremo	Integración	Unitarias
Cobertura	completa	grande	pequeña
Rendimiento	lenta	rápida	muy rápida
Fiabilidad	menos confiable	confiable	mas fiable
Aislar fallos	complicado	justo	fácil
Coste	muy alto	mediano	muy bajo
Simula el usuario real	sí	no	no

© JMA 2020. All rights reserved

Análisis estático con herramientas

- El objetivo del análisis estático es detectar defectos en el código fuente y en los modelos de software.
- El análisis estático se realiza sin que la herramienta llegue a ejecutar el software objeto de la revisión, como ocurre en las pruebas dinámicas, centrándose mas en como está escrito el código que en como se ejecuta el código.
- El análisis estático permite identificar defectos difíciles de encontrar mediante pruebas dinámicas.
- Al igual que con las revisiones, el análisis estático encuentra defectos en lugar de fallos.
- Las herramientas de análisis estático analizan el código del programa (por ejemplo, el flujo de control y flujo de datos) y las salidas generadas (tales como HTML o XML).
- Algunos de los posibles aspectos que pueden ser comprobados con análisis estático:
  - Reglas, estándares de programación y buenas practicas.
  - Diseño de un programa (análisis de flujo de control).
  - Uso de datos (análisis del flujo de datos).
  - Complejidad de la estructura de un programa (métricas, por ejemplo valor ciclomático).

© JMA 2020. All rights reserved

Valor del análisis estático

- La detección temprana de defectos antes de la ejecución de las pruebas.
- Advertencia temprana sobre aspectos sospechosos del código o del diseño mediante el cálculo de métricas, tales como una medición de alta complejidad.
- Identificación de defectos que no se encuentran fácilmente mediante pruebas dinámicas.
- Detectar dependencias e inconsistencias en modelos de software, como enlaces.
- Mantenibilidad mejorada del código y del diseño.
- Prevención de defectos, si se aprende la lección en la fase de desarrollo.

© JMA 2020. All rights reserved

Defectos habitualmente detectados

- Referenciar una variable con un valor indefinido.
- Interfaces inconsistentes entre módulos y componentes.
- Variables que no se utilizan o que se declaran de forma incorrecta.
- Código inaccesible (muerto).
- Ausencia de lógica o lógica errónea (posibles bucles infinitos).
- Construcciones demasiado complicadas.
- Infracciones de los estándares de programación.
- Vulnerabilidad de seguridad.
- Infracciones de sintaxis del código y modelos de software.

© JMA 2020. All rights reserved

Ejecución del análisis estático

- Las herramientas de análisis estático generalmente las utilizan los desarrolladores (cotejar con las reglas predefinidas o estándares de programación) antes y durante las pruebas unitarias y de integración, o durante la comprobación del código.
- Las herramientas de análisis estático pueden producir un gran número de mensajes de advertencias que deben ser bien gestionados para permitir el uso más efectivo de la herramienta.
- Los compiladores pueden constituir un soporte para los análisis estáticos, incluyendo el cálculo de métricas.
- El Compilador detecta errores sintácticos en el código fuente de un programa, crea datos de referencia del programa (por ejemplo lista de referencia cruzada, llamada jerárquica, tabla de símbolos), comprueba la consistencia entre los tipos de variables y detecta variables no declaradas y código inaccesible (código muerto).
- El Analizador trata aspectos adicionales tales como: Convenciones y estándares, Métricas de complejidad y Acoplamiento de objetos.

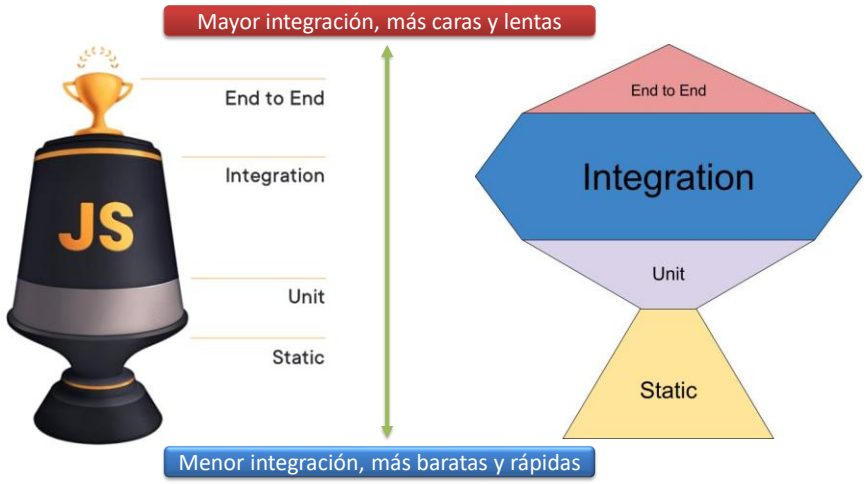
© JMA 2020. All rights reserved

El Trofeo de Pruebas

- Testing Trophy es un método de pruebas propuesto por Kent C. Dodds *para aplicaciones web*. Se trata de escribir suficientes pruebas, no muchas, pero si las pruebas correctas: proporciona la mejor combinación de velocidad, costo y confiabilidad.
- Se superpondrán las siguientes técnicas:
  - Usar un sistema de captura de errores de tipo, estilo y de formato utilizando linters, formateadores de errores y verificadores de tipo (ESLint, SonarQube, …).
  - Escribir pruebas unitarias efectivas que apunten solo al comportamiento crítico y la funcionalidad de la aplicación.
  - Desarrollar pruebas de integración para auditar la aplicación de manera integral y asegurarse de que todo funcione correctamente en armonía.
  - Crear pruebas funcionales de extremo a extremo (e2e) para pruebas de interacción automatizadas de las rutas críticas y los flujos de trabajo más utilizados por los usuarios.

© JMA 2020. All rights reserved

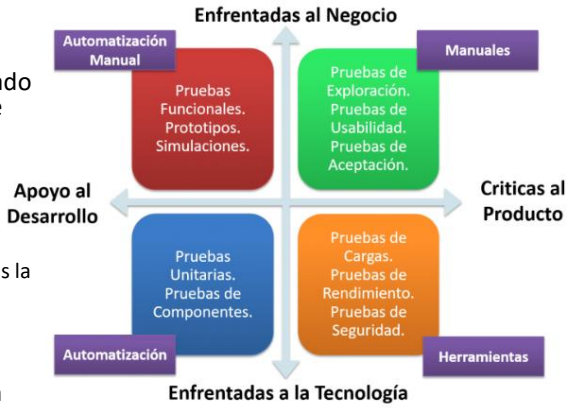
Testing Trophy



© JMA 2020. All rights reserved

Cuadrante de Pruebas

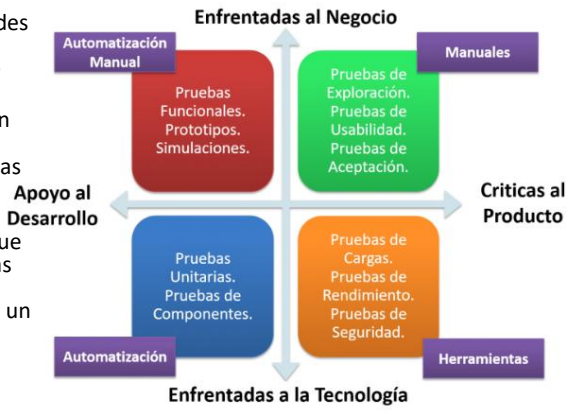
- El cuadrante de prueba divide todo el proceso de prueba en 4 partes. Esto hace que el proceso de prueba sea fácil de entender.
- El cuadrante de pruebas fue originalmente publicado por Brian Marick en 2003 como parte de una serie de artículos sobre Testeo Ágil y originalmente fue conocido como “Matriz de pruebas de Marick” (Marick Test Matrix”).
- Hay cuatro cuadrantes de prueba que son el resultado de dos ejes.
  - El eje vertical muestra la perspectiva tecnológica versus la perspectiva empresarial.
  - El eje horizontal trata sobre la orientación durante la creación de un producto versus la evaluación del producto una vez que está listo.
- Hay que asegurarse de que se organicen y realicen pruebas para los cuatro cuadrantes.



© JMA 2020. All rights reserved

Cuadrante de Pruebas

- El primer cuadrante (**abajo a la izquierda**) es donde la atención se centra en la tecnología y donde las actividades de prueba guían al equipo en la creación del producto. Suele implicar TDD para las unidades y la integración de las unidades. Las prueba deberían automatizarse.
- El segundo cuadrante (**arriba a la izquierda**) se centra en el negocio y también se centra en guiar al equipo. Esto implica, por ejemplo, el desarrollo impulsado por pruebas de aceptación (ATDD), donde se prueba un proceso de negocio, preferiblemente automatizadas.
- El tercer cuadrante (**arriba a la derecha**) tiene un enfoque empresarial en la evaluación del producto. Estas pruebas se ejecutan principalmente de forma manual, ya sea siguiendo escenarios de prueba preparados o mediante un enfoque exploratorio.
- El cuarto cuadrante (**abajo a la derecha**) se centra en la tecnología y evalúa los aspectos no funcionales de un producto que sólo pueden validarse una vez que el producto esté listo. Las pruebas de rendimiento dinámico son un buen ejemplo de ello.



© JMA 2020. All rights reserved

DISEÑO DE PRUEBAS

© JMA 2020. All rights reserved

Diseñar la prueba

- Para diseñar la prueba empiezas por identificar y describir los casos de prueba de cada componente.
- La selección de las técnicas de pruebas depende factores adicionales como pueden ser requisitos contractuales o normativos, documentación disponible, tiempo, presupuesto, conocimientos, experiencia, …
- Cuando dispongas de los casos de prueba, identificas y estructuras los procedimientos de prueba describiendo cómo ejecutar los casos de prueba.

© JMA 2020. All rights reserved

Terminología

- Un **caso de prueba** especifica una forma de probar el sistema, incluyendo la entrada con la que se ha de probar, los resultados que se esperan obtener y las condiciones bajo las que ha de probarse. La **base de prueba** es el punto de partida del caso de prueba (requisito, criterio de aceptación, …) y el **objeto de prueba** es el destinatario de la prueba (componente, sistema, documento, …).
- Un **procedimiento de prueba** (o caso de prueba lógico) especifica cómo realizar uno o varios casos de prueba. El procedimiento documenta los pasos que deben darse para cada uno de los casos de prueba. Los procedimientos de prueba pueden reutilizarse para varios casos de prueba similares. Así mismo, un caso de prueba puede estar incluido en varios procedimientos de prueba.
- Un **componente de prueba** (o caso de prueba físico) automatiza uno o varios procedimientos de prueba o partes de éstos. Los componentes de prueba se diseñan e implementan de forma específica para proporcionar las entradas, controlar la ejecución e informar de la salida de los elementos a probar.
- Una **suite de pruebas** es un conjunto organizado de pruebas que se ejecutan de manera automatizada para verificar un aspecto de la funcionalidad y calidad de un sistema, aplicación o componente.
- Un **ejecutor de pruebas** (o test runner) es la herramienta encargada de descubrir y ejecutar los componentes de pruebas y genera un informe para exportar e integrar el resultado de las prueba.

© JMA 2020. All rights reserved

Pruebas Unitarias

- Existen dos **enfoques** principales para el diseño de casos de prueba:
  - **Enfoque estructural o de caja blanca**. Se verifica la estructura interna del componente con independencia de la funcionalidad establecida para el mismo.

    Por tanto, no se comprueba la corrección de los resultados, sólo si éstos se producen. Ejemplos de este tipo de pruebas pueden ser ejecutar todas las instrucciones del programa, localizar código no usado, comprobar los caminos lógicos del programa, etc.
  - **Enfoque funcional o de caja negra**. Se comprueba el correcto funcionamiento de los componentes del sistema de información, analizando las entradas y salidas y verificando que el resultado es el esperado. Se consideran exclusivamente las entradas y salidas del sistema sin preocuparse por la estructura interna del mismo.
- El enfoque que suele adoptarse para una prueba unitaria está claramente orientado al diseño de casos de caja blanca, aunque se complemente con caja negra.

© JMA 2020. All rights reserved

Pruebas Unitarias

- Los **pasos necesarios** para llevar a cabo las pruebas unitarias son los siguientes:
  - **Ejecutar todos los casos de prueba** asociados a cada verificación establecida en el plan de pruebas, registrando su resultado. Los casos de prueba deben contemplar tanto las condiciones válidas y esperadas como las inválidas e inesperadas.
  - **Corregir los errores o defectos encontrados y repetir las pruebas que los detectaron**. Si se considera necesario, debido a su implicación o importancia, se repetirán otros casos de prueba ya realizados con anterioridad.

© JMA 2020. All rights reserved

Pruebas Unitarias

- La prueba unitaria se da por finalizada cuando se hayan realizado todas las verificaciones establecidas y no se encuentre ningún defecto, o bien se determine su suspensión.
- Al finalizar las pruebas, obtienes las **métricas de calidad del componente** y las contrastas con las existentes antes de la modificación:
  - Número ciclomático.
  - Cobertura de código.
  - Porcentaje de comentarios.
  - Defectos hallados contra especificaciones o estándares.
  - Rendimientos.

© JMA 2020. All rights reserved

Características de una buena prueba unitaria

- El principio FIRST fue definido por Robert Cecil Martin en su libro Clean Code. Este autor, entre otras muchas cosas, es conocido por ser uno de los escritores del Agile Manifesto, escrito hace más de 15 años y que a día de hoy se sigue teniendo muy en cuenta a la hora de desarrollar software.
  - Fast: Los tests deben ser rápidos, del orden de milisegundos, hay cientos de tests en un proyecto.
  - Isolated/Independent (Aislado/Independiente). Los tests no deben depender del entorno ni de ejecuciones de tests anteriores.
  - Repeatable. Los tests deben ser repetibles y ante la misma entrada de datos, los mismos resultados.
  - Self-Validating. Los tests tienen que ser autovalidados, es decir, NO debe de existir la intervención humana en la validación
  - Thorough and Timely (Completo y oportuno). Los tests deben de cubrir el escenario propuesto, no el 100% del código, y se han de realizar en el momento oportuno

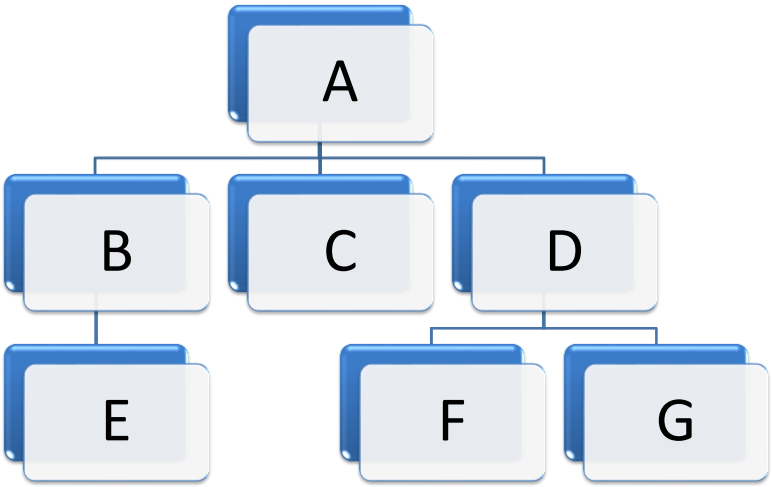
© JMA 2020. All rights reserved

Pruebas de Integración

- Los **tipos fundamentales de integración** son los siguientes:
  - **Integración incremental:** combinas el siguiente componente que debes probar con el conjunto de componentes que ya están probados y vas incrementando progresivamente el número de componentes a probar.
  - **Integración no incremental:** pruebas cada componente por separado y, luego, los integras todos de una vez realizando las pruebas pertinentes. Este tipo de integración se denomina también Big-Bang (gran explosión).
- Con el tipo de prueba incremental lo más probable es que los problemas te surjan al incorporar un nuevo componente o un grupo de componentes al previamente probado. Los problemas serán debidos a este último o a las interfaces entre éste y los otros componentes.

© JMA 2020. All rights reserved

Pruebas de Integración



© JMA 2020. All rights reserved

Estrategias de integración

- **De arriba a abajo (top-down):** el primer componente que se desarrolla y prueba es el primero de la jerarquía (A).
  - Los componentes de nivel más bajo se sustituyen por componentes auxiliares o dobles de pueba, para simular a los componentes invocados. En este caso no son necesarios componentes conductores.
  - Una de las ventajas de aplicar esta estrategia es que las interfaces entre los distintos componentes se prueban en una fase temprana y con frecuencia.
- **De abajo a arriba (bottom-up):** en este caso se crean primero los componentes de más bajo nivel (E, F, G) y se crean componentes conductores para simular a los componentes que los llaman.
  - A continuación se desarrollan los componentes de más alto nivel (B, C, D) y se prueban. Por último dichos componentes se combinan con el que los llama (A). Los componentes auxiliares son necesarios en raras ocasiones.
  - Este tipo de enfoque permite un desarrollo más en paralelo que el enfoque de arriba a abajo, pero presenta mayores dificultades a la hora de planificar y de gestionar.
- **Estrategias combinadas:** A menudo es útil aplicar las estrategias anteriores conjuntamente. De este modo, se desarrollan partes del sistema con un enfoque "top-down", mientras que los componentes más críticos en el nivel más bajo se desarrollan siguiendo un enfoque "bottom-up".
  - En este caso es necesaria una planificación cuidadosa y coordinada de modo que los componentes individuales se "encuentren" en el centro.

© JMA 2020. All rights reserved

Patrones

- Los casos de prueba se pueden estructurar siguiendo diferentes patrones:
  - ARRANGE-ACT-ASSERT: Preparar, Actuar, Afirmar
  - GIVEN-WHEN-THEN: Dado, Cuando, Entonces
  - BUILD-OPERATE-CHECK: Generar, Operar, Comprobar
- Con diferencias conceptuales, todos dividen el proceso en tres fases:
  - Una fase inicial donde montar el escenario de pruebas que hace que el resultado sea predecible.
  - Una fase intermedia donde se realizan las acciones que son el objetivo de la prueba.
  - Una fase final donde se comparan los resultados con el escenario previsto. Pueden tomar la forma de:
    - Aserción: Es una afirmación sobre el resultado que puede ser cierta o no.
    - Expectativa: Es la expresión del resultado esperado que puede cumplirse o no.

© JMA 2020. All rights reserved

Preparación mínima

- La sección de preparación, con la entrada del caso de prueba, debe ser lo más sencilla posible, lo imprescindible para comprobar el comportamiento que se está probando.
- Las pruebas se hacen más resistentes a los cambios futuros en el código base y más cercano al comportamiento de prueba que a la implementación.
- Las pruebas que incluyen más información de la necesaria tienen una mayor posibilidad de incorporar errores en la prueba y pueden hacer confusa su intención. Al escribir pruebas, el usuario quiere centrarse en el comportamiento. El establecimiento de propiedades adicionales en los modelos o el empleo de valores distintos de cero cuando no es necesario solo resta de lo que se quiere probar.

© JMA 2020. All rights reserved

Actuación mínima

- Al escribir las pruebas hay que evitar introducir condiciones lógicas como if, switch, while, for, etc.
- Minimiza la posibilidad de incorporar un error a las pruebas.
- El foco está en el resultado final, en lugar de en los detalles de implementación.
- Al incorporar lógica al conjunto de pruebas, aumenta considerablemente la posibilidad de agregar un error. Cuando se produce un error en una prueba, se quiere saber realmente que algo va mal con el código probado y no en el código que prueba. En caso contrario, restan confianza y las pruebas en las que no se confía no aportan ningún valor.
- El objetivo de la prueba debe ser único, si la lógica en la prueba parece inevitable, denota que el objetivo es múltiple y hay que considerar la posibilidad de dividirla en dos o más pruebas diferentes.

© JMA 2020. All rights reserved

Comprobación mínima

- Al escribir las pruebas, hay que intentar comprobar una única cosa, es decir, incluir solo una aserción por prueba.
  - Si se produce un error en una aserción, no se evalúan las aserciones posteriores.
  - Garantiza que no se estén declarando varios casos en las pruebas.
  - Proporciona la imagen exacta de por qué se producen errores en las pruebas.
- Al incorporar varias aserciones en un caso de prueba, no se garantiza que se ejecuten todas. Es un todas o ninguna, se sabe por cual fallo pero no si el resto también falla o es correcto, proporcionando la imagen parcial.
- Una excepción común a esta regla es cuando la validación cubre varios aspectos. En este caso, suele ser aceptable que haya varias aserciones para asegurarse de que el resultado está en el estado que se espera que esté.
- Los enfoques comunes para usar solo una aserción incluyen:
  - Crear una prueba independiente para cada aserción.
  - Usar pruebas con parámetros.

© JMA 2020. All rights reserved

Las pruebas deben

- Tener un objetivo único y un propósito claro.
- Centrarse en el comportamiento antes que en los detalles de la implementación.
- Empezar por los casos válidos (Happy Path) antes de pasar a los casos inválidos (extremos, limites).
- Tener nombres descriptivos y un código limpio, son parte de la documentación.
- Seguir el patrón AAA o una de sus variaciones.
- Ser breves, simples y eficientes.
- Ser deterministas y repetibles.
- Ser independientes entre si (inicializar el entorno), aisladas (dobles de pruebas), no tener efectos secundarios, no influir en el observado.

© JMA 2020. All rights reserved

Casos de prueba de mala calidad

- Sin aserciones
- No comprueban el resultado completo esperado
- No utilizan las aserciones mas especificas
- Cubren múltiples escenarios
- Complejos, con mucho código, no dejan claro que están probando y son frágiles
- Su código apesta, difíciles de entender y mantener
- Dependen de otros casos de pruebas
- Indeterministas, unas veces fallan y otras no, sin cambiar el código fuente ni la prueba
- Cruzan limites, no respetan los limites de las pruebas

© JMA 2020. All rights reserved

Características de las pruebas valiosa

- **Las pruebas formalizan y documentan los requisitos.**
  - Un conjunto de pruebas es una descripción formal, legible por humanos y máquinas, de cómo debe comportarse el código. Ayuda a los desarrolladores, en la creación, a comprender los requisitos que deben implementar. Ayuda a los desarrolladores, en el mantenimiento, a comprender los desafíos a que tuvieron que enfrentase los creadores.
  - *Una prueba valiosa describe claramente cómo debe comportarse el código de implementación.* La prueba utiliza un lenguaje adecuado para hablar con los desarrolladores y transmitir los requisitos. La prueba enumera los casos conocidos con los que tiene que lidiar la implementación.
- **Las pruebas aseguran que el código implemente los requisitos y no muestre fallos.**
  - Las pruebas aprovechan cada parte del código para encontrar fallos.
  - *Una prueba valiosa cubre los escenarios importantes*: tanto las entradas correctas como las incorrectas, los casos esperados y los casos excepcionales.

© JMA 2020. All rights reserved

Características de las pruebas valiosa

- **Las pruebas ahorran tiempo y dinero.**
  - Las pruebas intentan cortar los problemas de software de raíz. Las pruebas previenen errores antes de que causen un daño real, cuando todavía son manejables y están bajo control.
  - *Una prueba valiosa es rentable.* La prueba previene errores que, en última instancia, podrían inutilizar la aplicación. La prueba es barata de escribir en comparación con el daño potencial que previene.

- **Las pruebas hacen que el cambio sea seguro al evitar las regresiones.**
  - Las pruebas no solo verifican que la implementación actual cumpla con los requisitos. También verifican que el código aún funcione como se esperaba después de los cambios. Con las pruebas automatizadas adecuadas, es menos probable que se rompa accidentalmente. La implementación de nuevas funciones y la refactorización de código es más segura.
  - *Una prueba valiosa falla cuando se cambia o elimina el código esencial.* Las prueba se diseñan para fallar si se cambia el comportamiento dependiente y deberían seguir pasando ante cambios no dependientes.

© JMA 2020. All rights reserved

AUTOMATIZACIÓN DE PRUEBAS

© JMA 2020. All rights reserved

Automatización de la Prueba

- La automatización de la prueba permite ejecutar muchos casos de prueba de forma consistente y repetida en las diferentes versiones del sistema sujeto a prueba (SSP) y/o entornos. Pero la automatización de pruebas es más que un mecanismo para ejecutar un juego de pruebas sin interacción humana. Implica un proceso de diseño de productos de prueba, entre los que se incluyen:
  - Software.
  - Documentación.
  - Casos de prueba.
  - Entornos de prueba
  - Datos de prueba
- Una Solución de Automatización Pruebas (SAP) debe permitir:
  - Implementar casos de prueba automatizados.
  - Monitorizar y controlar la ejecución de pruebas automatizadas.
  - Interpretar, informar y registrar los resultados de pruebas automatizadas.

© JMA 2020. All rights reserved

Objetivos de la automatización de pruebas

- Mejorar la eficiencia de la prueba.
- Aportar una cobertura de funciones más amplia.
- Reducir el coste total de la prueba.
- Realizar pruebas que los probadores manuales no pueden.
- Acortar el período de ejecución de la prueba.
- Aumentar la frecuencia de la prueba y reducir el tiempo necesario para los ciclos de prueba.

© JMA 2020. All rights reserved

Ventajas y Desventajas de la automatización

Ventajas

- Se pueden realizar más pruebas por compilación.
- La posibilidad de crear pruebas que no se pueden realizar manualmente (pruebas en tiempo real, remotas, en paralelo).
- Las pruebas pueden ser más complejas.
- Las pruebas se ejecutan más rápido.
- Las pruebas están menos sujetas a errores del operador.
- Uso más eficaz y eficiente de los recursos de pruebas
- Información de retorno más rápida sobre la calidad del software.
- Mejora de la fiabilidad del sistema (por ejemplo, repetibilidad y consistencia).
- Mejora de la consistencia de las pruebas.

Desventajas

- Requiere tecnologías adicionales.
- Existencia de costes adicionales.
- Inversión inicial para el establecimiento de la SAP.
- Requiere un mantenimiento continuo de la SAP.
- El equipo necesita tener competencia en desarrollo y automatización.
- Puede distraer la atención respecto a los objetivos de la prueba, por ejemplo, centrándose en la automatización de casos de prueba a expensas del objetivo de las pruebas.
- Las pruebas pueden volverse más complejas.
- La automatización puede introducir errores adicionales.

© JMA 2020. All rights reserved

Limitaciones de la automatización de pruebas

- No todas las pruebas manuales se pueden automatizar.
- La automatización sólo puede comprobar resultados predecibles e interpretables por la máquina.
- La automatización sólo puede comprobar los resultados reales que pueden ser verificados por un oráculo de prueba automatizado.
- No es un sustituto de las pruebas exploratorias.

© JMA 2020. All rights reserved

Técnicas para la automatización de pruebas

- Las herramientas para la automatización de pruebas funcionales aplican técnicas diferentes para la creación y ejecución de pruebas:
  - Técnica de **Programación con Frameworks**: Los marcos de pruebas unitarias (unitarias, integración, sistema) simplifican la creación y ejecución de los procedimientos de pruebas en el mismo lenguaje que el objeto de la prueba. La ejecución y cobertura de las pruebas se pueden integrar en los entornos de desarrollo.
    - Unit Test Frameworks: JUnit, MSTest, NUnit, xUnit, Jest, PHPUnit, …
    - Mocking Frameworks: Mockito, Microsoft Fakes, Moq, NSubstitute, Rhino Mocks, …
    - Code Coverage: Jacoco, Clover, NCover, RCov, …
  - Técnica de **registro/reproducción** (Record & Playback): Esta técnica consiste en grabar una ejecución de la prueba realizada en la interfaz de la aplicación y su reproducción posterior.
  - Técnica de **Programación de Scripts Estructurados**: Difiere de la técnica de registro/reproducción en la introducción de una librería de scripts reutilizables, que realizan secuencias de instrucciones que se requieren comúnmente en una serie de pruebas. Los script de pruebas se crean en un lenguaje de scripting común.
  - Técnica de **Data-Driven**: Esta es una técnica que se enfoca en la separación de los datos de prueba de los scripts, y los almacena en archivos separados. Por lo tanto, los scripts sólo contendrá los procedimientos de prueba y las acciones para la aplicación.
  - Técnica de **Keyword-Driven**: Se basa en la recuperación de los procedimientos de prueba desde los scripts, quedando sólo los datos de prueba y acciones específicas de prueba, que se identifican por palabras clave.

© JMA 2020. All rights reserved

Herramientas de automatización de pruebas

Selenium
APACHE JMeter™
Telerik Test Studio
TestingWhiz Code Less, Test More
LAMBDATEST
Kobiton
appium
Playwright
SoapUI Supported by SMARTBEAR
FitNesse
cucumber
Katalon
HP UFT
POSTMAN
TestArchitect

© JMA 2020. All rights reserved

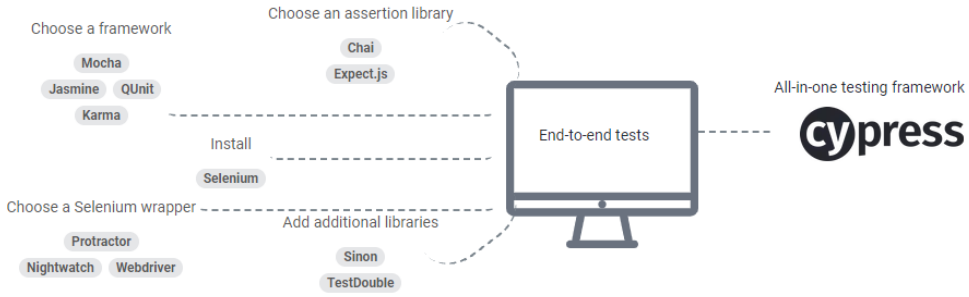
Entornos de pruebas

- Un enfoque de varios entornos permite compilar, probar y liberar código con mayor velocidad y frecuencia para que la implementación sea lo más sencilla posible. Permite quitar la sobrecarga manual y el riesgo de una versión manual y, en su lugar, automatizar el desarrollo con un proceso de varias fases destinado a diferentes entorno.
  - Desarrollo: es donde se desarrollan los cambios en el software.
  - Prueba: permite que los evaluadores humanos o las pruebas automatizadas prueben el código nuevo y actualizado. Los desarrolladores deben aceptar código y configuraciones nuevos mediante la realización de pruebas unitarias en el entorno de desarrollo antes de permitir que esos elementos entren en uno o varios entornos de prueba.
  - Ensayo/preproducción: donde se realizan pruebas finales inmediatamente antes de la implementación en producción, debe reflejar un entorno de producción real con la mayor precisión posible.
  - UAT: Las pruebas de aceptación de usuario (UAT) permiten a los usuarios finales o a los clientes realizar pruebas para comprobar o aceptar el sistema de software antes de que una aplicación de software pueda pasar a su entorno de producción.
  - Producción: es el entorno con el que interactúan directamente los usuarios.

© JMA 2020. All rights reserved

Marcos de pruebas JavaScript

- Escribir pruebas requiere muchas herramientas diferentes trabajando juntas: entornos de pruebas, bibliotecas de aserciones, dobles de pruebas, control remoto de navegadores, utilidades, …



© JMA 2020. All rights reserved

Marcos de pruebas JavaScript

- Test framework:
  - Jasmine: http://jasmine.github.io/
  - Jest: https://jestjs.io
  - Mocha: http://mochajs.org/
  - QUnit: http://qunitjs.com/
- Aserciones:
  - Chai: http://chaijs.com/
- Dobles de pruebas:
  - Sinon.JS: http://sinonjs.org/
- End to End (e2e)
  - Selenium: https://www.selenium.dev/
  - Playwright: https://playwright.dev/
  - Cypress: https://www.cypress.io/
- Test runner:
  - Karma: https://karma-runner.github.io/
  - Web Test Runner: https://modern-web.dev/guides/test-runner/

© JMA 2020. All rights reserved

HERRAMIENTAS DE DESARROLLO

© JMA 2020. All rights reserved

IDEs

- Visual Studio Code - http://code.visualstudio.com/
  - VS Code is a Free, Lightweight Tool for Editing and Debugging Web Apps.
  - Visual Studio Code for the Web provides a free, zero-install Microsoft Visual Studio Code experience running entirely in your browser: https://vscode.dev
- StackBlitz - https://stackblitz.com
  - The online IDE for web applications. Powered by VS Code and GitHub.
- CodeSandbox - https://codesandbox.io/
  - Create, share, and get feedback with collaborative sandboxes.
- IntelliJ IDEA - https://www.jetbrains.com/idea/
  - Capable and Ergonomic Java * IDE
- Webstorm - https://www.jetbrains.com/webstorm/
  - Lightweight yet powerful IDE, perfectly equipped for complex client-side development and server-side development with Node.js

© JMA 2020. All rights reserved

Instalación de utilidades

Consideraciones previas

- Las utilidades son de línea de comandos.
- Para ejecutar los comandos es necesario abrir la consola comandos (Símbolo del sistema)
- Siempre que se realice una instalación o creación es conveniente “Ejecutar como Administrador” para evitar otros problemas.
- En algunos casos el firewall de Windows, la configuración del proxy y las aplicaciones antivirus pueden dar problemas.

GIT: Software de control de versiones

- Descargar e instalar: https://git-scm.com/
- Verificar desde consola de comandos:
  - git

Node.js: Entorno en tiempo de ejecución

- Descargar e instalar: https://nodejs.org
- Verificar desde consola de comandos:
  - node --version

© JMA 2020. All rights reserved

npm: Node Package Manager

- Aunque se instala con el Node es conveniente actualizarlo:
  - npm update -g npm
- Verificar desde consola de comandos:
  - npm --version
- Configuración:
  - npm config edit
  - proxy=http://usr:pwd@proxy.dominion.com:8080 ← Símbolos: %HEX ASCII
- Generar fichero de dependencias package.json:
  - npm init
- Instalación de paquetes:
  - npm install -g grunt-cli grunt-init karma karma-cli ← Global (CLI)
  - npm install jasmine-core tslint --save --save-dev
  - npm install ← Dependencias en package.json
- Arranque del servidor:
  - npm start

© JMA 2020. All rights reserved

ESLint

- Los analizadores de código son herramientas que realizan la lectura del código fuente y devuelve observaciones o puntos en los que tu código puede mejorarse desde la percepción de buenas prácticas de programación y código limpio.
- ESLint (https://eslint.org/) es una herramienta para identificar e informar sobre patrones encontrados en código ECMAScript/JavaScript, con el objetivo de hacer que el código sea más consistente y evitar errores: toma nuestro código, lo escanea y, si encuentra un problema, devuelve un mensaje describiéndolo y mostrando su ubicación aproximada.
- Se puede instalar ESLint usando npm (las dependencias se instalarán automáticamente):
  - npm install --save-dev eslint typescript-eslint
- Luego se debe crear un archivo de configuración .eslintrc.json en el directorio:
  - npm init @eslint/config

    { "parserOptions": { "project": "./tsconfig.json" }, "extends": ["plugin:typescript-eslint/recommended"]}
- Se puede ejecutar ESLint con cualquier archivo o directorio:
  - npx eslint src/**/*{.ts,.tsx}
- Se puede agregar un nuevo script al package.json:
  - "lint": "eslint src/**/*{.ts,.tsx}"

© JMA 2020. All rights reserved

ESLint

- Las reglas son el componente básico de ESLint. Una regla valida si el código cumple con una determinada expectativa y qué hacer si no cumple con esa expectativa. Las reglas también pueden contener opciones de configuración adicionales específicas para esa regla. Hay paquetes adicionales con conjuntos de regalas específicas para muchos marcos.
- Para cambiar la gravedad de una regla, se asigna al ID de la regla uno de estos valores:
  - "off"o 0 desactiva la regla
  - "warn"o 1 activa la regla como advertencia (no afecta el código de salida)
  - "error"o 2 activa la regla como un error (el código de salida es 1 cuando se activa)
- En el archivo de configuración .eslintrc.json:

```
"@typescript-eslint/no-unused-vars": ["error", { "argsIgnorePattern": "^_" }]
```

- Para deshabilitar una regla específica en el código:

```
/* eslint-disable @typescript-eslint/no-explicit-any */
// eslint-disable-next-line @typescript-eslint/consistent-indexed-object-style
/* eslint-disable @typescript-eslint/no-unused-vars */
/* eslint-enable @typescript-eslint/no-unused-vars */
```

© JMA 2020. All rights reserved

Karma: Gestor de Pruebas unitarias de JavaScript

- Instalación general:
  - npm install -g karma
  - npm install -g karma-cli
- Generar fichero de configuración karma.conf.js:
  - karma init
- Levantar el servidor se pruebas:
  - karma start
- Ingenierías de pruebas unitarias disponibles:
  - http://jasmine.github.io/
  - http://qunitjs.com/
  - http://mochajs.org
  - https://github.com/caolan/nodeunit
  - https://github.com/nealxyc/nunit.js

© JMA 2020. All rights reserved

Web Test Runner

https://modern-web.dev/docs/test-runner/overview/

- Instalación como dependencia de desarrollo:
  - npm i --save-dev @web/test-runner
- Configurar su arranque en package.json:

  "test": "web-test-runner test/**/*.test.js --node-resolve"

  "test": "web-test-runner test/**/*.test.js --node-resolve --watch"

  "test": "web-test-runner test/**/*.test.js --node-resolve --coverage"
- Ejecutar las pruebas:
  - npm test

© JMA 2020. All rights reserved

GIT

- Preséntate a Git
  - git config --*global* user.name "Your Name Here“
  - git config --*global* user.email your_email@youremail.com
- Crea un repositorio central
  - https://github.com/
- Conecta con el repositorio remoto
  - git remote add origin https://github.com/username/myproject.git
  - git push -u origin master
- Actualiza el repositorio con los cambios:
  - git commit -m "first commit"
  - git push
- Para clonar el repositorio:
  - git clone https://github.com/username/myproject.git *local-dir*
- Para obtener las últimas modificaciones:
  - git pull

© JMA 2020. All rights reserved

TÉCNICAS ESTÁTICAS

© JMA 2020. All rights reserved

Técnicas estáticas

- Al contrario que las pruebas dinámicas, que exigen la ejecución de software, las técnicas de pruebas estáticas se basan en el examen manual (revisiones) y en el análisis automatizado (análisis estático) del código o de cualquier otra documentación del proyecto sin ejecutar el código.
- Las revisiones constituyen una forma de probar los productos de trabajo del software (incluyendo el código) y pueden realizarse antes de ejecutar las pruebas dinámicas. Los defectos detectados durante las revisiones al principio del ciclo (por ejemplo, los defectos encontrados en los requisitos) a menudo son mucho más baratos de eliminar que los detectados durante las pruebas realizadas ejecutando el código.
- Una revisión podría hacerse íntegramente como una actividad manual, pero también existen herramientas de soporte.

© JMA 2020. All rights reserved

Técnicas estáticas

- Las revisiones, el análisis estático y las pruebas dinámicas tienen el mismo objetivo: Identificar defectos.
  - Se trata de procesos complementarios.
  - Las distintas técnicas pueden encontrar distintos tipos de defectos de una manera eficiente y efectiva.
  - En comparación con las pruebas dinámicas, las técnicas estáticas localizan las causas o los defectos más que los propios fallos.
- Entre los defectos típicos que resultan más fáciles de localizar en revisiones que en las pruebas dinámicas se incluyen:
  - Desviaciones de los estándares,
  - Defectos de requisitos,
  - Defectos de diseño,
  - Mantenibilidad insuficiente y
  - Especificaciones de interfaz incorrectas.

© JMA 2020. All rights reserved

Revisión

- La principal actividad manual consiste en examinar un producto de trabajo y hacer comentarios al respecto. Cualquier producto de trabajo de software puede ser objeto de una revisión como por ejemplo:
  - Las especificaciones de requisitos,
  - Las especificaciones de diseño,
  - El código,
  - Los planes de pruebas,
  - Las especificaciones de pruebas,
  - Los casos de pruebas,
  - Los guiones de pruebas,
  - Las guías de usuario o las páginas web.
- Los beneficios de las revisiones incluyen la detección y corrección temprana de defectos, el desarrollo de mejoras de productividad, la reducción de los tiempos de desarrollo, el ahorro de tiempo y dinero invertido en pruebas, el menor coste de vida, menos defectos y comunicación mejorada. Las revisiones pueden encontrar omisiones, por ejemplo, en requisitos, que no suelen encontrarse en pruebas dinámicas.

© JMA 2020. All rights reserved

Análisis estático con herramientas

- El objetivo del análisis estático es detectar defectos en el código fuente y en los modelos de software.
- El análisis estático se realiza sin que la herramienta llegue a ejecutar el software objeto de la revisión, como ocurre en las pruebas dinámicas, centrándose mas en como está escrito el código que en como se ejecuta el código.
- El análisis estático permite identificar defectos difíciles de encontrar mediante pruebas dinámicas. Al igual que sucede con las revisiones, el análisis estático encuentra defectos en lugar de fallos.
- Las herramientas de análisis estático analizan el código del programa (por ejemplo, el flujo de control y flujo de datos) y las salidas generadas (tales como HTML o XML).
- Algunos de los posibles aspectos que pueden ser comprobados con análisis estático:
  - Reglas, estándares de programación y buenas practicas.
  - Diseño de un programa (análisis de flujo de control).
  - Uso de datos (análisis del flujo de datos).
  - Complejidad de la estructura de un programa (métricas, por ejemplo valor ciclomático).

© JMA 2020. All rights reserved

Valor del análisis estático

- La detección temprana de defectos antes de la ejecución de las pruebas.
- Advertencia temprana sobre aspectos sospechosos del código o del diseño mediante el cálculo de métricas, tales como una medición de alta complejidad.
- Identificación de defectos que no se encuentran fácilmente mediante pruebas dinámicas.
- Detectar dependencias e inconsistencias en modelos de software, como enlaces.
- Mantenibilidad mejorada del código y del diseño.
- Prevención de defectos, si se aprende la lección en la fase de desarrollo.

© JMA 2020. All rights reserved

Defectos habitualmente detectados

- Referenciar una variable con un valor indefinido.
- Interfaces inconsistentes entre módulos y componentes.
- Variables que no se utilizan o que se declaran de forma incorrecta.
- Código inaccesible (muerto).
- Ausencia de lógica o lógica errónea (posibles bucles infinitos).
- Construcciones demasiado complicadas.
- Infracciones de los estándares de programación.
- Vulnerabilidad de seguridad.
- Infracciones de sintaxis del código y modelos de software.

© JMA 2020. All rights reserved

Ejecución

- Las herramientas de análisis estático generalmente las utilizan los desarrolladores (cotejar con las reglas predefinidas o estándares de programación) antes y durante las pruebas unitarias y de integración, o durante la comprobación del código.
- Las herramientas de análisis estático pueden producir un gran número de mensajes de advertencias que deben ser bien gestionados para permitir el uso más efectivo de la herramienta.
- Los compiladores pueden constituir un soporte para los análisis estáticos, incluyendo el cálculo de métricas.
- El Compilador detecta errores sintácticos en el código fuente de un programa, crea datos de referencia del programa (por ejemplo lista de referencia cruzada, llamada jerárquica, tabla de símbolos), comprueba la consistencia entre los tipos de variables y detecta variables no declaradas y código inaccesible (código muerto).
- El Analizador trata aspectos adicionales tales como: Convenciones y estándares, Métricas de complejidad y Acoplamiento de objetos.

© JMA 2020. All rights reserved

Pruebas de mantenibilidad

- Los equipos suelen centrarse en el desarrollo rentable de sistemas de TI. El mantenimiento ocupa aproximadamente el 70% de la vida útil completa del sistema de TI. En el costo total de propiedad, los costos de mantenimiento suelen ser mucho más altos que los costos de desarrollo. El desarrollo rápido y sucio (que causa una gran deuda técnica) puede derivar en costos totales más altos que los sistemas bien diseñados y desarrollados. El estándar ISO25010 define cinco subcaracterísticas de mantenibilidad dentro de las llamadas no funcionales:
  - Modularidad: el grado en que un sistema o programa está compuesto de componentes discretos, de modo que un cambio en un componente tiene un impacto mínimo en otros componentes.
  - Reutilizabilidad: el grado en que un activo se puede utilizar en más de un sistema o en la construcción de otros activos.
  - Analizabilidad: el grado de efectividad y eficiencia con el que es posible evaluar el impacto en un producto o sistema de un cambio previsto en una o más de sus partes, o diagnosticar un producto en busca de deficiencias o causas de fallas, o identificar partes. ser modificado.
  - Modificabilidad: el grado en que un producto o sistema puede modificarse efectiva y eficientemente sin introducir fallas ni degradar la calidad del producto existente.
  - Capacidad de prueba: el grado de efectividad y eficiencia con el que se pueden establecer criterios de prueba para un sistema, producto o componente y se pueden realizar pruebas para determinar si se han cumplido esos criterios.

© JMA 2020. All rights reserved

Mantenibilidad

- La mantenibilidad consiste en modificar un sistema de TI de manera fácil y efectiva. Si se dedica más esfuerzo a mantener el código existente que a escribir código nuevo, esto es un indicio de que la capacidad de mantenimiento puede ser deficiente. Las modificaciones pueden incluir correcciones, mejoras o adaptaciones del software. Pero también se relaciona con cambios en el entorno y en los requisitos, especificaciones e historias de usuarios. También se refiere a la instalación de actualizaciones y mejoras. Los indicadores de mantenibilidad son la calidad del código, la calidad de la documentación, la calidad de las pruebas y muchos otros.
- Esta variedad de prueba no debe confundirse con las pruebas de mantenimiento. En las pruebas de mantenimiento, el motivo de la prueba es que se cambió el sistema. En las pruebas de mantenibilidad, el motivo de las pruebas es ver qué tan bien se puede actualizar, cambiar y mantener el sistema.
- La mantenibilidad se determinan midiendo un conjunto de propiedades del producto de software. Estas propiedades son volumen, duplicación, complejidad de la unidad, tamaño de la unidad, interfaz de la unidad, acoplamiento de módulos, equilibrio de componentes e independencia de los componentes

© JMA 2020. All rights reserved

Medición

- Los artefactos de software que se pueden evaluar son, por ejemplo, archivos de código fuente, archivos de definición de datos, archivos de manipulación de datos y contenido activo de páginas web.
- Las pruebas de mantenibilidad pueden realizarse o respaldarse con herramientas automatizadas.
- La medición estática, normalmente con herramientas automatizadas, puede obtener:
  - Volumen del código, por ejemplo medido en líneas de código.
  - Complejidad ciclomática de una unidad y/o del sistema en su conjunto.
  - Proporción de comentario a código
  - Profundidad de anidamiento de declaraciones de decisión
  - Acoplamiento: número de elementos de datos pasados entre módulos
- La medición dinámica de la mantenibilidad consiste principalmente en cronometrar la duración de tareas de mantenimiento específicas.
  - Tiempo transcurrido para implementar un cambio
  - Estabilidad del sistema, por ejemplo, tiempo medio entre fallos.

© JMA 2020. All rights reserved

Mantenibilidad del código

- Tenemos disponibles toda una serie de principios, patrones de diseño y buenas practicas para facilitar la mantenibilidad de los sistemas.
- Cuanto más complejo es un sistema más probable es que falle y mas difícil de entender, probar y mantener.
- El clean code (código limpio) esta basado en que el código se escribe una vez y se lee muchas veces, reúne una serie de principios que ayudan a producir código legible, intuitivo y fácil de modificar. El código que no sigue estos principios se denomina smell code (código apestoso).
- Cualquier proyecto no sólo debería tener código limpio, sino también una arquitectura limpia con un bajo acoplamiento (forma y nivel de las interdependencia entre las partes) y una alta cohesión (grado de por que los elementos de una parte están juntos). Esto conduce a una menor complejidad del módulo, una mayor reutilización del módulo y una mejor mantenibilidad del sistema, lo que hace que el código sea más fácil de entender, probar y depurar.
- Disponemos de principios arquitectónicos como:
  - SOLID (Principios de responsabilidad única, abierto-cerrado, sustitución de Liskov, segregación de interfaz e inversión de dependencia): se utiliza para hacer que el código de producción sea comprensible, flexible y mantenible.
  - YAGNI (You Aren't Gonna Need it o "no lo vas a necesitar"): Mantenga limpia su base de código y no desarrolle código que no sea necesario. Elimine el código si ya no se usa.
  - KISS (beso, Keep It Simple, Stupid, Hazlo simple, estúpido): No diseñe demasiado la base del código del sistema de TI. El código simple es más fácil de mantener y comprender.
  - DRY (seco, Don't Repeat Yourself, o no te repitas"): No repitas código, refactoriza. Lo contrario de DRY es WET (húmedo, We Enjoy Typing o nos lo pasamos bien tecleando).

© JMA 2020. All rights reserved

Análisis estático Web

- HTML
  - https://validator.w3.org/
- CSS
  - http://jigsaw.w3.org/css-validator/
- WAI
  - https://www.w3.org/WAI/ER/tools/
- JavaScript
  - http://jshint.com/
  - http://www.jslint.com/
- TypeScript
  - https://palantir.github.io/tslint/
- Lighthouse en Chrome DevTools (panel "Audits")
  - https://github.com/GoogleChrome/lighthouse

© JMA 2020. All rights reserved

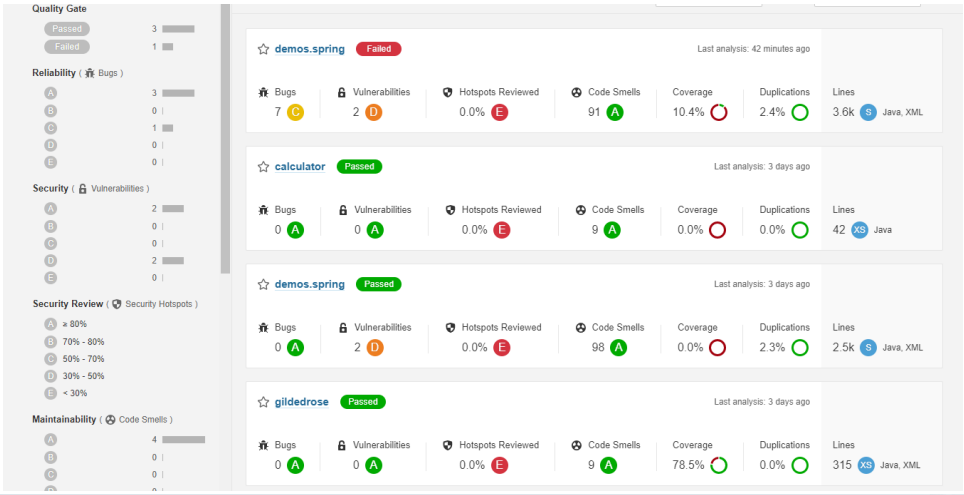
SonarQube

http://www.sonarqube.org/

- SonarQube (conocido anteriormente como Sonar) es una plataforma para la revisión y evaluación del código fuente.
- Es open source y realiza el análisis estático de código fuente integrando las mejores herramientas de medición de la calidad de código como Checkstyle, PMD o FindBugs, para obtener métricas que pueden ayudar a mejorar la calidad del código de un programa.
- Informa sobre código duplicado, estándares de codificación, pruebas unitarias, cobertura de código, complejidad ciclomática, posible errores, comentarios y diseño del software.
- Aunque pensado para Java, acepta mas de 20 lenguajes mediante extensiones.
- Se integra con Maven, Ant y herramientas de integración continua como Atlassian Bamboo, Jenkins y Hudson).

© JMA 2020. All rights reserved

Sonar



© JMA 2020. All rights reserved

DotNet

Cobertura de código de Visual Studio

- Solo se recomienda el uso de esta herramienta cuando el agente de compilación tenga instalado Visual Studio Enterprise (instalación):
  - SonarScanner.MSBuild.exe begin /k:"<sonar-project-key>" /d:sonar.login="<sonar-token>"
  - **dotnet build --no-incremental**
  - **dotnet test --collect "Code Coverage"**
  - SonarScanner.MSBuild.exe end /d:sonar.login="<sonar-token>"

Utilidad dotnet-coverage

- Esta es una alternativa moderna a la Cobertura de código de Visual Studio proporcionada por Microsoft que genera resultados en el mismo formato, es multiplataforma y no depende de tener instalado Visual Studio. Requiere .NET Core 3.1 o posterior. Se puede instalar como una herramienta local o global de dotnet:
  - dotnet tool install --global dotnet-sonarscanner
  - dotnet tool install --global dotnet-coverage
- El script de compilación se vería como algo así:
  - dotnet sonarscanner begin /k:"<sonar-project-key>" /d:sonar.login="<sonar-token>" **/d:sonar.cs.vscoveragexml.reportsPaths=coverage.xml**
  - **dotnet build --no-incremental**
  - **dotnet-coverage collect "dotnet test" -f xml -o "coverage.xml"**
  - dotnet sonarscanner end /d:sonar.login="<sonar-token>"

© JMA 2020. All rights reserved

QUALITY ASSURANCE: DISEÑAR PARA PROBAR

© JMA 2020. All rights reserved

Aseguramiento de la calidad

- El aseguramiento de la calidad son todas aquellas actividades y los procesos que se realizan para asegurar que los productos y servicios de un proyecto posean el nivel de calidad requerido, está orientado al proceso y se centra en el desarrollo del producto o servicio.
- El aseguramiento de la calidad del software (SQA) es un conjunto de prácticas, metodologías y actividades adoptadas en el proceso de desarrollo con el objetivo de garantizar que el software producido cumpla con altos estándares de calidad. El QA en el contexto del desarrollo de software es esencial para reducir defectos, mejorar la confiabilidad del software, garantizar la seguridad y satisfacer al cliente. Sus principales características son:
  - Preventivo: es un enfoque preventivo para la gestión de la calidad. Se centra en la definición de procesos, estándares y procedimientos para prevenir defectos y problemas de calidad desde el inicio.
  - Actividades de planificación: implica la elaboración de planes de calidad que establecen objetivos, estrategias, recursos y procesos para el control de calidad durante todo el ciclo de vida del proyecto.
  - Orientado a procesos: el enfoque principal está en la calidad de los procesos utilizados para desarrollar el software, se centra en asegurar que los procesos estén bien definidos, bien gestionados y que se sigan.
  - Prevención de defectos: el objetivo principal es prevenir defectos antes de que se produzcan y mejorar constantemente los procesos para minimizar la probabilidad de errores en el software.
  - Involucración continua: está involucrado en todo el ciclo de vida del proyecto, desde la planificación hasta la entrega, para garantizar que se cumplan los estándares de calidad en cada fase.

© JMA 2020. All rights reserved

Diseñar para probar

- Si bien la antigua fase de pruebas era la última del ciclo de vida, las actividades del proceso de pruebas deben ser incorporadas desde la fase de especificación y tenerlas en cuenta en las etapas de análisis y diseño con facilitadores de las pruebas.
- Fundamentos de diseño
  - Programar para las interfaces, no para la herencia.
  - Favorecer la composición antes que la herencia.
- Patrones:
  - Delegación
  - Doble herencia
  - Inversión de Control e Inyección de Dependencias
  - Segregación IU/Código: Modelo Vista Controlador (MVC), Model View ViewModel (MVVM)
- Metodologías (Test-first development):
  - Desarrollo Guiado por Pruebas (TDD)
  - Desarrollo Dirigido por Comportamiento (BDD)
  - Desarrollo Dirigido por Tests de Aceptación (ATDD)

© JMA 2020. All rights reserved

Simulación de objetos

- Las dependencias introducen ruido en las pruebas: ¿el fallo se está produciendo en el objeto de la prueba o en una de sus dependencias?
- Las dependencias con sistemas externos afectan a la complejidad de la estrategia de pruebas, ya que es necesario contar con sustitutos de estos servicios externos durante el desarrollo. Ejemplos típicos de estas dependencias son Servicios Web, Sistemas de envío de correo, Fuentes de Datos o simplemente dispositivos hardware.
- Estos sustitutos, muchas veces son exactamente iguales que el servicio original, pero en otro entorno o son simuladores que exponen el mismo interfaz pero realmente no realizan las mismas tareas que el sistema real, o las realizan contra un entorno controlado.
- Para poder emplear la técnica de simulación de objetos se debe diseñar el código a probar de forma que sea posible trabajar con los objetos reales o con los objetos simulados:
  - Doble herencia
  - IoC: Inversión de Control (Inversion Of Control) o
  - DI: Inyección de Dependencias (Dependency Injection)
  - Objetos Mock

© JMA 2020. All rights reserved

Dobles de prueba

- La regla de oro de las pruebas unitarias, es que una unidad (unit) tiene que ser probada sin utilizar ninguna de sus dependencias.
- Siguiendo la misma regla de oro, las pruebas de integración y sistema deben estar aisladas de sus dependencias salvo cuando se estén probando dichas dependencias. Así mismo, el resultado de las pruebas debe ser previsible.
- Entre las ventajas de esta aproximación se encuentran:
  - Devuelven resultados determinísticos
  - Permiten crear o reproducir determinados estados (por ejemplo errores de conexión)
  - Obtienen resultados mucho mas rápidamente y a menor coste, incluso offline.
  - Permiten el inicio temprano de las pruebas incluso cuando las dependencias todavía no están disponibles.
  - Permiten incluir atributos o métodos exclusivamente para el testeo.

© JMA 2020. All rights reserved

Dobles de prueba

- **Fixture**: Es el término se utiliza para hablar de los datos de contexto de las pruebas, aquellos que se necesitan para construir el escenario que requiere la prueba.
- **Dummy**: Objeto que se pasa como argumento pero nunca se usa realmente. Normalmente, los objetos dummy se usan sólo para rellenar listas de parámetros.
- **Fake**: Objeto que tiene una implementación que realmente funciona pero, por lo general, usa una simplificación que le hace inapropiado para producción (como una base de datos en memoria por ejemplo).
- **Stub**: Objeto que proporciona respuestas predefinidas a llamadas hechas durante los tests, frecuentemente, sin responder en absoluto a cualquier otra cosa fuera de aquello para lo que ha sido programado. Los stubs pueden también grabar información sobre las llamadas (**spy**).
- **Mock**: Objeto pre programado con expectativas que conforman la especificación de cómo se espera que se reciban las llamadas. Son más complejos que los stubs aunque sus diferencias son sutiles.

© JMA 2020. All rights reserved

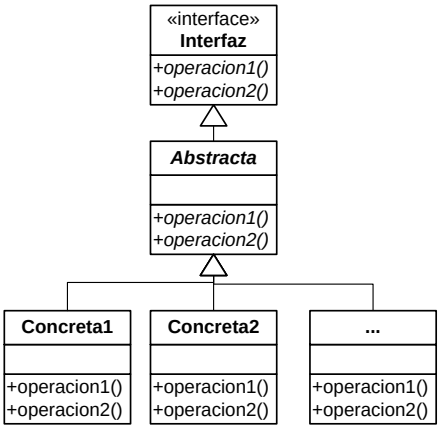
Programar con interfaces

- La herencia de clase define la implementación de una clase a partir de otra (excepto métodos abstractos)
- La implementación de interfaz define como se llamara el método o propiedad, pudiendo escribir distinto código en clases no relacionadas.
- Reutilizar la implementación de la clase base es la mitad de la historia.
- Ventajas:
  - Reducción de dependencias.
  - El cliente desconoce la implementación.
  - La vinculación se realiza en tiempo de ejecución.
  - Da consistencia (contrato).
- Desventaja:
  - Indireccionamiento.

© JMA 2020. All rights reserved

Doble Herencia

- Problema:
  - Organizar clases que tienen un comportamiento parecido para que sean intercambiables y consistentes (polimorfismo).
  - Mantener las clases que implementan el interfaz como internas del proyecto (internal o Friend), pero la interfaz pública.
- Solución:
  - Crear un interfaz paralelo a la clase base
  - Clase base es abstracta y sus herederos implementaciones concretas.
  - La clase base puede heredar de mas de una interfaz.
  - Cualquier clase puede implementar el interfaz independientemente de su jerarquía.



© JMA 2020. All rights reserved

Construcción vs Uso

- Los sistemas de software deben separar el proceso de inicio, en el que se instancian los objetos de la aplicación y se conectan las dependencias, de la lógica de ejecución que utilizan las instancias. La separación de conceptos es una de las técnicas de diseño más antiguas e importantes.
- El mecanismo mas potente es la Inyección de Dependencias, la aplicación de la Inversión de Control a la gestión de dependencias. Delega la instanciación en un mecanismo alternativo, que permite la personalización, responsable de devolver instancias plenamente formadas con todas las dependencias establecidas. Permite la creación de instancias bajo demanda de forma transparente al consumidor.
- En lugar de crear una dependencia en sí misma, una parte de la aplicación simplemente declara la dependencia. La tediosa tarea de crear y proporcionar la dependencia se delega a un inyector que se encuentra en la parte superior.
- Esta división del trabajo desacopla una parte de la aplicación de sus dependencias: una parte que no necesita saber cómo configurar una dependencia, y mucho menos las dependencias de la dependencia, etc.

© JMA 2020. All rights reserved

Inversión de Control

- Inversión de control (Inversion of Control en inglés, IoC) es un concepto junto con unas técnicas de programación:
  - en las que el flujo de ejecución de un programa se invierte respecto a los métodos de programación tradicionales,
  - en los que la interacción se expresa de forma imperativa haciendo llamadas a procedimientos (procedure calls) o funciones.
- Tradicionalmente el programador especifica la secuencia de decisiones y procedimientos que pueden darse durante el ciclo de vida de un programa mediante llamadas a funciones.
- En su lugar, en la inversión de control se especifican respuestas deseadas a sucesos o solicitudes de datos concretas, dejando que algún tipo de entidad o arquitectura externa lleve a cabo las acciones de control que se requieran en el orden necesario y para el conjunto de sucesos que tengan que ocurrir. Técnicas de implementación:
  - Service Locator: es un componente (contenedor) que contiene referencias a los servicios y encapsula la lógica que los localiza dichos servicios.
  - Inyección de dependencias.

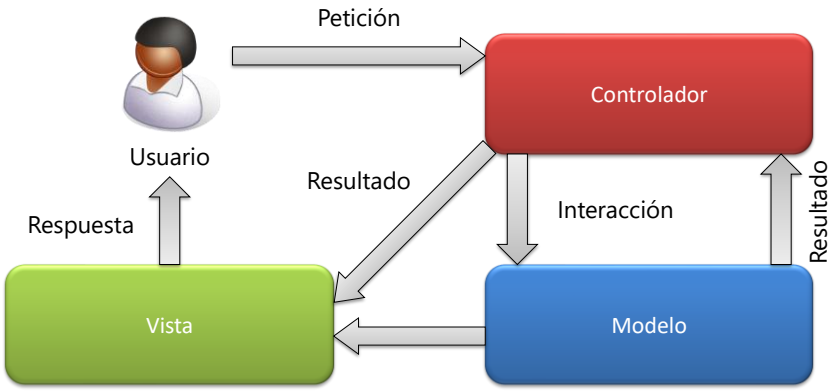
© JMA 2020. All rights reserved

Inyección de Dependencias

- Las dependencias son expresadas en términos de interfaces en lugar de clases concretas y se resuelven dinámicamente en tiempo de ejecución.
- La Inyección de Dependencias (en inglés Dependency Injection, DI) es un patrón de arquitectura orientado a objetos, en el que se inyectan objetos a una clase en lugar de ser la propia clase quien cree el objeto, básicamente recomienda que las dependencias de una clase no sean creadas desde el propio objeto, sino que sean configuradas desde fuera de la clase. La inyección de dependencias (DI) procede del patrón de diseño más general que es la Inversión de Control (IoC).
- Al aplicar este patrón se consigue que las clases sean independientes unas de otras e incrementando la reutilización y la extensibilidad de la aplicación, además de facilitar las pruebas unitarias de las mismas.
- Desde el punto de vista de Java o .NET, un diseño basado en DI puede implementarse mediante el lenguaje estándar, dado que una clase puede leer las dependencias de otra clase por medio del Reflection y crear una instancia de dicha clase inyectándole sus dependencias.

© JMA 2020. All rights reserved

El patrón MVC



© JMA 2020. All rights reserved

El patrón MVC

- Representación de los **datos del dominio**
- Lógica de **negocio**
- Mecanismos de **persistencia**

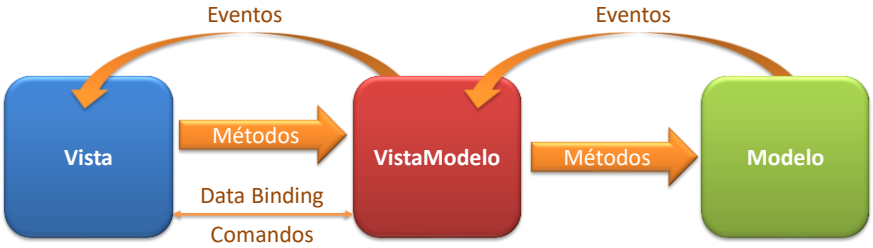
- **Interfaz** de usuario
- Incluye elementos de **interacción**

- **Intermediario** entre Modelo y Vista
- **Mapea acciones** de usuario → acciones del Modelo
- **Selecciona** las vistas y les **suministra** información

© JMA 2020. All rights reserved

Model View ViewModel (MVVM)

- El **Modelo** es la entidad que representa el concepto de negocio.
- La **Vista** es la representación gráfica del control o un conjunto de controles que muestran el Modelo de datos en pantalla.
- La **VistaModelo** es la que une todo. Contiene la lógica del interfaz de usuario, los comandos, los eventos y una referencia al Modelo.



© JMA 2020. All rights reserved

Test-First Development (TFD)

- La prueba gana peso hasta convertirse en "ciudadano de primera clase", es un enfoque que consiste empezar por la prueba para una característica determinada, para luego diseñarla y desarrollarla en un solo paso.
- Al escribir la prueba primero, estas pueden centrarse en los resultados esperados en lugar de en los detalles de implementación, facilitan el diseño de una solución clara y eficaz y, una vez que se completa el desarrollo, se ejecutan para ver si pasan o no.
- Como evolución del Test-first development (XP) las principales metodologías son:
  - Desarrollo Guiado por Pruebas (TDD)
  - Desarrollo Dirigido por Comportamiento (BDD)
  - Desarrollo Dirigido por Tests de Aceptación (ATDD)

© JMA 2020. All rights reserved

Desarrollo Guiado por Pruebas (TDD)

- El Desarrollo Guiado por Pruebas, es una técnica de programación (definida por KentBeck); consistente en desarrollar primero el código que pruebe una característica o funcionalidad deseada antes que el código que implementa dicha funcionalidad y refactorizar después de implementar dicha funcionalidad.
- El objetivo a lograr es que no exista ninguna funcionalidad que no esté avalada por una prueba.
- Lo primero que hay que aprender de TDD son sus 3 leyes:
  - No escribirás código de producción sin antes escribir un test que falle.
  - No escribirás más de un test unitario suficiente para fallar (y no compilar es fallar).
  - No escribirás más código del necesario para hacer pasar el test.

© JMA 2020. All rights reserved

Escribir la prueba

- La escritura de las pruebas sigue varios patrones identificados y detallados por Beck. En particular se destaca que las pruebas:
  - Son escritas por el propio desarrollador en el mismo lenguaje de programación que la aplicación, y su ejecución se automatiza. Esto último es primordial para que obtenga un retorno inmediato, indicándose que el ritmo de cambio entre prueba-código-prueba esta pautado por intervalos de no más de diez minutos. La automatización también permite aplicar, en todo momento, la batería de pruebas, implicando pruebas de regresión inmediatas y continuas.
  - Las pruebas se deben escribir pensando en primer lugar en probar las operaciones que se deben implementar.
  - Deben ser aisladas, de forma que puedan correrse independientemente de otras, no importando el orden. Este aislamiento determinará que las soluciones de código cohesivas y bajo acoplamiento, con componentes ortogonales.
  - Deben de escribirse antes que el código que se desea probar.

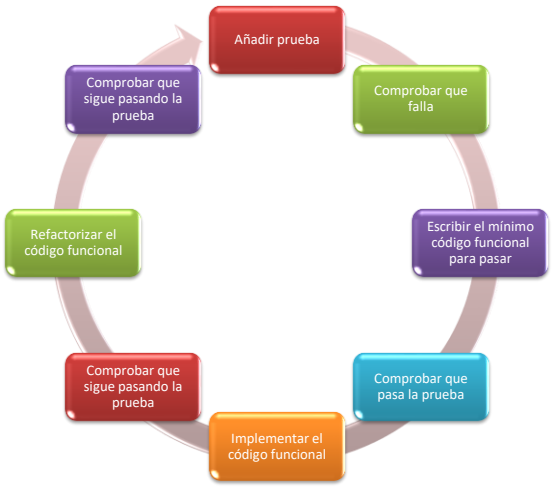
© JMA 2020. All rights reserved

Ritmo TDD

- TDD invita a seguir una serie de tareas ordenadas, que a menudo se denomina ritmo TDD, y que se basa en los siguientes pasos:
  1. Escoger un requisito.
  2. Escribir una prueba que demuestre la necesidad de escribir código (assert first).
  3. Escribir el mínimo código para que el código de pruebas compile (probar la prueba)
  4. Fijar la prueba
  5. Implementar exclusivamente la funcionalidad demandada por las pruebas
  6. Ejecutar todas las prueba.
  7. Mejorar el código (refactorización) sin añadir funcionalidad
  8. Actualizar la lista de requisitos (añadir los nuevos requisitos que hayan aparecido)
  9. Volver al primer paso
- Este ritmo permite formalizar las tareas que se han de realizar para conseguir un código fácil de mantener, bien diseñado y probado automáticamente.

© JMA 2020. All rights reserved

Ritmo TDD



© JMA 2020. All rights reserved

Refactorizar el código en pruebas

- Una refactorización es un cambio que está pensado para que el código se ejecute mejor o para que sea más fácil de comprender.
- No está pensado para alterar el comportamiento del código y, por tanto, no se cambian las pruebas.
- Se recomienda realizar los pasos de refactorización independientemente de los pasos que amplían la funcionalidad.
- Mantener las pruebas sin cambios aporta la confianza de no haber introducido errores accidentalmente durante la refactorización.

© JMA 2020. All rights reserved

Estrategia RED – GREEN – REFACTOR

- Se recomienda una estrategia de test unitarios conocida como RED (fallo) – GREEN (éxito), es especialmente útil en equipos de desarrollo ágil.
- Una vez que entendamos la lógica y la intención de un test unitario, hay que seguir estos pasos:
  - Escribe el código del test (**Stub**) para que compile (pase de **RED** a **GEEN**)
    - Inicialmente la compilación fallará **RED** debido a que falta código
    - Implementa sólo el código necesario para que compile **GREEN** (aún no hay implementación real).
  - Escribe el código fuente para que se **ejecute** (pase de **RED** a **GREEN**)
    - Inicialmente el test fallará **RED** ya que no existe funcionalidad.
    - Implementa la funcionalidad que va a probar el test hasta que se ejecute adecuadamente **GREEN.**
  - **Refactoriza** el test y el código una vez que este todo **GREEN** y la solución vaya evolucionando.

© JMA 2020. All rights reserved

Beneficios de TDD

- Facilita desarrollar ciñéndose a los requisitos.
  - Ayuda a encontrar inconsistencias en los requisitos
  - Reduce el número de errores y bugs ya que éstos se detectan antes incluso de crearlos.
- Facilita el mantenimiento del código:
  - Protege ante cambios, los errores que surgen al aplicar un cambio se detectan (y corrigen) antes de subir ese cambio.
  - Protegen ante errores de regresión (rollbacks a versiones anteriores).
  - Dan confianza.
- Las pruebas facilitan entender el código y que, eligiendo una buena nomenclatura, sirven de documentación.
- Ayudan a especificar comportamientos
- Ayudan a refactorizar para mejorar la calidad del código (Clean code)
- A medio/largo plazo aumenta (y mucho) la productividad.

© JMA 2020. All rights reserved

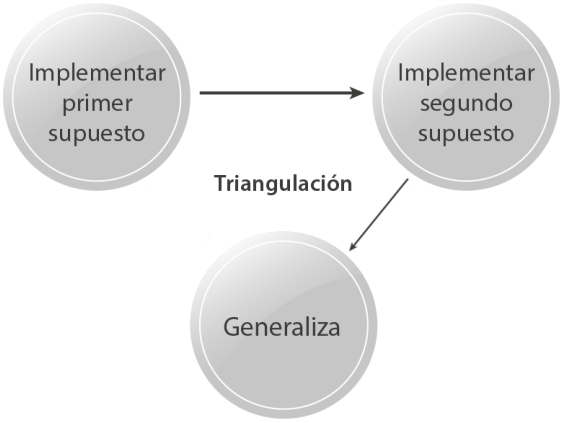
Inside-Out vs Outside-In

- ¿La mejor manera de empezar es por los detalles de lo que está construyendo, y dejar que la arquitectura vaya emergiendo con un enfoque de adentro hacia afuera (Inside Out)? ¿O bien, empezar por algo grande y dejar que los detalles vayan revelando a medida que se van usando desde afuera hacia adentro (Outside In)?.
- La escuela clásica (Inside Out/Bottom Up/Chicago School/Aproximación Clásica) toma su nombre por representar el concepto original definido en libros como «Test-driven Development By Example» de Kent Beck, y que se distingue por enfatizar la práctica de la triangulación (Inside Out).
- La escuela de Londres (Outside In TDD/Top Down/Mockist Approach) toma su nombre debido a que tiene su base, principalmente, en el libro «Growing Object Oriented Software Guide By Test» de Stephen Freeman y Nat Pryce, enfatizando los roles, responsabilidades e interacciones (Outside In).

© JMA 2020. All rights reserved

Técnica de la triangulación

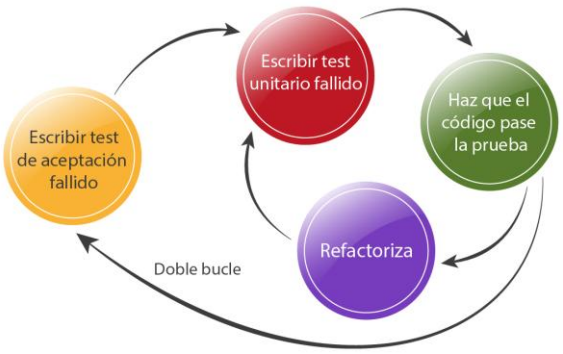
- Una vez que tenemos el test fallando, tenemos tres estrategias posibles para pasar a verde:
  - Fake It: crear un fake que devuelva una constante.
  - Obvious Implementation: implementamos la solución más simple posible si es obvia.
  - Triangulation
- Triangular, o la técnica de la triangulación, es el paso natural que sigue a la técnica de la implementación falsa:
  - Escoger el caso más simple que debe resolver el algoritmo.
  - Aplicar Red-Green-Refactor.
  - Repetir los pasos anteriores cubriendo las diferentes casuísticas.
- El concepto se basa en obtener dos o más casos similares antes de proceder con la implementación de una solución genérica.



© JMA 2020. All rights reserved

Técnica del doble bucle

- La escuela de Londres toma un enfoque distinto, centrándose en verificar que el comportamiento de los objetos es el esperado. Dado que este es el objetivo final, verificar las correctas interacciones entre objetos, y no el estado en sí mismo de los objetos, ahorrándonos todo el trabajo con los objetos reales (creación y mantenimiento) sustituyéndolos por dobles de prueba. Podremos empezar por la funcionalidad final que se necesita, implementando poco a poco toda la estructura que dé soporte a dicha funcionalidad, Outside-in, desde el exterior hacia dentro.
- Mediante el uso de la técnica del doble bucle:
  - Comenzaremos por un test de aceptación que falle, lo que nos guiará al bucle interno.
  - En el bucle interno, que representa la metodología de trabajo TDD («Red-Green-Refactor»), implementaremos la lógica de nuestra solución.
  - Realizaremos las iteraciones necesarias de este bucle interno hasta conseguir pasar el test de aceptación.



© JMA 2020. All rights reserved

Aproximaciones

Escuela Clásica

La aproximación de TDD De adentro hacia afuera (Inside Out) favorece a los desarrolladores a los que les gusta ir construyendo pieza a pieza. Puede ser más sencillo para empezar, siguiendo un enfoque metódico para identificar las entidades, y trabajando duro para llevar a cabo el comportamiento interno.

- Cuando se quiera verificar el estado de los objetos.
- Cuando las colaboraciones entre objetos son sencillas.
- Si no se quiere acoplar la implementación a las pruebas.
- Si se prefiere no pensar en la implementación mientras se escriben las pruebas.

Escuela de Londres

La aproximación de TDD De fuera hacia Adentro (Outside In) el desarrollador empieza a construir todo el sistema, y lo descompone en componentes más pequeños cuando se presentan oportunidades de refactorización. Esta ruta es más exploratorio, y es ideal en las situaciones donde hay una idea general del objetivo, pero los detalles finales de la implementación están menos claros.

- Cuando se quiera verificar el comportamiento de los objetos.
- Cuando las colaboraciones entre objetos sean complejas.
- Si no importa acoplar la implementación a las pruebas.
- Si se prefiere pensar en la implementación mientras se escriben las pruebas.

© JMA 2020. All rights reserved

Desarrollo Dirigido por Comportamiento (BDD)

- El Desarrollo Dirigido por Comportamiento (Behaviour Driver Development) es una evolución de TDD (Test Driven Development o Desarrollo Dirigido por Pruebas), el concepto de BDD fue inicialmente introducido por Dan North como respuesta a los problemas que surgían al enseñar TDD.
- En BDD también vamos a escribir las pruebas antes de escribir el código fuente, pero en lugar de pruebas unitarias, lo que haremos será escribir pruebas que verifiquen que el comportamiento del código es correcto desde el punto de vista de negocio. Tras escribir las pruebas escribimos el código fuente de la funcionalidad que haga que estas pruebas pasen correctamente. Después refactorizamos el código fuente.
- Partiremos de historias de usuario, siguiendo el modelo “Como [rol] quiero [característica] para [los beneficios]”. A partir de aquí, en lugar de describir en 'lenguaje natural' lo que tiene que hacer esa nueva funcionalidad, vamos a usar un lenguaje ubicuo (un lenguaje semiformal que es compartido tanto por desarrolladores como personal no técnico) que nos va a permitir describir todas nuestras funcionalidades de una única forma.

© JMA 2020. All rights reserved

BDD

- Para empezar a hacer BDD sólo nos hace falta conocer 5 palabras, con las que construiremos sentencias con las que vamos a describir las funcionalidades:
  - Feature (característica): Indica el nombre de la funcionalidad que vamos a probar. Debe ser un título claro y explícito. Incluimos aquí una descripción en forma de historia de usuario: “Como [rol] quiero [característica] para [los beneficios]”. Sobre esta descripción empezaremos a construir nuestros escenarios de prueba.
  - Scenario: Describe cada escenario que vamos a probar.
  - Given (dado): Provee el contexto para el escenario en que se va a ejecutar el test, tales como el punto donde se ejecuta el test, o prerequisitos en los datos. Incluye los pasos necesarios para poner al sistema en el estado que se desea probar.
  - When (cuando): Especifica el conjunto de acciones que lanzan el test. La interacción del usuario que acciona la funcionalidad que deseamos testear.
  - Then (entonces): Especifica el resultado esperado en el test. Observamos los cambios en el sistema y vemos si son los deseados.

© JMA 2020. All rights reserved

Ejemplo

```
# language: es
Característica: Suma de dos números
  Como matemático novato
  Yo quiero obtener la suma de dos cifras
  Para aprender a sumar

  Escenario: Sumar dos números positivos
  Dado que estoy en la aplicación
  Cuando pongo los números 1 y 3
  Y solicito el resultado del cálculo
  Entonces el resultado debe ser 4

  Escenario: Sumar dos números negativos
  Dado que estoy en la aplicación
  Cuando pongo los números -1 y -3
  Y solicito el resultado del cálculo
  Entonces el resultado debe ser -4

  Escenario: Sumar un numero positivo y uno negativo
  Dado que estoy en la aplicación
  Cuando pongo los números -2 y 3
  Y solicito el resultado del cálculo
  Entonces el resultado debe ser 1
```

```
public class CalculadoraStepDefinitions {
    Calculadora calc;
    int op1, op2, rslt;

    @Dado("que estoy en la aplicación")
    public void que_estoy_en_la_aplicación() {
        calc = new Calculadora();
    }
    @Cuando("pongo los números {int} y {int}")
    public void pongo_los_números_y(Integer a, Integer b) {
        op1 = a;
        op2 = b;
    }
    @Cuando("solicito el resultado del cálculo")
    public void solicito_el_resultado_del_cálculo() {
        rslt = calc.suma(op1, op2);
    }
    @Entonces("el resultado debe ser {int}")
    public void el_resultado_debe_ser(Integer r) {
        assertEquals(r, rslt);
    }
}
```

© JMA 2020. All rights reserved

Desarrollo Dirigido por Tests de Aceptación (ATDD)

- El Desarrollo Dirigido por Test de Aceptación (ATDD), técnica conocida también como Story Test-Driven Development (STDD), es una variación del TDD pero a un nivel diferente.
- Las pruebas de aceptación o de cliente son el criterio escrito de que un sistema cumple con el funcionamiento esperado y los requisitos de negocio que el cliente demanda. Son ejemplos escritos por los dueños de producto. Es el punto de partida del desarrollo en cada iteración.
- ATDD/STDD es una forma de afrontar la implementación de una manera totalmente distinta a las metodologías tradicionales. Cambia el punto de partida, la forma de recoger y formalizar las especificaciones, sustituye los requisitos escritos en lenguaje natural (nuestro idioma) por historias de usuario con ejemplos concretos ejecutables de como el usuario utilizara el sistema, que en realidad son casos de prueba. Los ejemplos ejecutables surgen del consenso entre los distintos miembros del equipo y el usuario final.
- La lista de ejemplos (pruebas) de cada historia, se escribe en una reunión que incluye a dueños de producto, usuarios finales, desarrolladores y responsables de calidad. Todo el equipo debe entender qué es lo que hay que hacer y por qué, para concretar el modo en que se certifica que el sotfware lo hace.

© JMA 2020. All rights reserved

ATDD

- El algoritmo o ritmo es el mismo de tres pasos que en el TDD practicado exclusivamente por desarrolladores pero a un nivel superior.
- En ATDD hay dos prácticas claves:
  - Antes de implementar (fundamental lo de antes de implementar) una necesidad, requisito, historia de usuario, etc., los miembros del equipo colaboran para crear escenarios, ejemplos, de cómo se comportará dicha necesidad.
  - Después, el equipo convierte esos escenarios en pruebas de aceptación automatizadas. Estas pruebas de aceptación típicamente se automatizan usando Selenium o similares, “frameworks” como Cucumber, etc.

© JMA 2020. All rights reserved

Data Driven Testing (DDT)

- Se basa en la creación de tests para ejecutarse en simultáneo con sus conjuntos de datos relacionados en un framework. El framework provee una lógica de test reusable para reducir el mantenimiento y mejorar la cobertura de test. La entrada y salida (del criterio de test) pueden ser resguardados en uno o más lugares del almacenamiento central o bases de datos, el formato real y la organización de los datos serán específicos para cada caso.
- Todo lo que tiene potencial de cambiar (también llamado "variabilidad," e incluye elementos como el entorno, puntos de salida, datos de test, ubicaciones, etc) está separado de la lógica del test (scripts) y movido a un 'recurso externo'. Esto puede ser configuración o conjunto de datos de test. La lógica ejecutada en el script está dictada por los valores.
- Los datos incluyen variables usadas tanto para la entrada como la verificación de la salida. En casos avanzados (y maduros) los entornos de automatización pueden ser obtenidos desde algún sistema usando los datos reales o un "sniffer", el framework DDT por lo tanto ejecuta pruebas sobre la base de lo obtenido produciendo una herramienta de test automáticos para regresión.

© JMA 2020. All rights reserved

Keyword Driven Development (KDD)

- Keyword Driven Development o KDD es un proceso de desarrollo software basado en KDT, que se impulsa por la palabra clave almacenada en una base de datos y en la automatización a la que hace referencia.
- Keyword Driven Testing o KDT es un framework de automatización de pruebas, donde se utiliza un formato de tabla para definir palabras claves o palabras de acción para cada función que nos gustaría utilizar. La ISO/IEC/IEEE 29119-5 define KDT como una forma de describir los casos de prueba mediante el uso de un conjunto predefinido de palabras clave, en un lenguaje de uso común. Estas palabras clave son nombres que están asociados con un conjunto de acciones que se requieren para realizar los pasos específico en un caso de prueba. Mediante el uso de palabras clave para describir las acciones de prueba en lugar de lenguaje natural, los casos de prueba pueden ser más fáciles de entender, mantener y automatizar.
- La definición de las keywords se produce de una forma independiente a las acciones asociadas, lo que provoca la separación del diseño de la prueba respecto a su ejecución. La reusabilidad de las keywords constituye otra de las potencialidades.

© JMA 2020. All rights reserved

Componentes de Keyword Driven Testing

- Almacén de palabras clave (Keywords): Las palabras que creamos las guardaremos en un determinado formato para que posteriormente sean accedidas por los scripts.
- Almacén de datos: Contiene los datos necesarios para las pruebas y se almacenan en un determinado formato similar al seleccionado para las palabras clave.
- Repositorio de objetos: Repositorio de las referencias necesarias a los objetos UI sobre los que se actuará. Pueden estar referenciadas por id, XPath, etc. En este punto es importante señalar que durante la creación del frontal de la aplicación es conveniente que los desarrolladores codifiquen acorde a los estándares e identifiquen los objetos UI de forma adecuada.
- Biblioteca de funciones: Contiene la codificación de los flujos de negocio para los distintos elementos UI.
- Scripts de pruebas: Son el conjunto de funciones que se llaman desde la biblioteca de funciones para ejecutar el código asociado a una palabra clave.

© JMA 2020. All rights reserved

https://jestjs.io

JEST

© JMA 2020. All rights reserved

Introducción

- Jest es un marco de pruebas (test runner) de JavaScript que se centra en la simplicidad.
- Jest es un marco de prueba diseñado para garantizar la corrección de cualquier base de código JavaScript. Permite escribir pruebas con una API accesible, familiar y rica en funciones que brinda resultados rápidamente.
- Jest está bien documentado, requiere poca configuración y puede ampliarse para adaptarse a las necesidades.
- Jest utiliza un cargador personalizado para las importaciones en sus pruebas, lo que facilita la simulación de cualquier objeto fuera del alcance de la prueba. Se puede usar importaciones simuladas con la rica API de funciones simuladas para espiar llamadas a funciones con sintaxis de prueba legible.
- Jest puede recopilar información de cobertura de código de proyectos completos, incluidos archivos no probados.

© JMA 2020. All rights reserved

Instalación, configuración y ejecución

- Para su instalación mediante npm:
  - npm install --save-dev jest @types/jest
- Por defecto, se utiliza la extensión .test.js para los ficheros de pruebas.
- La configuración se puede definir en el archivo package.json del proyecto, a través de un archivo jest.config.js o jest.config.ts y a través de la opción --config <path/to/file.js|ts|cjs|mjs|json>.

```
"scripts": {
  :
  "test": "stencil test --spec --e2e",
  "test.watch": "stencil test --spec --e2e --watchAll",
},
"jest": {
  "displayName": "Curso de Stencil"
},
```

- Se puede automatizar su ejecución con webpack, Babel, Parcel, …
- Se puede ejecutar directamente desde Jest CLI (npm i jest --global):
  - jest my-test --notify --config=config.json

© JMA 2020. All rights reserved

Suites

- Una “suite” es un nombre que describe al género o sección qué se va a pasar por un conjunto de pruebas unitarias, además es una herramienta que es el núcleo que se necesita para poder tener un orden en el momento de crear las pruebas.
- Las “suites” se crean con la función **describe**, que es una función global y con la cual se inicia toda prueba unitaria, además consta con dos parámetros:

```
describe("Una suite es sólo una función", function() {
//…
});
```

- El primer parámetro es una cadena de caracteres donde se define el nombre descriptivo de la prueba unitaria y el segundo es una función con el código que ejecutará la prueba de código.
- Se pueden anidar “suites” para estructurar conjuntos complejos y facilitar la legibilidad y la búsqueda, basta con crear un describe dentro de otro.

© JMA 2020. All rights reserved

Especificaciones

- Una especificación contiene una o más expectativas (algo que se espera) que ponen a prueba el estado del código. Una expectativa de Jest es una afirmación que debe ser verdadera pero puede ser falsa.
- Una especificación con todas las expectativas verdaderas es una especificación que pasa la prueba, pero con una o más falsas es una especificación que falla.
- Las especificaciones se definen dentro de una Suite llamando a la función global del **test** (o el alias **it**) que, al igual que describe, recibe una cadena (el título de la especificación) y una función (la especificación o caso de prueba), opcionalmente puede recibir un timeout.

```
test("y así es una especificación", function() {
//…
});
```

- **describe** y **test** son funciones: pueden contener cualquier código ejecutable necesario para implementar la prueba y las reglas de alcance de JavaScript se aplican, por lo que las variables declaradas en un describe están disponibles para cualquier bloque test dentro de la suite.

© JMA 2020. All rights reserved

Expectativas

- Las expectativas se construyen con la función expect que obtiene un valor real de una expresión y lo comparan mediante una función Matcher con un el valor esperado (constante).

  expect(valor actual).*matchers*(valor esperado);
- Los matchers (comparadores) son funciones que implementan comparaciones booleanas entre el valor actual y el esperado, ellos son los responsables de informar a Jest si la expectativa se cumple o es falsa.
- Cualquier matcher puede evaluarse como una afirmación negativa mediante el encadenamiento a la llamada expect de un not antes de llamar al matcher.

  expect(valor actual)**.not**.matchers(valor esperado);
- También existe la posibilidad de escribir matchers personalizados para cuando el dominio de un proyecto consiste en afirmaciones específicas que no se incluyen en los ya definidos.

© JMA 2020. All rights reserved

Matchers

.toEqual(y); verifica si ambos valores son iguales ==.

.toBe(y); verifica si ambos objetos son idénticos ===.

.toMatch(pattern); verifica si el valor pertenece al patrón establecido.

.toMatchObject(object); verifica que un objeto coincide en propiedad y valor con un subconjunto de otro objeto.

.toBeDefined(); verifica si el valor está definido.

.toBeUndefined(); verifica si el valor es indefinido.

.toBeNull(); verifica si el valor es nulo.

.toBeNaN(); verifica si el valor es NaN.

.toBeCloseTo (n, d); verifica la precisión matemática (número de decimales).

.toContain(y); verifica si el valor actual contiene el esperado.

© JMA 2020. All rights reserved

Matchers

.toBeTruthy(); verifica si el valor es verdadero.

.toBeFalsy(); verifica si el valor es falso.

.toBeLessThan(y); verifica si el valor actual es menor que el esperado.

.toBeLessThanOrEqual (y); verifica si el valor actual es menor o igual que el esperado.

.toBeGreaterThan(y); verifica si el valor actual es mayor que el esperado.

.toBeGreaterThanOrEqual (y); verifica si el valor actual es mayor o igual que el esperado.

.toBeInstanceOf(Class); verifica que es instancia de cierta clase.

.toThrow(); verifica si una función lanza una excepción.

.toThrow(error?); verifica si una función lanza una excepción específica.

© JMA 2020. All rights reserved

Pruebas de instantáneas

- La prueba de instantáneas son de gran utilidad cuando se quiere asegurar que la UI no cambia inesperadamente. Se puede tomar un enfoque similar cuando se trata de comprobar múltiples propiedades o la salida HTML de los componentes.

```
it('will fail every time', () => {
  const actual = {id: Math.floor(Math.random() * 20), name: 'LeBron James', createdAt: new Date(), };
  expect(actual).toMatchSnapshot();
});
```

- La primera vez que se ejecuta esta prueba, Jest crea un archivo de instantánea en el subdirectorio __snapshots__, añadiendo la extensión .snap al nombre del fichero de pruebas, con una entrada por cada especificación que utilice toMatchSnapshot:

```
exports[`will fail every time 1`] = `
{
  "id": 3,
  "name": "LeBron James",
  "createdAt": 2018-05-19T23:36:09.816Z,}
`;
```

© JMA 2020. All rights reserved

Pruebas de instantáneas

- El artefacto de la instantánea debe estar incluido (commit) con los cambios en el código, y revisado como parte de la revisión de código (merge). En ejecuciones de prueba posteriores, Jest comparará la salida renderizada con la instantánea anterior. Si coinciden, pasará la prueba. Si no coinciden, falla la expectativa, por lo que el código que debe corregirse, o falla la prueba porque la implementación ha cambiado y la instantánea debe actualizarse. Similar al merge de Git, cuando falla la expectativa, se muestran las discrepancias entre Snapshot (esperado) y Received (actual). Las instantáneas fallidas pueden actualizarse de forma interactiva en el modo de observación o volver a generar con:
  - jest --updateSnapshot
- A menudo, hay campos en el objeto del que se desea obtener una instantánea que se generan (como identificadores y fechas) pero que producirían un fallo en la instantánea en cada ejecución. Para estos casos, Jest permite proporcionar un comparador asimétrico para cualquier propiedad que se comprueban antes de escribir o probar la instantánea y luego se guardan en el archivo de instantánea en lugar del valor recibido:

```
expect(user).toMatchSnapshot({id: expect.any(Number), createdAt: expect.any(Date),});
```

- Las instantáneas en línea se comportan de manera idéntica a las instantáneas externas (.snap), excepto que los valores de las instantáneas se escriben automáticamente en el código fuente.

```
expect(tree).toMatchInlineSnapshot(`…`)
```

© JMA 2020. All rights reserved

Código asincrónico

- Es común en JavaScript ejecutar código de forma asincrónica. En estos casos, Jest debe saber cuando ha terminado el código que se está probando, antes de que puede pasar a otra test.
- Jest tiene varias formas de manejar esto.
  - Devoluciones de llamada (Callbacks)
  - Promesas
  - Async / Await

© JMA 2020. All rights reserved

Devoluciones de llamada (Callbacks)

- El patrón asincrónico más común son las devoluciones de llamada: pasar como argumento la función que se desea ejecutar cuando termine el proceso asíncrono.
- El problema es que la prueba se completará tan pronto como se complete la llamada asíncrona, antes de llamar a la devolución de llamada.
- Hay una forma alternativa de test que soluciona este problema: definir como parámetro (por convenio denominada done) con la función culla llamada Jest esperará para dar como finalizada la prueba.

```
test('the data is peanut butter', done => {
  function callback(data) {
    try {
      expect(data).toBe('peanut butter');
      done();
    } catch (error) { done(error); }
  }
  fetchData(callback);
});
```

- Si done() nunca se llama, la prueba fallará (con un error de tiempo de espera), que es lo que se espera que suceda. Si la declaración expect falla, arroja un error y done() no se llama por lo que hay que invocarla en un bloque try.

© JMA 2020. All rights reserved

Promesas

- Si el código usa promesas, existe una forma más sencilla de manejar las pruebas asincrónicas.
- Si la función test devuelve una promesa, Jest esperará a que esa promesa se resuelva. Si se rechaza la promesa, la prueba fallará automáticamente.

```
test('the fetch fails with an error', () => {
  expect.assertions(1); // verificar que invoca la aserción
  return fetchData().catch(e => expect(e).toMatch('error'));
});
```

- También se puede utilizar los marcadores .resolves y .rejects en la declaración de "expect" y Jest esperará a que esa promesa resuelva o rechace.

```
test('the data is peanut butter', () => {
  return expect(fetchData()).resolves.toBe('peanut butter');
});
```

© JMA 2020. All rights reserved

Async / Await

- Como sintaxis alternativa a las promesas se puede usar async/await en las pruebas. Para escribir una prueba asíncrona, hay que usar la palabra clave async delante de la función pasada a test.

```
test('the data is peanut butter', async () => {
  const data = await fetchData();
  expect(data).toBe('peanut butter');
});
test('the fetch fails with an error', async () => {
  expect.assertions(1);
  try {
    await fetchData();
  } catch (e) {
    expect(e).toMatch('error');
  }
});
```

© JMA 2020. All rights reserved

Montaje y desmontaje

- Para montar el escenario de pruebas suele ser necesario definir e inicializar un conjunto de variables. Para evitar la duplicidad de código y mantener las variables inicializadas en un solo lugar además de mantener la modularidad, Jest suministra las funciones globales, a nivel de fichero, o local, dentro de una suite:
  - beforeAll(fn) se ejecuta solo una vez antes empezar a ejecutar las especificaciones del “describe”.
  - beforeEach(fn) se ejecuta antes de cada especificación dentro del “describe”.
  - afterEach(fn) se ejecuta después de cada especificación dentro del “describe”.
  - afterAll(fn) se ejecuta solo una vez después de ejecutar todas las especificaciones del “describe”.

```
describe("operaciones aritméticas", function(){
    let cal;
    beforeEach(function(){ calc = new Calculadora(); });
    it("adicion", function(){ expect(calc.suma(4)).toEqual(4); });
    it("multiplicacion", function(){ expect(calc.multiplica(7)).toEqual(0); });
    // …
```

- Otra manera de compartir las variables entre una beforeEach, test y afterEach es a través de la palabra clave this. Cada expectativa beforeEach/test/afterEach tiene el mismo objeto vacío this que se restablece de nuevo a vacío para de la siguiente expectativa beforeEach/test/afterEach.

© JMA 2020. All rights reserved

Control de ejecución

- El marcador .**skip** permite desactivar las suites y las especificaciones, estas se omiten cuando se ejecutan las pruebas y aparecerán como skipped entre los resultados de la prueba.
- De igual forma, las especificaciones pendientes de implementar se pueden marcar con .**todo** (solo texto, sin función) y también aparecerán como skipped.
- El marcador .**only** permite limitar las pruebas que se ejecutan a determinadas suites y especificaciones, el resto se omiten cuando se ejecutan las pruebas y aparecerán como skipped entre los resultados de la prueba. Si una suite .only no tiene ninguna especificación .only se ejecutan todas sus especificaciones, pero si tiene alguna solo ellas se ejecutaran. Una especificación .only se ejecuta siempre aunque su suite este desactivada.

```
describe.skip('Pruebas de expect', () => {
  test.todo('Pendiente...');
  test.only('null', () => {
```

© JMA 2020. All rights reserved

Pruebas parametrizadas

- Las pruebas parametrizadas permiten ejecutar una prueba, suite o especificación, varias veces con diferentes argumentos.
- Se declaran con el marcador .each asociado a una matriz que proporcionará los argumentos para cada invocación, una fila por cada juego de argumentos, a utilizar en los parámetros de la función de prueba.
- Se pueden generar títulos de prueba únicos mediante la inyección posicional de parámetros con formato printf:

```
describe.each([
  [1, 1, 2],  [1, 2, 3],  [2, 1, 3],
])('.add(%i, %i)', (a, b, expected) => {
  test(`returns ${expected}`, () => {
    expect(a + b).toBe(expected);
  });
  test(`returned value not be greater than ${expected}`, () => {
    expect(a + b).not.toBeGreaterThan(expected);
  });
  test(`returned value not be less than ${expected}`, () => {
    expect(a + b).not.toBeLessThan(expected);
  });
});
```

© JMA 2020. All rights reserved

Mock

- Jest tiene funciones dobles de prueba llamados mock, también se conocen como "espías", porque permiten espiar el comportamiento de una función que es llamada indirectamente por algún otro código, en lugar de solo probar la salida.
- Se puede crear una función simulada con jest.fn(). Si no se proporciona una implementación, la función simulada devolverá undefined cuando se invoca.

```
const mockCallback = jest.fn(x => 42 + x);
```

- Las funciones mock también se pueden usar para inyectar valores de retorno simulados durante una prueba:

```
const myMock = jest.fn();
myMock.mockReturnValueOnce(false).mockReturnValueOnce(true);
const result = [11, 12, 15].filter(num => myMock(num));
expect(result).toHaveLength(2); // [12, 15]
expect(result[0]).toBe(12);
```

© JMA 2020. All rights reserved

Valores de retorno

- .mockReturnValue(value) acepta un valor que se devolverá siempre que se llame a la función simulada, pero con .mockReturnValueOnce(value) solo se devolverá el valor para una llamada a la función simulada. Se pueden encadenar para que las sucesivas llamadas a la función simulada devuelvan valores diferentes.

```
const myMockFn = jest.fn()
  .mockReturnValue('default')
  .mockReturnValueOnce('first call')
  .mockReturnValueOnce('second call’);
```

- Si solo cuenta con .mockReturnValueOnce, el último fijara el valor a devolver en las siguientes llamadas.
- .mockResolvedValue(value),.mockResolvedValueOnce(value), .mockRejectedValue(value) y .mockRejectedValueOnce(value) son versiones con azúcar sintáctica para devolver promesas.

```
test.only('async test', async () => {
    const asyncMock = jest.fn().mockResolvedValue(43);
    let rslt = await asyncMock(); // 43
    expect(rslt).toBe(43);
});
```

© JMA 2020. All rights reserved

Seguimiento de llamadas

- Todas las funciones simuladas tienen la propiedad especial .mock, que es donde se guardan los datos sobre cómo se ha llamado a la función y qué devolvió. La propiedad .mock también rastrea el valor de this para cada llamada.
  - .calls: Una matriz que contiene arrays con los argumentos de llamada pasados en cada invocación a la función simulada.
  - .results: Una matriz que contiene los resultados de cada llamada: un objeto con las propiedades type (return, throw, incomplete) y value.
  - .instances: Una matriz que contiene todas las instancias que se han creado utilizando la función simulada con new (como función constructora).
- Las propiedades de .mock son muy útiles en las pruebas para afirmar cómo se llaman, instancian o devuelven estas funciones.

```
expect(someMockFunction.mock.calls.length).toBe(1);
expect(someMockFunction.mock.calls[1][0]).toBe('second exec, 'first arg');
expect(someMockFunction.mock.results[0].value).toBe('return value');
expect(someMockFunction.mock.instances.length).toBe(2);
expect(someMockFunction.mock.instances[0].name).toEqual('test');
```

© JMA 2020. All rights reserved

Seguimiento de llamadas

- Hay comparadores (matchers) especiales para interactuar con los espías.
  .toHaveBeenCalled() pasará si el espía fue llamado.
  .toHaveBeenCalledTimes(n) pasará si el espía se llama el número de veces especificado.
  .toHaveBeenCalledWith(…) pasará si la lista de argumentos coincide con alguna de las llamadas grabadas a la espía.
  .toHaveBeenNthCalledWith(nthCall, arg1, arg2, ...) pasará si n llamada se invoco con determinados argumentos.
  .toHaveReturned() pasará si el espía devolvió con éxito (es decir, no arrojó un error) al menos una vez.
  .toHaveReturnedTimes(number) para asegurarse de que el espía devuelve correctamente (es decir, no arroje un error) un número exacto de veces.
  .toHaveReturnedWith(value) pasará si el espía devolvió un valor específico.
  .toHaveLastReturnedWith(esperado): pasará si el espía devolvió el valor en la última llamada.
  .toHaveNthReturnedWith(nthCall, value) para probar el valor específico que devolvió el espía para la enésima llamada.

© JMA 2020. All rights reserved

Cambiar comportamiento

- Un mock puede interceptar cualquier función y hacer un seguimiento a las llamadas y todos los argumentos.

```
let c = new Calc();
const spy = jest.spyOn(c, 'sum');
spy.mockReturnValue(4);
expect(c.sum(1,2)).toBe(4);
expect(spy).toBeCalled();
```

- Se pueden interceptar módulos completos:

```
import axios from 'axios';
import UsersRest from './users';
jest.mock('axios');
test('should fetch users', () => {
  const users = [{name: 'Bob’}];
  axios.get.mockResolvedValue({data: users});
  return UsersRest.getAll().then(data => expect(data).toEqual(users));
});
```

© JMA 2020. All rights reserved

Otras opciones

- Opcionalmente, se puede proporcionar un nombre para las funciones simuladas, que se mostrarán en lugar de "jest.fn ()" en la salida de error de la prueba.

```
const spy = jest.spyOn(c, 'sum').mockName('Mock sum');
```

- jest.isMockFunction(fn) determina si la función dada es una función simulada.
- jest.clearAllMocks() borra las propiedades mock.calls y mock.instances de todos los simulacros.
- jest.resetAllMocks() restablece el estado de todos los simulacros.
- jest.restoreAllMocks() restaura todas las imitaciones a su valor original.
- mockFn.mockClear(), mockFn.mockReset() y mockFn.mockRestore() lo realiza a nivel de funciones simuladas individuales.

© JMA 2020. All rights reserved

Pruebas de integración

- El código que manipula directamente el DOM suele considerarse difícil de probar. Cargar y ejecutar el código en un navegador suele ser complejo y lento.
- jsdom es una implementación en JavaScript puro de muchos estándares web, en particular los estándares WHATWG DOM y HTML, y algunas API del navegador que se ejecuta en Node.js. En general, el objetivo del jsdom es emular una parte suficiente de un navegador web para que sea útil para hacer las pruebas de integración sin tener que abrir un navegador real.
- También hay un paquete llamado global-jsdom que se puede usar para configurar el entorno global para simular las API del navegador: inyecta document y window en el entorno Node.js.
- Para comenzar a utilizar el entorno de prueba JSDOM, debe instalarse el paquete:
  - npm install --save-dev jest-environment-jsdom

© JMA 2020. All rights reserved

Pruebas de integración

```
jest.mock('../fetchCurrentUser');

test('displays a user after a click', () => {
  document.body.innerHTML =  // Set up our document body
    '<div>' +
    '  <span id="username" />' +
    '  <button id="button" />' +
    '</div>';
  require('../displayUser'); // This module has a side-effect
  const $ = require('jquery');
  const fetchCurrentUser = require('../fetchCurrentUser');
  // Tell the fetchCurrentUser mock function to automatically invoke
  // its callback with some data
  fetchCurrentUser.mockImplementation(cb => {
    cb({ fullName: 'Johnny Cash', loggedIn: true, });
  });
  $('#button').click(); // Use jquery to emulate a click on our button
  expect(fetchCurrentUser).toHaveBeenCalled();
  expect($('#username').text()).toBe('Johnny Cash - Logged In');
});
```

© JMA 2020. All rights reserved

Cobertura de código

- Con Jest podemos ejecutar pruebas unitarias y crear informes de cobertura de código. Los informes de cobertura de código nos permiten ver que parte del código ha sido o no probada adecuadamente por nuestras pruebas unitarias.
- Para generar el informe de cobertura:

```
"scripts": {
  :
  "test": "react-scripts test --collectCoverage"
},
```

- Una vez que las pruebas se completen, aparecerá una nueva carpeta /coverage en el proyecto. Si se abre el archivo index.html en el navegador se debería ver un informe con el código fuente y los valores de cobertura del código.
- Usando los porcentajes de cobertura del código, podemos establecer la cantidad de código (instrucciones, líneas, caminos, funciones) que debe ser probado. Depende de cada organización determinar la cantidad de código que deben cubrir las pruebas unitarias.

© JMA 2020. All rights reserved

Cobertura de código

- Para establecer un mínimo de 80% de cobertura de código cuando las pruebas unitarias se ejecuten en el proyecto:

```
"jest": {
  "displayName": "Curso de React",
  "coverageThreshold": {
    "global": { "branches": 80, "functions": 80, "lines": 80, "statements": 80 }
  }
```

- Si se especifican rutas al lado global, los datos de cobertura para las rutas coincidentes se restarán de la cobertura general y los umbrales se aplicarán de forma independiente.

```
"coverageThreshold": {
    "global": { "branches": 50,"functions": 50,"lines": 50,"statements": 50 },
    "./src/components/": { "branches": 40,"statements": 40 },
    "./src/reducers/**/*.js": { "statements": 90 },
    "./src/api/critical.js": { "branches": 100,"functions": 100,"statements": 100 }
  }
```

© JMA 2020. All rights reserved

MÉTRICAS PARA EL PROCESO DE PRUEBAS DE SOFTWARE

© JMA 2020. All rights reserved

Cobertura

- La cobertura es la relación entre lo que se puede probar y lo que realmente se prueba con el equipo de prueba.
- La cobertura es un concepto útil que ayuda a los evaluadores con preguntas complejas, como:
  - Dado que es imposible probarlo todo y, por lo tanto, nos vemos obligados a probar solo un subconjunto de todas las posibilidades, ¿cuál es el mejor subconjunto?
  - ¿Cuál es la diferencia entre "pruebas elementales" y "pruebas exhaustivas"? ¿Qué significa concretamente para los casos de prueba que necesitamos para esto?
- La cobertura tiene mucho que ver con el deseo de encontrar la mayor cantidad de defectos posibles con el menor número de casos de prueba posibles.
- En lugar de simplemente probar "cualquier" subconjunto de opciones, nuestro objetivo es compilar un conjunto de casos de prueba que creen el mayor cambio posible para encontrar los defectos que existen.

© JMA 2020. All rights reserved

Tipo e índice de cobertura

- El tipo de cobertura es la forma en que se expresa la cobertura de las situaciones de prueba deducibles de la base de la prueba. Indica qué tipo de posibilidades se distinguen y están involucradas en las situaciones de prueba.
- El índice de cobertura es el porcentaje de situaciones de prueba, según lo definido por el tipo de cobertura, que cubre la prueba. Indica cuántas de estas posibilidades se prueban y generalmente se expresa como porcentaje. Muestra qué parte de todas las alternativas realmente se han probado. El número (porcentaje) solo tiene significado cuando se asocia con el tipo de cobertura.
- No podemos hablar de la cobertura, sino de muchas formas o tipos de cobertura. Los diferentes tipos de cobertura tienden a complementarse en lugar de reemplazarse entre sí, NO PUEDEN compararse en términos de "X es mejor que Y", las pruebas más exhaustivas debe realizarse seleccionando varios tipos de cobertura. La elección de los tipos de cobertura depende de dónde se ubican los mayores riesgos y que tipos se adaptan mejor a dichas zona.
- Los tipos de cobertura se pueden dividir en cuatro grupos de cobertura básicos, las técnicas de diseño de pruebas permiten encontrar los casos de pruebas de un grupo concreto.

© JMA 2020. All rights reserved

Grupos de tipos cobertura

- **Proceso**: Los procesos se pueden identificar en varios niveles. Existen algoritmos de control de flujos, procesos de negocio. Se pueden utilizar tipos de cobertura como caminos, cobertura de declaraciones y transiciones de estado para probar estos procesos.
- **Condiciones**: En casi todos los sistemas existen puntos de decisión que consisten en condiciones en las que el comportamiento del sistema puede ir en diferentes direcciones, dependiendo del resultado de dicho punto de decisión. Las variaciones de estas condiciones y sus resultados se pueden probar utilizando tipos de cobertura como cobertura de decisión, cobertura de decisión/condición modificada y cobertura de condición múltiple.
- **Datos**: Los datos se crean y se eliminan cuando finalizan. En el medio, los datos se utilizan consultándolos o actualizándolos. Se puede probar este ciclo de vida de los datos, pero también combinaciones de datos de entrada, así como los atributos (dominios) de los datos de entrada o de salida. Algunos tipos de cobertura aquí son valores límite, CRUD, flujos de datos y sintaxis.
- **Apariencia**: Cómo opera un sistema, cómo se desempeña, cuál debe ser su apariencia, a menudo se describe en requisitos no funcionales. Dentro de este grupo encontramos tipos de cobertura como perfiles operativos, de carga y presentación.

© JMA 2020. All rights reserved

Métricas de código

- La mayor complejidad de las aplicaciones de software moderno también aumenta la dificultad de hacer que el código confiable y fácil de mantener.
- Las métricas de código son un conjunto de medidas de software que proporcionan a los programadores una mejor visión del código que están desarrollando.
- Aprovechando las ventajas de las métricas del código, los desarrolladores pueden entender qué tipos o métodos deberían rehacerse o hacer más pruebas.
- Los equipos de desarrollo pueden identificar los posibles riesgos, comprender el estado actual de un proyecto y realizar un seguimiento del progreso durante el desarrollo de software. Los desarrolladores pueden generar datos de métricas de código con que medir la complejidad y el mantenimiento del código administrado.
- Se insiste mucho en que la cobertura de código de test unitarios de los proyectos sea lo más alta posible, pero es evidente que cantidad (de test, en este caso) no siempre implica calidad, la calidad no se puede medir "al peso", y es la calidad lo que realmente importa. El criterio de la cobertura de código se basa en la suposición que a mayor cantidad de código ejecutada por las pruebas, menor la probabilidad de que el código presente defectos.

© JMA 2020. All rights reserved

Contadores de cobertura

- Instrucciones (Cobertura C0) - Instructions counters
  - Los recuentos de unidades más pequeñas son instrucciones de código de un solo byte de Java. La cobertura de instrucciones proporciona información sobre la cantidad de código que se ha ejecutado o se ha perdido. Esta métrica es completamente independiente del formato de origen y siempre está disponible, incluso en ausencia de información de depuración en los archivos de clase.
- Bifurcaciones (Cobertura C1) - Branches counters
  - También calcula la cobertura de bifurcaciones para todos los if y switch. Esta métrica cuenta el número total de tales ramas en un método y determina el número de ramas ejecutadas o perdidas. La cobertura de bifurcaciones siempre está disponible, incluso en ausencia de información de depuración en los archivos de clase. El manejo de excepciones no se considera ramas en el contexto de esta definición de contador.
    - Sin cobertura: No se han ejecutado ramas en la línea (diamante rojo)
    - Cobertura parcial: Solo se ha ejecutado una parte de los ramales de la línea (diamante amarillo)
    - Cobertura total: se han ejecutado todas las ramas de la línea (diamante verde)

© JMA 2020. All rights reserved

Contadores de cobertura

- Líneas - Lines counters
  - Para todos los archivos de clase que se han compilado con información de depuración, se puede calcular la información de cobertura para líneas individuales. Una línea fuente se considera ejecutada cuando se ha ejecutado al menos una instrucción asignada a esta línea.
    - Sin cobertura: no se ha ejecutado ninguna instrucción en la línea (fondo rojo)
    - Cobertura parcial: solo se ha ejecutado una parte de la instrucción en la línea (fondo amarillo)
    - Cobertura total: se han ejecutado todas las instrucciones de la línea (fondo verde)
- Métodos - Methods counters
  - Cada método no abstracto contiene al menos una instrucción. Un método se considera ejecutado cuando se ha ejecutado al menos una instrucción. Como funciona a nivel de código de bytes, también los constructores y los inicializadores estáticos se cuentan como métodos. Es posible que algunos de estos métodos no tengan una correspondencia directa en el código fuente de Java, como constructores o inicializadores implícitos.
- Clases - Type counters
  - Una clase (o interfaz con código) se considera ejecutada cuando se ha ejecutado al menos uno de sus métodos (incluidos constructores e inicializadores).

© JMA 2020. All rights reserved

Contadores de cobertura

- Complejidad ciclomática
  - También se calcula la complejidad ciclomática para cada método no abstracto y se resume para clases, paquetes y grupos. La complejidad ciclomática es el número mínimo de rutas pasar por todas las instrucciones. Por lo tanto, el valor de complejidad puede servir como una indicación del número de casos de prueba unitarios para cubrir por completo el código.
  - La definición formal de la complejidad ciclomática se basa en la representación del gráfico de flujo de control de un método como un grafo dirigido:
    - Complejidad = [número de aristas] – [número de nodos]+ 2
  - El contador calcula la complejidad ciclomática de un método con la ecuación equivalente:
    - Complejidad = [número de ramas] – [número de puntos de decisión] + 1
  - Según el estado de cobertura de cada bifurcación, también se calcula la complejidad cubierta y perdida para cada método. La complejidad perdida nuevamente es una indicación de la cantidad de casos de prueba que faltan para cubrir completamente un módulo. Dado que no se considera el manejo de excepciones como bifurcaciones, los bloques try / catch de las ramas tampoco aumentarán la complejidad.

© JMA 2020. All rights reserved

Cobertura de código

- La cobertura de código se cuenta en bloques.
- Un bloque es un fragmento de código con un punto de entrada y de salida exactamente.
- Si el flujo de control del programa pasa a través de un bloque durante una serie de pruebas, ese bloque se cuenta como cubierto.
- El número de veces que se utiliza el bloque no tiene ningún efecto en el resultado.
- También se pueden mostrar los resultados en líneas eligiendo Agregar o quitar columnas en el encabezado de tabla.
- Algunos usuarios prefieren un recuento de líneas porque los porcentajes corresponden más al tamaño de los fragmentos que aparece en el código fuente.
- Un bloque grande de cálculo contaría como un único bloque aunque ocupe muchas líneas.

© JMA 2020. All rights reserved

Calidad de las pruebas

- Se insiste mucho en que la cobertura de test unitarios de los proyectos sea lo más alta posible, pero es evidente que cantidad (de test, en este caso) no siempre implica calidad, la calidad no se puede medir "al peso", y es la calidad lo que realmente importa.
- La cobertura de prueba tradicional (líneas, instrucciones, rama, etc.) mide solo qué código ejecuta las pruebas. No comprueba que las pruebas son realmente capaces de detectar fallos en el código ejecutado, solo pueden identificar el código que no se ha probado.
- Los ejemplos más extremos del problema son las pruebas sin afirmaciones (poco comunes en la mayoría de los casos). Mucho más común es el código que solo se prueba parcialmente, cubrir todo los caminos no implica ejercitar tolos las clases de equivalencia y valores límite.
- La calidad de las pruebas también debe ser puesta a prueba: No serviría de tener una cobertura del 100% en test unitarios, si no son capaces de detectar y prevenir problemas en el código.
- La herramienta que testea los test unitarios son los test de mutaciones: Es un test de los test.

© JMA 2020. All rights reserved

Pruebas de mutaciones

- Los pruebas de mutaciones son las pruebas de las pruebas unitarias y el objetivo es tener una idea de la calidad de las pruebas en cuanto a fiabilidad.
- Su funcionamiento es relativamente sencillo: la herramienta que se utilice debe generar pequeños cambios en el código fuente. A estos pequeños cambios se les conoce como mutaciones y crean mutantes.
- Una vez creados los mutantes, se lanzan todos los tests:
  - Si los test unitarios fallan, es que han sido capaces de detectar ese cambio de código. A esto se le llama matar al mutante.
  - Si, por el contrario, los test unitarios pasan, el mutante sobrevive y la fiabilidad (y calidad) de los tests unitarios queda en entredicho.
- Los test de mutaciones presentan informes del porcentaje de mutantes detectados: cuanto más se acerque este porcentaje al 100%, mayor será la calidad de nuestros test unitarios.

© JMA 2020. All rights reserved

Pruebas de mutaciones

- Los mutantes cuyo comportamiento es siempre exactamente igual al del programa original se los llama mutantes funcionalmente equivalentes o, simplemente, mutantes equivalentes, y representan “ruido” que dificulta el análisis de los resultados.
- Para poder matar a un mutante:
  - La sentencia mutada debe estar cubierta por un caso de prueba.
  - Entre la entrada y la salida debe crearse un estado intermedio erróneo.
  - El estado incorrecto debe propagarse hasta la salida.
- La puntuación de mutación para un conjunto de casos de prueba es el porcentaje de mutantes no equivalentes muertos por los datos de prueba:
  - Mutación Puntuación = 100 * D / (N - E)

  donde D es el número de mutantes muertos, N es el número de mutantes y E es el número de mutantes equivalentes

© JMA 2020. All rights reserved

StrykerJS

- StrykerJS permite realizar pruebas de mutación para JavaScript y compañia. Stryker comenzó como un marco de prueba de mutación de JavaScript puro, por lo que se lo conocía como Stryker, pero desde entonces se renombró como StrykerJS.
- Hoy en día, StrykerJS admite la mayoría de los proyectos de JavaScript, incluidos TypeScript, React, Angular, VueJS, Svelte y NodeJS con las pruebas en jest, mocha, jasmine, cucumber, vitest, karma o tap.
- La forma más sencilla de comenzar a utilizar Stryker en un proyecto es mediante el comando.
  - npm init stryker
- Este comando primero instalará Stryker y luego ejecutará el inicializador de Stryker. Durante la inicialización, se le harán una serie de preguntas para ayudarlo a configurar Stryker para su proyecto. Una vez finalizada la inicialización, se puede personalizar en el archivo stryker.config.mjs.
- Matemos a algunos mutantes:
  - npx stryker run
  - npx stryker run --logLevel trace
- Una vez ejecutado, se emitirá un informe html que mostrará visualmente el proyecto y todas las mutaciones. Con la opción --reporter se emitirá en: "cleartext", "cleartexttree", "json", "markdown", … Se puede integrar el resultado en paneles de control.

© JMA 2020. All rights reserved

SUPERTEST

© JMA 2020. All rights reserved

SuperTest

- Este módulo proporciona una abstracción de alto nivel para probar HTTP y, al mismo tiempo, permitirle acceder a la API de nivel inferior proporcionada por el módulo superagente. SuperTest funciona con cualquier marco de prueba. Para su instalación mediante npm:
  - npm install --save-dev supertest @types/supertest
- Puede pasar a request() un http.Server o un función, si el servidor aún no está escuchando las conexiones, entonces está vinculado a un puerto efímero, por lo que no es necesario realizar un seguimiento de los puertos.

```
const request = require('supertest');
const assert = require('assert');
const express = require('express');
const app = express();
app.get('/user', function(req, res) { res.status(200).json({ name: 'john' }); });
request(app).get('/user')
  .expect('Content-Type', /json/)
  .expect(200)
  .end(function(err, res) { if (err) throw err;  });
```

© JMA 2020. All rights reserved

Petición

- Se puede iniciar una solicitud invocando el método apropiado (.get(), .post(), …) en el objeto request al que se le pasa la url de destino y luego llamando a .then()/.catch() o con sintaxis async/await para enviar la solicitud.

```
request.get('/fake')
    .then(res => {
      // res.body, res.headers, res.status
    })
    .catch(err => {
      // err.message, err.response
    });
```

- Con el método final .end() también se inicia la solicitud y admiten devoluciones de llamada al estilo antiguo, pero no se recomiendan. Si lanza un error la expectativa falla.

```
request('GET', '/fake').end(function(err, res) {
    if (err) throw err;
    // res.ok, res.body, res.headers, res.status
});
```

© JMA 2020. All rights reserved

Configurar la petición

- Para configurar los campos del encabezado se invoca .set() con el nombre y el valor de campo:

```
.set('authorization', token)
```

- El método .query() acepta objetos que formarán una cadena de consulta:

```
.query({ page: 1, rows: 20 })
```

- Para rellenar el cuerpo de la solicitud se invoca .send() y se añade el encabezado Content-Type adecuado:

```
.set('Content-Type', 'application/json')
.send({ id: 0, name: "Nuevo" })
```

- El encabezado Content-Type es opcional, si .send() recibe un objeto, lo serializa a JSON y establecerá el Content-Type a application/json. Si recibe una cadena establecerá el Content-Type en application/x-www-form-urlencoded, y se concatenarán varias llamadas con &:

```
.send('id=0')
.send('name=Nuevo')
// id=0&name=Nuevo
```

© JMA 2020. All rights reserved

Negociar contenidos

- Los formatos son extensibles, sin embargo, de forma predeterminada se admiten "json" y "form".
- Como abreviatura, el método .type() también está disponible, aceptando el nombre del tipo MIME completo con tipo/subtipo, o simplemente el nombre de la extensión como "xml", "json", "png", etc.:

```
.type('application/json')
.type('png')
```

- De manera similar al método .type(), también es posible configurar el encabezado Accept mediante el método abreviado .accept(). Lo cual hace referencia request.types y también permite especificar el nombre de tipo MIME completo como type/subtype, o el formato del sufijo de extensión como "xml", "json", "png", etc. para mayor comodidad:

```
.accept('application/json')
.accept('json')
```

- Si se desea enviar la carga útil en un formato personalizado, se puede reemplazar la serialización incorporada con el método .serialize() por solicitud.

© JMA 2020. All rights reserved

Propiedades de respuesta

- La propiedad res.text contiene la cadena del sin analizar.
- Se des serializan automáticamente los datos del cuerpo de la respuesta, actualmente admite application/json, application/x-www-form-urlencoded y multipart/form-data. El objeto analizado estará disponible a través de res.body.
- La propiedad res.header contiene un objeto de campos de encabezado analizados, nombres de campos en minúsculas como lo hace Node.

```
res.header['content-length']
```

- El encabezado de respuesta Content-Type está en mayúsculas y minúsculas especiales, deja en res.type el formato MIME y en res.charset el juego de caracteres (si lo hay). Por ejemplo, el tipo de contenido "text/html; charset=utf8" proporcionará "text/html" como res.type y la propiedad res.charset propiedad contendrá "utf8".
- La propiedad res.status contiene el código de estado de la respuesta.

© JMA 2020. All rights reserved

Expectativas

- Los métodos .expect() permiten establecer las afirmaciones que debe cumplir la respuesta. Una vez falla una no se evalúan el resto. Las expectativas se ejecutan en el orden de definición, esta característica se puede utilizar para modificar el cuerpo de la respuesta o los encabezados antes de ejecutar una aserción.

  .expect(status[, fn]): Establecer el código de estado esperado.

  ```
  .expect(200)
  ```

  .expect(field, value[, fn]): Establecer el valor esperado en una cabecera.

  ```
  .expect('Content-Type', /json/)
  ```

  .expect(body[, fn]): Establecer los valores esperados en el cuerpo.

  ```
  .expect({ id: 0, name: "Nuevo" })
  ```

  .expect(status, body[, fn]): Establecer el código de estado y los valores en el cuerpo esperados.

© JMA 2020. All rights reserved

Expectativas

- El método .expect(function(res) {}) recibe la función que realiza la verificación, se le pasa el objeto de respuesta y lanzara un error para que la expectativa falle. También permite modificar la respuesta para las siguientes expectativas.

```
.expect(res => {
  if (res.status !== 200) throw new Error("error data");
  if (!res.body.data) throw new Error("missing data");
  if (!res.body.data.length) throw new Error("empty data");
})
```

- Los métodos .expect() reciben la función done cuando el caso de prueba trabaja con promesas:

```
it('responds with json', done => {
   request(app).get('/fake')
     .expect('Content-Type', /json/)
     .expect(200, done);
});
```

© JMA 2020. All rights reserved

Casos de prueba

```
it('responds with json, inject done', done => {
  request(app).get('/fake')
    .then(res => {
      expect(res.header['content-length']).toContain('json')
      expect(res.status).toBe(200)
      done()
    }).catch(err => done(err))
});
it('responds with json, return promise', () => {
  return request(app).get('/fake')
    .then(res => {
      expect(res.header['content-length']).toContain('json')
      expect(res.status).toBe(200)
    })
});
it('responds with json, async/await', async () => {
  const res = await request(app).get('/fake')
  expect(res.header['content-length']).toContain('json')
  expect(res.status).toBe(200)
});
```

© JMA 2020. All rights reserved

https://jasmine.github.io/

JASMINE

© JMA 2020. All rights reserved

Jasmine

- Jasmine es un framework de desarrollo dirigido por comportamiento (behavior-driven development, BDD) para probar código JavaScript.
  - No depende de ninguna otra librería JavaScript.
  - No requiere un DOM.
  - Tiene una sintaxis obvia y limpia para que se pueda escribir pruebas fácilmente.
- En resumen, podríamos decir que desde que los creadores del conocido PivotalTracker sacaron a la luz este framework de test, prácticamente se ha convertido en el estándar de facto para el desarrollo con JavaScript.
- Para su instalación "standalone", descargar y descomprimir:
  - https://github.com/jasmine/jasmine/releases
- Mediante npm: (node)
  - npm install -g jasmine

© JMA 2020. All rights reserved

SpecRunner.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Jasmine Spec Runner v2.5.0</title>
  <link rel="shortcut icon" type="image/png" href="lib/jasmine-2.5.0/jasmine_favicon.png">
  <link rel="stylesheet" href="lib/jasmine-2.5.0/jasmine.css">
  <script src="lib/jasmine-2.5.0/jasmine.js"></script>
  <script src="lib/jasmine-2.5.0/jasmine-html.js"></script>
  <script src="lib/jasmine-2.5.0/boot.js"></script>
  <script type="text/javascript" src="angular.js"></script>
  <script type="text/javascript" src="angular-mocks.js"></script>
  <!-- include source files here... -->
  <script src="src/Player.js"></script>
  <script src="src/Song.js"></script>
  <!-- include spec files here... -->
  <script src="spec/SpecHelper.js"></script>
  <script src="spec/PlayerSpec.js"></script>
</head>
<body></body>
</html>
```

© JMA 2020. All rights reserved

Test Runner de Navegador

- Instalar el paquete como dependencia de desarrollo:
  - npm install --save-dev jasmine-browser-runner
- Inicializar para crear directorios y configuración:
  - npx jasmine-browser-runner init
- Ajustar configuración en:
  - spec/support/jasmine-browser.json
- Incorporar el comando al package.json
  - "scripts": { "test": "jasmine-browser-runner runSpecs" }
- Para ejecutar los test interactivamente:
  - npx jasmine-browser-runner serve
- Ejecutar los test:
  - npm test

© JMA 2020. All rights reserved

ESLint for Jasmine

- Instalar el paquete como dependencia de desarrollo:
  - npm install --save-dev eslint-plugin-jasmine
- Ajustar configuración en .eslintrc:

```
plugins:
  - jasmine
env:
  jasmine: true
extends: 'plugin:jasmine/recommended'
```

© JMA 2020. All rights reserved

Suites

- Una “suite” es un nombre que describe a qué género, sección o característica se va a pasar por un conjunto de pruebas unitarias, además es una herramienta que es el núcleo que se necesita para poder tener un orden al momento de crear las pruebas.
- Las “suites” se crean con la función **describe**, que es una función global y con la cual se inicia toda prueba unitaria, además consta con dos parámetros:

```
describe("Una suite es sólo una función", function() {
//…
});
```

- El primer parámetro es una cadena de caracteres donde se define el nombre de la prueba unitaria.
- El segundo parámetro es una función donde está el código que ejecutará con la prueba de código.
- Se pueden anidar “suites” para estructurar conjuntos complejos y facilitar la legibilidad y la búsqueda, basta con crear un describe dentro de otro.

© JMA 2020. All rights reserved

Especificaciones

- Una especificación es un caso de prueba, contiene una o más expectativas (algo que se espera) que ponen a prueba el estado del código. Una expectativa de Jasmine es una afirmación que debe ser verdadera pero puede ser falsa.
- Una especificación con todas las expectativas verdaderas es una especificación que pasa la prueba, pero con una o más falsas es una especificación que falla.
- Las especificaciones se definen dentro de una Suite llamando a la función global del Jasmine **it**, que al igual que describe, recibe una cadena y una función, mas un timeout opcional a cuya expiración fallara. La cadena es el título de la especificación y la función es la especificación o prueba.

```
it("y así es una especificación", function() {
//…
});
```

- **describe** y **it** son funciones: pueden contener cualquier código ejecutable necesario para implementar la prueba y se aplican las reglas de alcance de JavaScript: las variables declaradas en un describe están disponibles para cualquier bloque it dentro de la suite.

© JMA 2020. All rights reserved

Expectativas

- Las expectativas se construyen con la función expect que obtiene un valor real de una expresión y lo comparan mediante una función comparadora (matcher) con un el valor esperado (constante).

  expect(valor obtenido).matchers(valor esperado);
- Los matchers son funciones que implementan comparaciones booleanas entre el valor actual y el esperado, ellos son los responsables de informar a Jasmine si la expectativa se cumple o es falsa.
- Cualquier comparador puede evaluarse como una afirmación negativa mediante el encadenamiento a la llamada expect de un not antes de llamar al comparador.

  expect(valor obtenido)**.not**.matchers(valor esperado);
- También existe la posibilidad de escribir comparadores personalizados para cuando el dominio de un proyecto consiste en afirmaciones específicas que no se incluyen en los ya definidos.
- Cuando hay múltiples expectativas se puede documentar la expectativa concreta que falla.

  expect(valor obtenido).withContext('mensaje a mostrar').matchers(valor esperado);

© JMA 2020. All rights reserved

Comparadores

- .toEqual(y); verifica si ambos valores son iguales ==.
- .toBe(y); verifica si ambos objetos son idénticos ===.
- .toMatch(pattern); verifica si el valor pertenece al patrón establecido.
- .toBeDefined(); verifica si el valor está definido.
- .toBeUndefined(); verifica si el valor es indefinido.
- .toBeNull(); verifica si el valor es nulo.
- .toBeNaN(); verifica si el valor es NaN.
- .nothing(); no espera nada, nunca falla.
- .toBeCloseTo (n, d); verifica la precisión matemática (número de decimales).
- .toContain(y); verifica si el valor actual contiene el esperado.
- .toBeInstanceOf(tipo); verifica si es del tipo esperado.
- .toHaveClass(className); verifica que la etiqueta tenga el class esperado.
- .toHaveSize(y); verifica que la colección tenga el tamaño esperado.

© JMA 2020. All rights reserved

Comparadores

- .toBeTruthy(); verifica si el valor es verdadero.
- .toBeTrue(); verifica si el valor es estrictamente true.
- .toBeFalsy(); verifica si el valor es falso.
- .toBeFalse(); verifica si el valor es estrictamente false.
- .toBeNegativeInfinity(); verifica si el valor es infinito negativo.
- .toBePositiveInfinity(); verifica si el valor es infinito positivo.
- .toBeLessThan(y); verifica si el valor actual es menor que el esperado.
- .toBeLessThanOrEqual (y); verifica si el valor actual es menor o igual que el esperado.
- .toBeGreaterThan(y); verifica si el valor actual es mayor que el esperado.
- .toBeGreaterThanOrEqual (y); verifica si el valor actual es mayor o igual que el esperado.
- .toThrow(); verifica si la función pasada a expect lanza una excepción.
- .toThrowError(e); verifica si una función lanza una excepción específica.
- .toThrowMatching(predicate); verifica que excepción recibida cumple el predicado.

© JMA 2020. All rights reserved

Forzar fallos

- La función fail(msg) hace que una especificación falle. Puede llevar un mensaje de fallo o error de un objeto como un parámetro.

```
describe("Una especificación utilizando la función a prueba", function() {
  var foo = function(x, callBack) {
    if (x) {
      callBack();
    }
  };
  it("no debe llamar a la devolución de llamada", function() {
    foo(false, function() {
      fail("Devolución de llamada ha sido llamada");
    });
  });
});
```

© JMA 2020. All rights reserved

Montaje y desmontaje

- Para montar el escenario de pruebas suele ser necesario definir e inicializar un conjunto de variables. Para evitar la duplicidad de código y mantener las variables inicializadas en un solo lugar además de mantener la modularidad, Jasmine suministra las funciones globales :
  - beforeEach(fn) se ejecuta antes de cada especificación dentro del “describe”.
  - beforeAll(fn) se ejecuta solo una vez antes empezar a ejecutar las especificaciones del “describe”.
  - afterEach(fn) se ejecuta después de cada especificación dentro del “describe”.
  - afterAll(fn) se ejecuta solo una vez después de ejecutar todas las especificaciones del “describe”.

```
describe("operaciones aritméticas", function(){
var cal;
beforeEach(function(){
calc = new Calculadora();
});
it("adicion", function(){
expect(calc.suma(4)).toEqual(4);
});
it("multiplicacion", function(){
expect(calc.multiplica(7)).toEqual(0);
});
// …
```

- Otra manera de compartir las variables entre una beforeEach, it y afterEach es a través de la palabra clave this. Cada expectativa beforeEach/it/afterEach tiene el mismo objeto vacío this que se restablece de nuevo a vacío para de la siguiente expectativa beforeEach/it/afterEach.

© JMA 2020. All rights reserved

Desactivación parcial

- Las Suites se pueden desactivar renombrando la función describe por xdescribe. Estas suites y las especificaciones dentro de ellas se omiten cuando se ejecuta y por lo tanto sus resultados no aparecerán entre los resultados de la prueba.
- De igual forma, las especificaciones se desactivan renombrando it por xit, pero en este caso aparecen en los resultados como pendientes (pending).
- Cualquier especificación declarada sin un cuerpo función también estará marcada pendiente en los resultados.

```
it('puede ser declarada con "it", pero sin una función');
```

- Y si se llama a la función de pending en cualquier parte del cuerpo de las especificaciones, independientemente de las expectativas, la especificación quedará marcada como pendiente. La cadena que se pasa a pending será tratada como una razón y aparece cuando termine la suite.

```
it('se puede llamar a "pending"en el cuerpo de las especificaciones', function() {
  expect(true).toBe(false);
  pending('esto es por lo que está pendiente');
});
```

© JMA 2020. All rights reserved

Ejecución de pruebas específicas

- En determinados casos (desarrollo) interesa limitar las pruebas que se ejecutan. Si se pone el foco en determinadas suites o especificaciones solo se ejecutaran las pruebas que tengan el foco, marcando el resto como pendientes.
- Las Suites se enfocan renombrando la función describe por fdescribe. Estas suites y las especificaciones dentro de ellas son las que se ejecutan.
- De igual forma, las especificaciones se enfocan renombrando it por fit.
- Si se enfoca una suite que no tiene enfocada ninguna especificación se ejecutan todas sus especificaciones, pero si tiene alguna enfocada solo se ejecutaran las que tengan el foco.
- Si se enfoca una especificación se ejecutara independientemente de que su suite esté o no enfocada.
- La funciones de montaje y desmontaje se ejecutaran si la suite tiene alguna especificación con foco.
- Si ninguna suite o especificación tiene el foco se ejecutaran todas las pruebas normalmente.

© JMA 2020. All rights reserved

Pruebas asíncronas

- El código asíncrono es común y Jasmine necesita saber cuándo finaliza el trabajo asíncrono para evaluar el resultado.
- Jasmine admite dos formas de gestionar el trabajo asíncrono: basado en promesas (async/ await) o en devoluciones de llamada.

```
it('does a thing', async function() {
  const result = await someAsyncFunction();
  expect(result).toEqual(someExpectedValue);
});
```

- Cuando no se puedan utilizar promesas, si la función de la especificación, montaje o desmontaje define un argumento (tradicionalmente llamado done), Jasmine pasará una función para ser invocada cuando se haya completado el trabajo asincrónico, debe ser lo último que realice la función asincrónica o cualquiera de las funciones a las que llama para evitar errores o anomalías.

```
it('does a thing', function(done) {
  someAsyncFunction(function(result) {
    expect(result).toEqual(someExpectedValue);
    done();
  });
});
```

© JMA 2020. All rights reserved

Comparadores asíncronos

- Las expectativas sobre resultados asíncronos se pueden construir con la función expectAsync. Los comparadores asincrónicos operan con un valor obtenido que es una promesa y devuelven, a su vez, una promesa.
- La mayoría de los comparadores asíncronos esperarán indefinidamente a que la promesa se resuelva o se rechace, lo que dará como resultado la especificación falle por timeout si nunca sucede.
- Son accesibles mediate expectAsync(promesa) o expectAsync(promesa).already (cuando no esta pendiente):

```
await expectAsync(promesa).toBeResolved().then(function() {
  // more spec code
});
await expectAsync(aPromise).toBeRejected();
```

- Los comparadores asincrónicos son: toBePending(), toBeResolved(), toBeResolvedTo(expected), toBeRejected(), toBeRejectedWith(expected), toBeRejectedWithError(expected, message).
- Como cualquier comparador, pueden evaluarse como una afirmación negativa mediante el encadenamiento de un not antes de llamar al comparador.

© JMA 2020. All rights reserved

Pruebas dinámicas

- Se pueden generar pruebas dinámicamente utilizando JavaScript para crear múltiples funciones de especificación.

```
describe('Calculos', () => {
  describe('Sumas', function () {
    [[1, 2, 3], [2, 2, 4], [3, -2, 1]].forEach(item => {
      it(`Prueba que ${item[0]} mas ${item[1]} es ${item[2]}`,
        () => expect(item[0] + item[1]).toBe(item[2]))
    });
  });
})
```

© JMA 2020. All rights reserved

Dependencias

- Las dependencias con sistemas externos afectan a la complejidad de la estrategia de pruebas, ya que es necesario contar con sustitutos de estos servicios externos durante el desarrollo. Ejemplos típicos de estas dependencias son servicios web, sistemas de envío de correo, fuentes de datos o simplemente dispositivos hardware.
- Estos sustitutos, dobles de pruebas, muchas veces son exactamente iguales que el servicio original, pero en otro entorno, o son simuladores, que exponen el mismo interfaz pero realmente no realizan las mismas tareas que el sistema real, o las realizan contra un entorno controlado.
- Para poder emplear la técnica de simulación de objetos se debe diseñar el código a probar de forma que sea posible trabajar con los objetos reales o con los objetos simulados:
  - Doble herencia
  - IoC: Inversión de Control (Inversion Of Control)
  - DI: Inyección de Dependencias (Dependency Injection)
  - Simuladores de objetos

© JMA 2020. All rights reserved

Dobles de prueba

- La regla de oro de las pruebas unitarias, es que una unidad (unit) tiene que ser testeada sin utilizar ninguna de sus dependencias.
- Siguiendo la misma regla de oro, las pruebas de integración y sistema deben estar aisladas de sus dependencias salvo cuando se estén probando dichas dependencias. Así mismo, el resultado de las pruebas debe ser previsible.
- Usar dobles de prueba (como los dobles en el cine) tiene ventajas:
  - Devuelven resultados determinísticos
  - Permiten crear o reproducir determinados estados
  - Obtienen resultados mucho mas rápidamente y a menor coste, incluso offline.
  - Permiten el inicio temprano de las pruebas incluso cuando las dependencias todavía no están disponibles.
  - Permiten incluir atributos o métodos exclusivamente para el testeo.
    - Memorizan los valores con los que se llama a cada uno de sus miembros
    - Permiten verificar si los valores esperados coinciden con los recibidos

© JMA 2020. All rights reserved

Simulación de objetos

- **Fixture**: Es el término se utiliza para hablar de los datos de contexto de las pruebas, aquellos que se necesitan para construir el escenario que requiere la prueba.
- **Dummy**: Objeto que se pasa como argumento pero nunca se usa realmente. Normalmente, los objetos dummy se usan sólo para rellenar listas de parámetros.
- **Fake**: Objeto que tiene una implementación que realmente funciona pero, por lo general, usa una simplificación que le hace inapropiado para producción (como una base de datos en memoria por ejemplo).
- **Stub**: Objeto que proporciona respuestas predefinidas a llamadas hechas durante los tests, frecuentemente, sin responder en absoluto a cualquier otra cosa fuera de aquello para lo que ha sido programado. Los stubs pueden también grabar información sobre las llamadas (**spy**).
- **Mock**: Objeto preprogramado con expectativas que conforman la especificación de cómo se espera que se reciban las llamadas. Son más complejos que los stubs aunque sus diferencias son sutiles.

© JMA 2020. All rights reserved

Espías

- Jasmine tiene funciones dobles de prueba llamados espías.
- Un espía puede interceptar cualquier función y hacer un seguimiento a las llamadas y todos los argumentos.

```
beforeEach(function() {
    fnc = spyOn(calc, 'suma');
    prop = spyOnProperty(calc, 'pantalla', 'set')
});
```

- Un espía sólo existe en el bloque describe o it en que se define, y se eliminará después de cada especificación.
- Hay comparadores (matchers) especiales para interactuar con los espías.
  - .toHaveBeenCalled() pasará si el espía fue llamado.
  - .toHaveBeenCalledTimes(n) pasará si el espía se llama el número de veces especificado.
  - .toHaveBeenCalledWith(…) pasará si la lista de argumentos coincide con alguna de las llamadas grabadas a la espía.
  - .toHaveBeenCalledBefore(esperado): pasará si el espía se llama antes que el espía pasado por parámetro.

© JMA 2020. All rights reserved

Seguimiento de llamadas

- El proxi del espía añade la propiedad calls que permite:
  - all(): Obtener la matriz de llamadas sin procesar para este espía.
  - allArgs(): Obtener todos los argumentos para cada invocación de este espía en el orden en que fueron recibidos.
  - any(): Comprobar si se ha invocado este espía.
  - argsFor(índice): Obtener los argumentos que se pasaron a una invocación específica de este espía.
  - count(): Obtener el número de invocaciones de este espía.
  - first(): Obtener la primera invocación de este espía.
  - mostRecent(): Obtener la invocación más reciente de este espía.
  - reset(): Restablecer el espía como si nunca se hubiera llamado.
  - saveArgumentsByValue(): Establecer que se haga un clon superficial de argumentos pasados a cada invocación.

```
spyOn(foo, 'setBar');
expect(foo.setBar.calls.any()).toEqual(false);
foo.setBar();
expect(foo.setBar.calls.count()).toBy(1);
```

© JMA 2020. All rights reserved

Cambiar comportamiento

- Adicionalmente el proxi del espía puede añadir los siguientes comportamiento:
  - callFake(fn): Llamar a una implementación falsa cuando se invoca.
  - callThrough(): Llamar a la implementación real cuando se invoca.
  - exec(): Ejecutar la estrategia de espionaje actual.
  - identity(): Devolver la información de identificación para el espía.
  - returnValue(valor): Devolver un valor cuando se invoca.
  - returnValues(... values): Devolver uno de los valores especificados (secuencialmente) cada vez que se invoca el espía.
  - stub(): No haga nada cuando se invoca. Este es el valor predeterminado.
  - throwError(algo): Lanzar un error cuando se invoca.

```
spyOn(foo, "getBar").and.returnValue(745);
spyOn(foo, "getBar").and.callFake(function(arguments, can, be, received) {
  return 745;
});
spyOn(foo, "forbidden").and.throwError("quux");
```

© JMA 2020. All rights reserved

Reloj simulado

- Hay situaciones en las que es útil controlar el objeto date y los temporizadores para anular su comportamiento o evitar pruebas lentas:
  - Operaciones que dependen de marcas temporales obtenida a través del objeto Date.
  - Operaciones diferidas con setTimeout donde no es necesario esperar.
  - Sondeos con setInterval que se quieren acelerar.
- El reloj simulado de Jasmine se utiliza al probar el código dependiente del tiempo.
- jasmine.clock().install() anula las funciones globales nativas relacionadas con el tiempo para que puedan ser controladas sincrónicamente a través de jasmine.clock().tick(), que mueve el reloj hacia adelante, ejecutando los tiempos de espera en cola por el camino. Esto incluye controlar: setTimeout, clearTimeout, setInterval, clearInterval y el objeto Date.
- Por defecto el reloj comienza en la época de Unix (marca de tiempo de 0). Esto significa que cuando se cree una instancia new Date en la aplicación, es inicializa al 01/01/1970 00:00:00. La marca temporal inicial se puede modificar con jasmine.clock().mockDate().
- Con jasmine.clock().uninstall() se restauran los métodos integrados originales, se devuelven las funciones temporales suplantadas a su estado por defecto.

© JMA 2020. All rights reserved

Reloj simulado

- jasmine.clock().withMock() permite ejecuta una función con un reloj simulado: llamara a install antes de ejecutar la función y a uninstall después de que se complete la función.

```
beforeEach(function() {
  jasmine.clock().install();
});
afterEach(function() { jasmine.clock().uninstall(); });
it('does something after 10 seconds', function() {
   const callback = jasmine.createSpy('callback');
   doSomethingLater(callback);
   jasmine.clock().tick(10000);
   expect(callback).toHaveBeenCalledWith(12345);
 });
```

© JMA 2020. All rights reserved

Solicitudes de red

- Las solicitudes a recursos externos tienen el potencial de impactar negativamente en las ejecuciones de prueba debido a tiempos de carga lentos. En otros escenarios es difícil de obtener estados específicos del servidor, incluyendo status, headers o body de la respuesta o retrasos de la red, para poder realizar los casos de pruebas apropiados.
- Dado que las respuestas reales pasan por cada capa del servidor (controladores, modelos, vistas, etc.) y, es posible, que se deba inicializar una fuente de datos antes de cada prueba para generar un estado predecible, suelen ser muy lentas.
- Jasmine suministra un complemento llamado jasmine-ajax que permite simular las llamadas ajax en las pruebas. Para usarlo, se debe descargar el archivo mock-ajax.js y agregarlo a los ayudantes de Jasmine para que se cargue antes de cualquier especificación que lo use (https://github.com/jasmine/jasmine-ajax).
- jasmine-ajax proporciona un sustituto del objeto XMLHttpRequest intercepta las peticiones y simula la respuesta de la solicitud.
- Las respuestas simuladas pueden establecer directamente estados específicos.
- Las respuestas simuladas no requieren servidor externo ni conexión.
- Las simulaciones son extremadamente rápidas, la mayoría de las respuestas se devolverán en menos de 20 ms.

© JMA 2020. All rights reserved

Solicitudes de red

```
describe('AJAX Mock', () => {
  beforeEach(function () { jasmine.Ajax.install(); });
  afterEach(function () { jasmine.Ajax.uninstall(); });

  it("specifying response when you need it", function () {
    var doneFn = jasmine.createSpy("success");
    var xhr = new XMLHttpRequest();
    xhr.onreadystatechange = function (args) {
      if (this.readyState == this.DONE) {
        doneFn(this.responseText);
      }
    };
    xhr.open("GET", "/some/cool/url");
    xhr.send();
    expect(jasmine.Ajax.requests.mostRecent().url).toBe('/some/cool/url');
    expect(doneFn).not.toHaveBeenCalled();
    jasmine.Ajax.requests.mostRecent().respondWith({
      "status": 200, "contentType": 'text/plain', "responseText": 'awesome response'
    });
    expect(doneFn).toHaveBeenCalledWith('awesome response');
  });
});
```

© JMA 2020. All rights reserved

Depuración de pruebas

1. Seleccionar la ventana del navegador Karma.
2. Hacer clic en el botón DEBUG; se abre una nueva pestaña del navegador que permite volver a ejecutar las pruebas.
3. Abrir “Herramientas de Desarrollo” del navegador.
4. Seleccionar la sección de código fuentes.
5. Abrir el archivo con el código de prueba.
6. Establecer un punto de interrupción en la prueba.
7. Actualizar el navegador, que se detiene en el punto de interrupción.

© JMA 2020. All rights reserved

https://mochajs.org/

MOCHA

© JMA 2020. All rights reserved

Introducción

- Mocha es un marco de prueba de JavaScript rico en funciones que se ejecuta en Node.js y en el navegador, lo que hace que las pruebas asincrónicas sean simples y divertidas. Las pruebas de Mocha se ejecutan en serie, lo que permite informes flexibles y precisos, al tiempo que asigna excepciones no detectadas a los casos de prueba correctos.
- Las pruebas se pueden ejecutar sobre NodeJS y sobre navegador. Sobre NodeJS se pueden configurar para ejecutarse en paralelo.
- Para instalarlo globalmente con npm:
  - npm install --global mocha
- Para instalarlo como una dependencia de desarrollo del proyecto:
  - npm install --save-dev mocha
- Para ejecutarlo en modo continuo:
  - mocha --watch

© JMA 2020. All rights reserved

Suites

- Una “suite” es un nombre que describe al género o sección qué se va a pasar por un conjunto de pruebas unitarias, además es una herramienta que es el núcleo que se necesita para poder tener un orden en el momento de crear las pruebas.
- Las “suites” se crean con la función **describe** (o sus alias **context** y **suite**), que es una función global y con la cual se inicia toda prueba unitaria, además consta con dos parámetros:

```
describe("Una suite es sólo una función", function() {
//…
});
```

- El primer parámetro es una cadena de caracteres donde se define el nombre descriptivo de la prueba unitaria y el segundo es una función con el código que ejecutará la prueba de código.
- Se pueden anidar “suites” para estructurar conjuntos complejos y facilitar la legibilidad y la búsqueda, basta con crear un describe dentro de otro.

© JMA 2020. All rights reserved

Especificaciones

- Una especificación contiene una o más expectativas (algo que se espera) que ponen a prueba el estado del código. Una expectativa de Mocha es una afirmación que debe ser verdadera pero puede ser falsa. Una especificación con todas las expectativas verdaderas es una especificación que pasa la prueba, pero con una o más falsas es una especificación que falla.
- Las especificaciones se definen dentro de una Suite llamando a la función global del **it** (o sus alias **test** y **specify**) que, al igual que describe, recibe una cadena (el título de la especificación) y una función (la especificación o caso de prueba).

```
it("y así es una especificación", function() {
//…
});
```

- **describe** y **it** son funciones: pueden contener cualquier código ejecutable necesario para implementar la prueba y las reglas de alcance de JavaScript se aplican, por lo que las variables declaradas en un describe están disponibles para cualquier bloque test dentro de la suite. Se desaconseja pasar funciones de flecha o lambdas porque se unen léxicamente a this y no pueden acceder al contexto Mocha.

© JMA 2020. All rights reserved

Aserciones

- Mocha permite usar cualquier biblioteca de expectativas o aserciones que se desee. Por defecto se puede utilizar el módulo Assert de NodeJS. En general, si se lanza un Error dentro de la especificación entonces se marcará la prueba como fallida y como superada en caso contrario.
- Esto significa que se pueden usar bibliotecas como:
  - chai: aserciones expect(), assert() y estilo should (usada mayoritariamente)
  - unexpected: kit de herramientas de aserción BDD extensible
  - should.js: aserciones estilo BDD
  - expect.js: aserciones expect()
  - better-assert: aserciones assert() de auto-documentación estilo C

© JMA 2020. All rights reserved

Montaje y desmontaje

- Para montar el escenario de pruebas suele ser necesario definir e inicializar un conjunto de variables. Para evitar la duplicidad de código y mantener las variables inicializadas en un solo lugar además de mantener la modularidad, Mocha suministra las funciones globales, a nivel de fichero, o local, dentro de una suite:
  - before(fn) o suiteSetup(fn) se ejecuta solo una vez antes empezar a ejecutar las especificaciones del "describe".
  - beforeEach(fn) o setup(fn) se ejecuta antes de cada especificación dentro del "describe".
  - afterEach(fn) o teardown(fn) se ejecuta después de cada especificación dentro del "describe".
  - after(fn) o suiteTeardown(fn) se ejecuta solo una vez después de ejecutar todas las especificaciones del "describe".

```
describe("operaciones aritméticas", function(){
    let cal;
    beforeEach(function(){ calc = new Calculadora(); });
    it("adicion", function(){ expect(calc.suma(4)).toEqual(4); });
    it("multiplicacion", function(){ expect(calc.multiplica(7)).toEqual(0); });
    // …
```

© JMA 2020. All rights reserved

Control de ejecución

- El marcador .**skip** (o la versión xdescribe(), xit()) permite desactivar las suites y las especificaciones, estas se omiten cuando se ejecutan las pruebas y aparecerán como pending entre los resultados de la prueba.
- De igual forma, las especificaciones pendientes de implementar (solo texto, sin función) también aparecerán como pending. También puede omitir en tiempo de ejecución:

```
it('should only test in the correct environment', function() {
  if (/* check test environment */) this.skip();
```

- El marcador .**only** permite limitar las pruebas que se ejecutan a determinadas suites y especificaciones, el resto se omiten cuando se ejecutan las pruebas y aparecerán como pending entre los resultados de la prueba. Si una suite .only no tiene ninguna especificación .only se ejecutan todas sus especificaciones, pero si tiene alguna solo ellas se ejecutaran. Una especificación .only se ejecuta siempre aunque su suite este desactivada.

```
describe.skip('Pruebas de expect', function() {
  it('Pendiente...');
  it.only('null', function() {
```

© JMA 2020. All rights reserved

Reintentar pruebas

- Se puede optar por volver a intentar las pruebas fallidas hasta un cierto número de veces. Esta característica está diseñada para manejar pruebas de extremo a extremo (pruebas funcionales, Selenium...) donde las dependencias no se pueden simular/mockear fácilmente. No se recomienda utilizar reintentos para pruebas unitarias.

```
describe('retries', function () {
  this.retries(4); // reintenta todas las pruebas de esta suite hasta 4 veces
  beforeEach(function () { browser.get('http://www.yahoo.com'); });
  it('should succeed on the 3rd try', function () {
    this.retries(2); // reintenta esta prueba hasta 2 veces
    expect($('.foo').isDisplayed()).to.eventually.be.true;
  });
});
```

- Los reintentos vuelven a ejecutar una prueba fallida y sus beforeEach/afterEach correspondientes, pero no los before/after de la suite, y no tienen ningún efecto sobre los fallos de montaje y desmontaje.

© JMA 2020. All rights reserved

Tiempos de espera

- Los tiempos de espera permiten controlar el tiempo máximo que puede tardar la ejecución de un caso de prueba para evitar bucles infinitos, cumplimientos SLA, …
- Los tiempos de espera a nivel de suite se pueden aplicar a suites de prueba completas o this.timeout(0) para deshabilitar los tiempos de espera por completo, estos tiempos serán aplicados por todas las suites anidadas, montaje/desmontaje y casos de prueba que no establezcan su propio valor.

```
describe('a suite of tests', function () {
  this.timeout(500);
  beforeEach(function (done) {
    this.timeout(3000); // A very long environment setup.
    :
  });
  it('should take less than 500ms', function (done) {
    this.timeout(500);
    :
  });
});
```

© JMA 2020. All rights reserved

Código asincrónico

- Es común en JavaScript ejecutar código de forma asincrónica. En estos casos, Mocha debe saber cuando ha terminado el código que se está probando, antes de que puede pasar a otra test.
- Mocha tiene varias formas de manejar esto.
  - Devoluciones de llamada (Callbacks)
  - Promesas
  - Async / Await

© JMA 2020. All rights reserved

Devoluciones de llamada (Callbacks)

- El patrón asincrónico más común son las devoluciones de llamada: pasar como argumento la función que se desea ejecutar cuando termine el proceso asíncrono.
- El problema es que la prueba se completará tan pronto como se complete la llamada asíncrona, antes de llamar a la devolución de llamada.
- Hay una forma alternativa de test que soluciona este problema: definir como parámetro (por convenio denominada done) con la función culla llamada Mocha esperará para dar como finalizada la prueba.

```
test('the data is peanut butter', done => {
  function callback(data) {
    try {
      expect(data).toBe('peanut butter');
      done();
    } catch (error) { done(error); }
  }
  fetchData(callback);
});
```

- Si done() nunca se llama, la prueba fallará (con un error de tiempo de espera), que es lo que se espera que suceda. Si la declaración expect falla, arroja un error y done() no se llama por lo que hay que invocarla en un bloque try.

© JMA 2020. All rights reserved

Promesas

- Si el código usa promesas, existe una forma más sencilla de manejar las pruebas asincrónicas.
- Si la función test devuelve una promesa, Mocha esperará a que esa promesa se resuelva. Si se rechaza la promesa, la prueba fallará automáticamente.

```
beforeEach(function() {
  return db.clear().then(function() {
    return db.save([tobi, loki, jane]);
  });
});
describe('#find()', function() {
  it('respond with matching records', function() {
    return db.find({type: 'User'}).should.eventually.have.length(3);
  });
});
```

© JMA 2020. All rights reserved

Async / Await

- Como sintaxis alternativa a las promesas se puede usar async/await en las pruebas. Para escribir una prueba asíncrona, hay que usar la palabra clave async delante de la función pasada a test.

```
beforeEach(async function() {
  await db.clear();
  await db.save([tobi, loki, jane]);
});
describe('#find()', function() {
  it('responds with matching records', async function() {
    const users = await db.find({type: 'User'});
    users.should.have.length(3);
  });
});
```

© JMA 2020. All rights reserved

Pruebas parametrizadas

- Las pruebas parametrizadas permiten ejecutar una prueba, suite o especificación, varias veces con diferentes argumentos.
- Se declaran con ejecutando un .forEach sobre a una matriz que proporcionará los argumentos para cada invocación, una fila por cada juego de argumentos, a utilizar en los parámetros de la función de prueba.
- Se pueden generar títulos de prueba mediante la interpolación.

```
describe('Calculos', () => {
  describe('Sumas', function () {
    [[1, 2, 3], [2, 2, 4], [3, -2, 1]].forEach(item => {
      it(`Prueba que ${item[0]} mas ${item[1]} es ${item[2]}`, function() {
        expect(item[0] + item[1]).toBe(item[2]))
      }
    });
  });
})
```

© JMA 2020. All rights reserved

DSL

- El sistema de interfaz de Mocha permite a los desarrolladores elegir su estilo de DSL. Mocha tiene interfaces para BDD, TDD, QUnit y Require-style. Aunque son alias y se pueden utilizar indistintamente, se debe mantener la coherencia y no mezclar la terminología de las diferentes metodologías.
  - Mocha: describe(), it(), before(), after(), beforeEach() y afterEach().
  - BDD: describe(), context(), it(), specify(), before(), after(), beforeEach() y afterEach().
  - TDD: suite(), test(), suiteSetup(), suiteTeardown(), setup() y teardown()
  - QUnit: suite(), test(), before(), after(), beforeEach() y afterEach().
- La interfaz require permite crear un DSL personalizado:

```
var testSuite = require('mocha').describe;
var pre = require('mocha').before;
var testCase = require('mocha').it;
testSuite('Array', function () {
  pre(function () { … });
  testCase ('indexOf() should return -1 when not present', function () {
```

© JMA 2020. All rights reserved

Informes

- Los informes de Mocha se ajustan a la ventana de la terminal y siempre deshabilitan la coloración de escape ANSI cuando las transmisiones de stdio no están asociadas con un TTY, se pueden exportar a ficheros. Mocha le permite definir informes personalizados.
  - mocha --reporter=json --reporter-option output=filename.json
- Los informes disponibles son:
  - spec: Este es el informe predeterminado. El informe de especificaciones genera una vista jerárquica anidada al igual que los casos de prueba.
  - list: El informe de lista genera una lista de especificaciones simples a medida que los casos de prueba pasan o fallan, mostrando los detalles de la falla en la parte inferior de la salida.
  - dot: El informe Dot Matrix es una serie de caracteres que con una representación minima casos de prueba: punto para las superadas, las fallidas con signos de exclamación rojos (!), las pendientes con una coma azul (,) y las lentas en amarillo.
  - progress: El informe de progreso implementa una barra de progreso simple.
  - min: El informe Min muestra solo el resumen, mientras sigue generando errores en caso de falla. Este informe funciona muy bien con --watch ya que limpia la terminal para mantener el resumen de la prueba en la parte superior.

© JMA 2020. All rights reserved

Informes

- Los informes disponibles son (continuación):
  - nyan: El informe de Nyan es exactamente lo que cabría esperar, un gato
  - landing: El informe de Landing Strip es un informe que simula el aterrizaje de un avión
  - tap: El informe TAP emite líneas para un cliente Test-Anything-Protocol.
  - json: El informe JSON genera un solo objeto JSON grande cuando las pruebas se han completado.
  - json-stream: El informe de JSON Stream genera "eventos" JSON delimitados por saltos de línea a medida que ocurren, comenzando con un evento de "inicio", seguido de pruebas superadas o fallidas, y luego el evento final de "finalización".
  - doc: El informe Doc genera un fragmento HTML para incrustarlo en el cuerpo de una página HTML.
  - markdown: El informe de Markdown genera la documentación de conjunto de pruebas con un TOC y el cuerpo de cada prueba, no refleja la ejecución.
  - xunit: El informe de XUnit produce un documento XML compatible con XUnit, a menudo aplicable en servidores CI.
  - html: El informe HTML está diseñado para usarse en las pruebas en navegador, no para usarse en la línea de comandos.

© JMA 2020. All rights reserved

https://www.chaijs.com/

CHAI JS

© JMA 2020. All rights reserved

Introducción

- Chai es una biblioteca de aserciones BDD / TDD para NodeJS y el navegador que se puede combinar con cualquier marco de prueba de JavaScript.
- Chai cuenta con varios sabores que permiten al desarrollador elegir el más cómodo. Los estilos BDD fluyentes proporcionan un lenguaje expresivo y legible, mientras que el estilo de aserciones TDD proporciona una forma más clásica.
- Cada interfaz cuenta con sus propios objetos y comparadores para realizar las afirmaciones. En TDD se utilizan las aserciones (assert) y en BDD se puede optar por expectativas (expect) o por encadenar lo que debería (should) cumplir.
- Los complementos (plugins) extienden las afirmaciones de Chai a nuevos contextos, como la integración de proveedores y la construcción de objetos. Los desarrolladores pueden crear sus propios complementos o acceder a un amplio ecosistema compartido por la comunidad.
- El paquete está disponible a través de npm:
  - npm install chai
- Para su uso en el navegador:

```
<script src="chai.js" type="text/javascript"></script>
```

© JMA 2020. All rights reserved

Estilos

- El estilo TDD se expone a través de la interfaz assert. Las aserciones permiten incluir un mensaje opcional como último parámetro.

```
var assert = require('chai').assert
assert.equal(foo, 'bar', 'foo equal `bar`');
```

- Los estilos BDD son expect y should. Ambos usan el mismo lenguaje encadenable para construir aserciones, pero difieren en la forma en que se construye inicialmente una aserción: valor o métodos. El should tiene algunas peculiaridades adicionales.
- La interfaz de expect proporciona una función para la afirmación basadas en el valor obtenido:

```
var expect = chai.expect;
expect(foo).to.equal('bar');
expect(tea).to.have.property('flavors').with.lengthOf(3);
```

- La interfaz should extiende cada objeto (se agrega al Object.Prototype) con un método should para la aserción sobre el valor del objeto.

```
chai.should();
foo.should.be.a('string');
tea.should.have.property('flavors').with.lengthOf(3);
```

© JMA 2020. All rights reserved

assert

Generales: assert, fail

Excepciones: throws, doesNotThrow, ifError

Comparadores: equal, notEqual, strictEqual, notStrictEqual, deepEqual, notDeepEqual, isAbove, isAtLeast, isBelow, isAtMost, match, notMatch

Numéricos: operator, closeTo, approximately

Booleanos: isOk, isNotOk, isTrue, isNotTrue, isFalse, isNotFalse,

Tipos: isNull, isNotNull, isNaN, isNotNaN, exists, notExists, isUndefined, isDefined, isFunction, isNotFunction, isObject, isNotObject, isArray, isNotArray, isString, isNotString, isNumber, isNotNumber, isFinite, isBoolean, isNotBoolean, typeOf, notTypeOf, instanceOf, notInstanceOf

Arrays, cadenas, conjuntos: lengthOf, oneOf, include, notInclude, deepInclude, notDeepInclude, nestedInclude, notNestedInclude, deepNestedInclude, notDeepNestedInclude, ownInclude, notOwnInclude, deepOwnInclude, notDeepOwnInclude

Objetos: property, notProperty, propertyVal, notPropertyVal, deepPropertyVal, notDeepPropertyVal, nestedProperty, notNestedProperty, nestedPropertyVal, notNestedPropertyVal, deepNestedPropertyVal, notDeepNestedPropertyVal, hasAnyKeys, hasAllKeys, containsAllKeys, doesNotHaveAnyKeys, doesNotHaveAllKeys, hasAnyDeepKeys, hasAllDeepKeys, containsAllDeepKeys, doesNotHaveAnyDeepKeys, doesNotHaveAllDeepKeys

Conjuntos: sameMembers, notSameMembers, sameDeepMembers, notSameDeepMembers, sameOrderedMembers, notSameOrderedMembers, sameDeepOrderedMembers, notSameDeepOrderedMembers, includeMembers, notIncludeMembers, includeDeepMembers, notIncludeDeepMembers, includeOrderedMembers, notIncludeOrderedMembers, includeDeepOrderedMembers, notIncludeDeepOrderedMembers

Funciones: changes, changesBy, doesNotChange, changesButNotBy, increases, increasesBy, doesNotIncrease, increasesButNotBy, decreases, decreasesBy, doesNotDecrease, doesNotDecreaseBy, decreasesButNotBy

Objetos: isExtensible, isNotExtensible, isSealed, isNotSealed, isFrozen, isNotFrozen, isEmpty, isNotEmpty

© JMA 2020. All rights reserved

expect y should

- expect y should encadenan afirmaciones de lenguaje natural. Las afirmaciones negativas tienen el encabezado not. prefijando a la afirmación. Expect también permite incluir mensajes para afirmaciones fallidas.

```
expect([1, 2, 3]).to.have.lengthOf(3)
  .and.include.ordered.members([1, 2])
  .but.not.include.ordered.members([2, 3]);
```

- Los métodos y valores suministrados son: deep, nested, own, ordered, any, all, a, include, ok, true, false, null, undefined, NaN, exist, empty, arguments, equal, eql, above, least, below, most, within, instanceof, property, ownPropertyDescriptor, lengthOf, match, string, keys, throw, respondTo, itself, satisfy, closeTo, members, oneOf, change, increase, decrease, by, extensible, sealed, frozen, finite, fail
- Las siguientes propiedades se proporcionan como captadores concatenables para mejorar la legibilidad de la afirmaciones pero no influyen en la misma: to, be, been, is, that, which, and, has, have, with, at, of, same, but, does, still, also.

© JMA 2020. All rights reserved

BUENAS PRACTICAS

© JMA 2020. All rights reserved

Características de una buena prueba unitaria

- Rápida. No es infrecuente que los proyectos maduros tengan miles de pruebas unitarias. Las pruebas unitarias deberían tardar muy poco tiempo en ejecutarse.
- Aislada. Las pruebas unitarias son independientes, se pueden ejecutar de forma aislada y no tienen ninguna dependencia en ningún factor externo, como sistemas de archivos o bases de datos.
- Reiterativa. La ejecución de una prueba unitaria debe ser coherente con sus resultados, es decir, devolver siempre el mismo resultado si no cambia nada entre ejecuciones.
- Autocomprobada. La prueba debe ser capaz de detectar automáticamente si el resultado ha sido correcto o incorrecto sin necesidad de intervención humana.
- Oportuna. Una prueba unitaria no debe tardar un tiempo desproporcionado en escribirse en comparación con el código que se va a probar. Si observa que la prueba del código tarda mucho en comparación con su escritura, considere un diseño más fácil de probar.

© JMA 2020. All rights reserved

Asignar nombre a las pruebas

- El nombre de la prueba debe constar de tres partes:
  - Nombre del método que se va a probar.
  - Escenario en el que se está probando.
  - Comportamiento esperado al invocar al escenario.
- Los estándares de nomenclatura son importantes porque expresan de forma explícita la intención de la prueba.

© JMA 2020. All rights reserved

Organizar el código de la prueba

- Prepara, actuar, afirmar es un patrón común al realizar pruebas unitarias. Como el propio nombre implica, consta de tres acciones principales:
  - Prepara los objetos, crearlos y configurarlos según sea necesario.
  - Actuar en un objeto.
  - Afirmar que algo es como se espera.
- Separa claramente en secciones lo que se está probando de los pasos preparación y verificación. Esta separación permite identificar donde esta el problema en caso de producirse una excepción no controlada en el código de la prueba.
- Las secciones solo deben una vez como máximo y en el orden establecido.
- Minimiza la posibilidad de mezclar aserciones con el código para "actuar", solo aceptable en pruebas de flujos.

© JMA 2020. All rights reserved

Preparación mínima

- La sección de preparación, con la entrada del caso de prueba, debe ser lo más sencilla posible, lo imprescindible para comprobar el comportamiento que se está probando.
- Las pruebas se hacen más resistentes a los cambios futuros en el código base y más cercano al comportamiento de prueba que a la implementación.
- Las pruebas que incluyen más información de la necesaria tienen una mayor posibilidad de incorporar errores en la prueba y pueden hacer confusa su intención. Al escribir pruebas, el usuario quiere centrarse en el comportamiento. El establecimiento de propiedades adicionales en los modelos o el empleo de valores distintos de cero cuando no es necesario solo resta de lo que se quiere probar.

© JMA 2020. All rights reserved

Actuación mínima

- Al escribir las pruebas hay que evitar introducir condiciones lógicas como if, switch, while, for, etc.
- Minimiza la posibilidad de incorporar un error a las pruebas.
- El foco está en el resultado final, en lugar de en los detalles de implementación.
- Al incorporar lógica al conjunto de pruebas, aumenta considerablemente la posibilidad de agregar un error. Cuando se produce un error en una prueba, se quiere saber realmente que algo va mal con el código probado y no en el código que prueba. En caso contrario, restan confianza y las pruebas en las que no se confía no aportan ningún valor.
- El objetivo de la prueba debe ser único, si la lógica en la prueba parece inevitable, denota que el objetivo es múltiple y hay que considerar la posibilidad de dividirla en dos o más pruebas diferentes.

© JMA 2020. All rights reserved

Sustituir literales por constantes

- Las valores literales pueden provocar confusión al lector de las pruebas. Si una cadena tiene un aspecto fuera de lo normal, puede preguntarse por qué se ha elegido un determinado valor para un parámetro o valor devuelto. Esto obliga a un vistazo más detallado a los detalles de implementación, en lugar de centrarse en la prueba. La asignación de literales a constantes permite dar nombre a los valores, aportando semántica.
- Evita la necesidad de que el lector de la prueba inspeccione el código de producción con el fin de averiguar lo que significa un valor, que hace el valor sea especial.

```
Assert.IsTrue(rslt.Length <= 10)
```

- Muestra explícitamente lo que se intenta probar, en lugar de lo que se intenta lograr.

```
const string VARCHAR_LEN = 10;
Assert.IsTrue(rslt.Length <= VARCHAR_LEN)
```

© JMA 2020. All rights reserved

Evitar varias aserciones

- Al escribir las pruebas, hay que intentar incluir solo una aserción por prueba. Los enfoques comunes para usar solo una aserción incluyen:
  - Crear una prueba independiente para cada aserción.
  - Usar pruebas con parámetros.
- Si se produce un error en una aserción, no se evalúan las aserciones posteriores.
- Garantiza que no se estén declarando varios casos en las pruebas.
- Proporciona la imagen exacta de por qué se producen errores en las pruebas.
- Al incorporar varias aserciones en un caso de prueba, no se garantiza que se ejecuten todas. Es un todas o ninguna, se sabe por cual fallo pero no si el resto también falla o es correcto, proporcionando la imagen parcial.
- Una excepción común a esta regla es cuando la validación cubre varios aspectos. En este caso, suele ser aceptable que haya varias aserciones para asegurarse de que el resultado está en el estado que se espera que esté.

© JMA 2020. All rights reserved

Refactorizar código

- La refactorización del código de prueba favorece le reutilización y la legibilidad, simplifican las pruebas.
- Salvo que todos los métodos de prueba usen los mismos requisitos, si se necesita un objeto o un estado similar para las pruebas, es preferible usar métodos auxiliares a los métodos de instalación y desmontaje (si existen):
  - Menos confusión al leer las pruebas, puesto que todo el código es visible desde dentro de cada prueba.
  - Menor posibilidad de configurar mas o menos de lo necesario para la prueba.
  - Menor posibilidad de compartir el estado entre las pruebas, lo que crea dependencias no deseadas entre ellas.
- Cada prueba normalmente tendrá requisitos diferentes para funcionar y ejecutarse. Los métodos de instalación y desmontaje son únicos, pero se pueden crear tantos métodos auxiliares como escenarios reutilizables se necesiten.

© JMA 2020. All rights reserved

No validar métodos privados

- En la mayoría de los casos, no debería haber necesidad de probar un método privado.
- Los métodos privados son un detalle de implementación.
- Se puede considerar de esta forma: los métodos privados nunca existen de forma aislada. En algún momento, va a haber un método público que llame al método privado como parte de su implementación. Lo que debería importar es el resultado final del método público que llama al privado.

© JMA 2020. All rights reserved

Aislar las pruebas

- Las dependencias externas afectan a la complejidad de la estrategia de pruebas, hay que aislar a las pruebas de las dependencias externas, sustituyendo las dependencias por dobles de prueba, salvo que se este probando específicamente dichas dependencias.
- Siguiendo la misma regla de oro, las pruebas de integración y sistema deben estar aisladas de sus dependencias salvo cuando se estén probando dichas dependencias. Así mismo, el resultado de las pruebas debe ser previsible.
- Entre las ventajas de esta aproximación se encuentran:
  - Devuelven resultados determinísticos
  - Permiten crear o reproducir determinados estados (por ejemplo errores de conexión)
  - Obtienen resultados mucho mas rápidamente y a menor coste, incluso offline.
  - Permiten el inicio temprano de las pruebas incluso cuando las dependencias todavía no están disponibles.
  - Permiten incluir atributos o métodos exclusivamente para el testeo.

© JMA 2020. All rights reserved

Cubrir aspectos no evidentes

- Las pruebas no deben cubrir solo los casos evidentes, los correctos (happy path), sino que deben ampliarse a los casos incorrectos.
- Un juego de pruebas debe ejercitar la resiliencia: la capacidad de resistir los errores y la recuperación ante los mismos.
- En los cálculos no hay que comprobar solamente si realiza correctamente el calculo, también hay que verificar que es el calculo que se debe realizar.
- Los dominios de los datos determinan la validez de los mismos y fijan la calidad de la información, dichos dominios deben ser ejercitados profundamente.

© JMA 2020. All rights reserved

Respetar los limites de las pruebas

- La pruebas unitarias ejercitan profundamente los componentes de formar aislada centrándose en la funcionalidad, los cálculos, las reglas de dominio y semánticas de los datos. Opcionalmente la estructura del código, es decir, sentencias, decisiones, bucles y caminos distintos.
- Las pruebas de integración se basan en componentes ya probados (unitaria o integración) o en dobles de pruebas y se centran en la estructura de llamadas, secuencias o colaboración, y la transición de estados.
- Hay muchos tipos de pruebas de sistema y cada uno pone el foco en un aspecto muy concreto, cada prueba solo debe un solo aspecto. Las pruebas funcionales del sistema son las pruebas de integración de todo el sistema centrándose en compleción de la funcionalidades y los procesos de negocio, su estructura, disponibilidad y accesibilidad.

© JMA 2020. All rights reserved

PRUEBAS E2E

© JMA 2020. All rights reserved

Introducción

- A medida que las aplicaciones crecen en tamaño y complejidad, se vuelve imposible depender de pruebas manuales para verificar la corrección de las nuevas características, errores de captura y avisos de regresión. Las pruebas unitarias son la primera línea de defensa para la captura de errores, pero a veces las circunstancias requieran la integración entre componentes que no se pueden capturar en una prueba unitarias.
- Algunas pruebas deben tener una vista de pájaro de alto nivel de la aplicación. Simulan a un usuario interactuando con la aplicación: navegando a una dirección, leyendo texto, haciendo clic en un enlace o botón, llenando un formulario, moviendo el mouse o escribiendo en el teclado. Estas pruebas generan las expectativas sobre lo que el usuario ve y lee en el navegador.
- Desde la perspectiva del cliente, solo importa el interfaz (UI o API) y no como esté la aplicación implementada. Los detalles técnicos como la estructura interna de su código no son relevantes. No hay distinción entre front-end y back-end, entre partes del código. Se prueba la experiencia completa.
- Estas pruebas se denominan pruebas de extremo a extremo (E2E) ya que integran todas las partes de la aplicación desde un extremo (el cliente) hasta el otro extremo (los rincones más oscuros del back-end). Las pruebas de extremo a extremo también forman la parte automatizada de las pruebas de aceptación, ya que indican si la aplicación funciona para el usuario.

© JMA 2020. All rights reserved

Ventajas e inconvenientes

- Las pruebas de extremo a extremo han sido ampliamente adoptadas porque:
  - Ayuda a los equipos a expandir su cobertura de pruebas agregando casos de pruebas más detallados que otros métodos de prueba, como pruebas unitarias y funcionales.
  - Garantiza que la aplicación funcione correctamente ejecutando los casos de prueba en función del comportamiento del usuario final.
  - Ayuda a los equipos de lanzamiento a reducir el tiempo de paso a producción al permitirles automatizar las rutas críticas de los usuarios.
  - Reduce el costo general de creación y mantenimiento del software al disminuir el tiempo que lleva probar el software.
  - Ayuda a detectar errores de manera predecible y confiable, aunque no de forma concluyente.
- Los inconvenientes se presentan porque las pruebas de extremo a extremo:
  - Toman mucho tiempo, son frágiles y no son concluyentes (detectan el fallo pero no donde se produce).
  - Deben estar diseñadas para reproducir escenarios del mundo real.
  - Requiere una buena comprensión de los objetivos del usuario: Los usuarios no buscan características, buscan resolver sus problemas específicos.
- Las pruebas de extremo a extremo requieren un grupo multidisciplinario que incluye desarrolladores, probadores, administradores y usuarios.

© JMA 2020. All rights reserved

Estrategias de automatización

- **Record-and-replay (R&R):** Se orientan a la grabación de pruebas exploratorias sobre el GUI que luego se pueden reproducir automáticamente. La fase de grabación (recording) incluye aserciones de verificación y genera un script con las instrucciones a reproducir. El script puede estar en un lenguaje propio de la herramienta usada o puede tener instrucciones de un API para automatización. Tienen el beneficio que son fáciles de aprender, grabar y reproducir, sin embargo, pero son difíciles de mantener o personalizar al estar limitadas por la herramienta utilizada.
- **APIs de automatización:** Existen librerías y frameworks que permiten la automatización de pruebas de GUI mediante código, a través de APIs que permiten controlar externamente el GUI. Si bien se benefician de un mayor control y flexibilidad, tienen una curva de aprendizaje mucho mayor, deben ocuparse de los detalles, como los tiempos de espera, y dependen de un código que hay que escribir y mantener, mucho mas costoso que una grabación.

© JMA 2020. All rights reserved

Conceptos a tener en cuenta

- Flujo funcional:
  - Proceso en el que se define un comportamiento a partir de unas entradas, y un tratamiento de esas entradas por parte de todos los sistemas o componentes, hasta llegar a un resultado final.
- Camino crítico:
  - En un flujo funcional participan normalmente muchos sistemas o componentes. Sería un error pretender hacer la validación del todo, porque por una parte normalmente no hay tiempo material para hacerlo, y por otra, muchos de los sistemas no tienen relevancia. El camino crítico lo componen sólo los sistemas a validar en la prueba E2E.
- Pruebas horizontales o verticales:
  - Las horizontales cubren aspectos de toda la aplicación mientras que las pruebas verticales cubren un único aspecto.

© JMA 2020. All rights reserved

Proceso de Prueba

- Identifica con qué navegadores probar
- Elige el mejor lenguaje/entorno para ti y tu equipo.
- Configura entorno para que funcione con cada navegador que te interese.
- Descompón la aplicación web existente para identificar qué probar
- Escribe pruebas mantenibles y reutilizables que sean compatibles y ejecutables en todos los navegadores.
- Crea un circuito de retroalimentación integrado para automatizar las ejecuciones de prueba y encontrar problemas rápidamente.
- Configura tu propia infraestructura o conéctate a un proveedor en la nube.
- Mejora drásticamente los tiempos de prueba con la paralelización
- Mantente actualizado en el mundo de las herramientas de pruebas.

© JMA 2020. All rights reserved

Selenium

- El Selenium es un conjunto de herramientas para automatizar los navegadores web, robot que simula la interacción del usuario con el navegador, originalmente pensado como entorno de pruebas de software para aplicaciones basadas en la web. Como principales herramientas Selenium cuenta con:
  - Selenium IDE: una herramienta para grabar y reproducir secuencias de acciones con el navegador que permite crear pruebas sin usar un lenguaje de scripting para pruebas.
  - Selenium Core: API para escribir pruebas automatizadas y de regresión en un amplio número de lenguajes como C#, Java, JavaScript, Ruby, Groovy, Perl, Php y Python.
  - WebDriver: interfaces que permite ejecutar las pruebas de forma nativa usando la mayoría de los navegadores web modernos en diferentes sistemas operativos como Windows, Linux y OSX.
  - Selenium Grid: Permite ejecutar muchas pruebas de un mismo grupo en paralelo o pruebas en múltiples entornos, un conjunto de pruebas muy grande puede dividirse en varias maquinas remotas para una ejecución más rápida o si se necesitan repetir en múltiples entornos.

© JMA 2020. All rights reserved

Selenium IDE

- Es el entorno de desarrollo integrado para pruebas con Selenium que permite grabar, editar y depurar fácilmente las pruebas.
- Solo está disponible como una extensión de Firefox y Chrome.
- Se pueden desarrollar automáticamente scripts al crear una grabación y de esa manera se puede editar manualmente con sentencias y comandos para que la reproducción de nuestra grabación sea correcta
- Los scripts se generan en un lenguaje de scripting especial para Selenium a menudo denominado Selanese (JSON).
- Selanese provee comandos que dicen al Selenium que hacer y pueden ser:
  - **Acciones**: son comandos que generalmente manipulan el estado de la aplicación, ejecutan acciones sobre objetos del navegador, como hacer click en un enlace, escribir en cajas de texto o seleccionar de una lista de opciones. Muchas acciones pueden ser llamadas con el sufijo "AndWait“ que indica la acción hará que el navegador realice una llamada al servidor y que se debe esperar a una nueva página se cargue.
  - **Descriptores de acceso**: examinan el estado de la página y almacenan los resultados en variables.
  - **Aserciones**: son como descriptores de acceso, pero las muestras confirman que el estado de la solicitud se ajusta a lo que se esperaba, verifican la presencia de un texto en particular o la existencia de elementos.

© JMA 2020. All rights reserved

Selenium IDE

- Dispone de una selección inteligente de campos usando ID, nombre, Xpath o DOM según se necesite.
- Para la depuración permite la configuración de los puntos de interrupción, iniciar y detener la ejecución de un caso de prueba desde cualquier punto dentro del caso de prueba e inspeccionar la forma en el caso de prueba se comporta en ese punto.
- Permite exportar los casos de prueba a Java, JavaScript, C#, Python y Ruby, actuando como embriones en la creación de los casos de prueba para WebDriver.
- Selenium IDE dispone de un amplio conjunto de extensiones adicionales que ayudan o simplifican la elaboración de los casos de pruebas.

© JMA 2020. All rights reserved

Localizadores

- Localizar por Id:
  - id=loginForm
- Localizar por Name
  - name=username
- Localizar por el texto en los hipervínculos
  - link=Continue
- Localizar por CSS
  - css=input[name="username"]
- Localizar por XPath
  - xpath=//form[@id='loginForm']

© JMA 2020. All rights reserved

Variables

- Se puede usar variable en Selenium para almacenar constantes al principio de un script. Además, cuando se combina con un diseño de prueba controlado por datos, las variables de Selenium se pueden usar para almacenar valores pasados a la prueba desde la línea de comandos, desde otro programa o desde un archivo.
  - store target:valor value:varName
- Para acceder al valor de una variable:
  - ${userName}
- Hay métodos diponibles para recuperar información de la página y almacenarla en variables:
  - storeAttribute, storeText, storeValue, storeTitle, storeXpathCount

© JMA 2020. All rights reserved

Afirmar y Verificar

- Una "afirmación" hará fallar la prueba y abortará el caso de prueba actual, mientras que una "verificación" hará fallar la prueba pero continuará ejecutando el caso de prueba.
  - Tiene muy poco sentido para comprobar que el primer párrafo de la página sea el correcto si la prueba ya falló al comprobar que el navegador muestra la página esperada. Por otro lado, es posible que desee comprobar muchos atributos de una página sin abortar el caso de prueba al primer fallo, ya que esto permitirá revisar todas los fallos en la página y tomar la acción apropiada.
- Selenese permite múltiples formas de comprobar los elementos de la interfaz de usuario pero hay que decidir el métodos mas apropiado:
  - ¿Un elemento está presente en algún lugar de la página?
  - ¿El texto especificado está en algún lugar de la página?
  - ¿El texto especificado está en una ubicación específica en la página?
- Métodos:
  - assert, assertAlert, assertChecked, assertNotChecked, assertConfirmation, assertEditable, assertNotEditable, assertElementPresent, assertElementNotPresent, assertPrompt, assertSelectedValue, assertNotSelectedValue, assertSelectedLabel, assertText, assertNotText, assertTitle, assertValue
  - verify, verifyChecked, verifyNotChecked, verifyEditable, verifyNotEditable, verifyElementPresent, verifyElementNotPresent, verifySelectedValue, verifyNotSelectedValue, verifyText, verifyNotText, verifyTitle, verifyValue, verifySelectedLabel

© JMA 2020. All rights reserved

WebDriver

```
@BeforeClass
 public static void setUpClass() throws Exception {
                System.setProperty("webdriver.chrome.driver", "C:/Archivos/.../chromedriver.exe");
 }
 @Before
 public void setUp() throws Exception {
  driver = new ChromeDriver();
  baseUrl = "http://localhost/";
  driver.manage().timeouts().implicitlyWait(30, TimeUnit.SECONDS);
 }
@Test
 public void testLoginOK() throws Exception {
  driver.get(baseUrl + "/login.php");
  driver.findElement(By.id("login")).sendKeys("admin");
  driver.findElement(By.id("password")).sendKeys("admin");
  driver.findElement(By.cssSelector("input[type=\"submit\"]")).click();
  try {
   assertEquals("", driver.findElement(By.cssSelector("img[title=\"Main Menu\"]")).getText());
  } catch (Error e) {
   verificationErrors.append(e.toString());
  }
 }
```

Maven

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>3.13.0</version>
</dependency>
```

© JMA 2020. All rights reserved

Ejecutar en línea de comandos

- Requiere tener instalado NodeJS (https://nodejs.org)
- Instalar CLI
  - npm install -g selenium-side-runner
  - npm install -g chromedriver edgedriver geckodriver
- Instalar los WebDriver:
  - Crear una carpeta y referenciarla en el PATH del sistema.
  - Descargar los drivers (https://www.seleniumhq.org/download/)
  - Copiarlos a la carpeta creada.
- Para ejecutar las suites de pruebas:
  - selenium-side-runner project.side project2.side *.side
- Para ejecutar en diferentes navegadores:
  - selenium-side-runner *.side -c "browserName=Chrome"
  - selenium-side-runner *.side -c "browserName=firefox"

© JMA 2020. All rights reserved

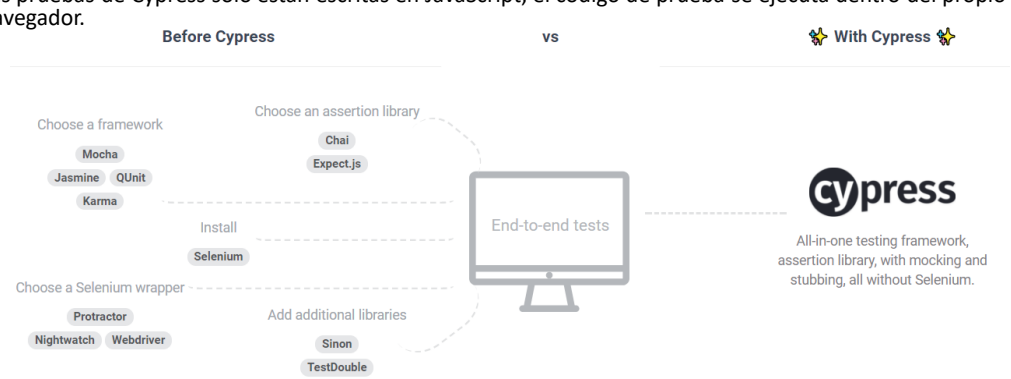
Cypress

- Cypress es una herramienta de prueba de front-end de próxima generación creada para la web moderna, que permite configurar, escribir, ejecutar y depurar pruebas.
- Aunque Cypress se compara con Selenium, es fundamental y arquitectónicamente diferente, controla los navegadores a través de JavaScript y las DevTools, por lo que no está limitado por las mismas restricciones que el Selenium. Esto permite escribir pruebas más rápidas, fáciles y confiables. Esta destinado a desarrolladores o ingenieros de control de calidad que crean aplicaciones web utilizando marcos de JavaScript modernos.
- Aunque Cypress puede probar cualquier cosa que se ejecute en un navegador, esta destinado a:
  - Pruebas de un extremo a extremo
  - Pruebas de integración
  - Pruebas unitarias (componentes)
- Cypress se ha construido para que las pruebas y el desarrollo se puedan realizar simultáneamente. Es suficientemente rápido para habilitar metodologías TDD, BDD o ATDD, donde los cambios en el código o en las pruebas se reflejan en tiempo real.

© JMA 2020. All rights reserved

Cypress

- Escribir pruebas e2e requiere muchas herramientas diferentes trabajando juntas. Cypress es un todo en uno, no es necesario instalar 10 herramientas y bibliotecas independientes para configurar el entorno de pruebas. Han tomado algunas de las mejores herramientas de su clase y las han hecho funcionar juntas sin problemas.
- Las pruebas de Cypress solo están escritas en JavaScript, el código de prueba se ejecuta dentro del propio navegador.



© JMA 2020. All rights reserved

Playwright

- Playwright es una biblioteca Node.js respaldada por Microsoft para automatizar navegadores con una sola API. Playwright está diseñado para habilitar la automatización web entre exploradores que es perenne, capaz, confiable y rápida.
- Cualquier navegador & Cualquier plataforma & One API
  - Cross-browser: Playwright es compatible con todos los motores de renderizado modernos, incluidos Chromium, WebKit y Firefox.
  - Cross-platform: Pruebas en Windows, Linux y macOS, localmente o en CI, sin cabeza o con cabeza.
  - Cross-language: Usa la API de Playwright en TypeScript, JavaScript, Python, .NET (MSTest o NUnit) o Java.
  - Prueba Web Móvil. Emulación móvil nativa de Google Chrome para Android y Mobile Safari. El mismo motor de renderizado funciona en el escritorio y en la nube.
- Potentes Herramientas
  - Codegen: Genera pruebas registrando sus acciones. Guárdelos en cualquier idioma.
  - Playwright inspector: Inspeccione la página, genere selectores, pase por la ejecución de la prueba, vea los puntos de clic, explore los registros de ejecución.
  - Trace Viewer: Capture toda la información para investigar el fallo de la prueba. La traza de Playwright contiene screencast de ejecución de prueba, instantáneas DOM en vivo, explorador de acción, fuente de prueba y muchos más.

© JMA 2020. All rights reserved

Appium

- Appium es una herramienta de automatización de código abierto para ejecutar scripts y probar aplicaciones nativas, aplicaciones web y aplicaciones híbridas sobre Android o iOS utilizando un webdriver basado en la W3C WebDriver Specification al estilo Selenium. Appium, originalmente en C#, se porto a Node.js en 2013, siendo actualmente su plataforma de ejecución.
- Appium es un proyecto de código abierto y un ecosistema relacionado software, diseñado para facilitar la automatización de UI de muchas plataformas de aplicaciones siempre que se disponga del correspondiente driver:
  - Móvil: iOS, Android, Tizen, …
  - Navegador: Chrome, Firefox, Safari, …
  - Escritorio: macOS, Windows, …
  - TV: Roku, tvOS, Android TV, Samsung, LG WebOS, …
- Appium Client admite que el código de automatización este escrito en diferentes lenguajes: C#, JavaScript, Java, Python, Ruby, …

© JMA 2020. All rights reserved

Postman

https://www.getpostman.com/

- Postman surgió originariamente como una extensión para el navegador Google Chrome que permitía realizar peticiones API REST con métodos diferentes al GET. A día de hoy dispone de aplicaciones nativas para MAC, Windows y algunas producciones Linux.
- Está compuesto por diferentes herramientas y utilidades gratuitas (en la versión free) que permiten realizar tareas diferentes dentro del mundo API REST: creación de peticiones a APIs internas o de terceros, elaboración de tests para validar el comportamiento de APIs, posibilidad de crear entornos de trabajo diferentes (con variables globales y locales), y todo ello con la posibilidad de ser compartido con otros compañeros del equipo de manera gratuita (exportación de toda esta información mediante URL en formato JSON).
- Además, dispone de un modo cloud colaborativo (de pago) para que equipos de trabajo puedan desarrollar entre todos colecciones para APIs sincronizadas en la nube para una integración más inmediata y sincronizada.
- Quizás sea una de las herramientas más utilizadas para hacer testing exploratorio de API REST.

© JMA 2020. All rights reserved

SoapUI

https://www.soapui.org

- SoapUI es una herramienta para probar servicios web que pueden ser servicios web SOAP, servicios web RESTful u otros servicios basados en HTTP.
- SoapUI es una herramienta de código abierto y completamente gratuita con una versión comercial, ReadyAPI, que tiene una funcionalidad adicional para empresas con servicios web de misión crítica.
- SoapUI se considera el estándar de facto para las Pruebas de servicio API. Esto significa que hay mucho conocimiento en la red sobre la herramienta y blogs para obtener más información sobre el uso de SoapUI en la vida real.
- SoapUI permite realizar pruebas funcionales, pruebas de rendimiento, pruebas de interoperabilidad, pruebas de regresión y mucho más. Su objetivo es que la prueba sea bastante fácil de comenzar, por ejemplo, para crear una Prueba de carga, simplemente hay que hacer clic derecho en una prueba funcional y ejecutarla como una prueba de carga.
- SoapUI puede simular servicios web (mocking). Se puede grabar pruebas y usarlas más tarde.

© JMA 2020. All rights reserved

PRUEBAS DE RENDIMIENTO

© JMA 2020. All rights reserved

Pruebas de rendimiento

© JMA 2020. All rights reserved

Pruebas de carga, estrés y picos

- Pruebas de carga (load test): pruebas para determinar y validar la respuesta de la aplicación cuando es sometida a una carga de usuarios y/o transacciones que se espera en el ambiente de producción. Ejemplo: verificar la correcta respuesta de la aplicación ante el alta de 100 usuarios en forma simultanea. Se compara con el volumen esperado.
- Pruebas de estrés (stress test): pruebas para encontrar el volumen de datos o de tiempo en que la aplicación comienza a fallar o es incapaz de responder a las peticiones. Son pruebas de carga o rendimiento, pero superando los límites esperados en el ambiente de producción y/o determinados en las pruebas. Ejemplo: encontrar la cantidad de usuarios simultáneos, en que la aplicación deja de responder (cuelgue o time out) en forma correcta a todas las peticiones.
- Pruebas de picos (spike testing): es un sub tipo de prueba de estrés que mide el rendimiento del software bajo un «pico» significativo y repentino o una carga de trabajo creciente como la de los usuarios simulados. Indica si el software puede manejar ese aumento abrupto de la carga de trabajo de forma repetida y rápida.

© JMA 2020. All rights reserved

Pruebas de resistencia, volumen y escalabilidad

- Pruebas de resistencia (soak testing, endurance testing): evalúa el rendimiento del software durante un periodo prolongado bajo una carga de trabajo regular y fija, determina cuánto tiempo puede soportar el software una carga de trabajo constante para proporcionar sostenibilidad a largo plazo. Durante estas pruebas, los equipos de pruebas supervisan los KPI como las fugas de memoria, de proceso, etc. Las pruebas de resistencia también analizan los tiempos de respuesta y el rendimiento tras un uso prolongado para mostrar si estas métricas son consistentes.
- Pruebas de volumen (volume testing): comprueban la eficacia del software cuando se somete a grandes volúmenes de datos. Comprueba la pérdida de datos, el tiempo de respuesta del sistema, la fiabilidad del almacenamiento de datos, etc.
- Pruebas de escalabilidad (scalability testing): miden la eficacia del software a la hora de manejar una cantidad creciente de carga de trabajo, añadiendo volumen de datos o usuarios de forma gradual mientras se supervisa el rendimiento del software. La prueba informará sobre el comportamiento cuando aumenten o disminuyan los atributos de rendimiento del software.

© JMA 2020. All rights reserved

JMeter

- http://jmeter.apache.org
- JMeter es una herramienta de pruebas de carga para llevar acabo simulaciones sobre cualquier recurso de Software.
- Inicialmente diseñada para pruebas de estrés en aplicaciones web, hoy en día, su arquitectura ha evolucionado no sólo para llevar acabo pruebas en componentes habilitados en Internet (HTTP), sino también en Bases de Datos , programas en Perl, peticiones FTP y prácticamente cualquier otro medio.
- Además, posee la capacidad de realizar desde una solicitud sencilla hasta secuencias de peticiones que permiten diagnosticar el comportamiento de una aplicación en condiciones de producción.
- En este sentido, simula todas las funcionalidades de un navegador, o de cualquier otro cliente, siendo capaz de manipular resultados en determinada requisición y reutilizarlos para ser empleados en una nueva secuencia.

© JMA 2020. All rights reserved

PRUEBAS DE ACCESIBILIDAD

© JMA 2020. All rights reserved

Definiciones

- La accesibilidad Web significa que personas con algún tipo de discapacidad van a poder hacer uso de la Web. En concreto, al hablar de accesibilidad Web se está haciendo referencia a un diseño Web que va a permitir que estas personas puedan percibir, entender, navegar e interactuar con la Web, aportando a su vez contenidos. La accesibilidad Web también beneficia a otras personas, incluyendo personas de edad avanzada que han visto mermadas sus habilidad a consecuencia de la edad.
- La accesibilidad web o de la interfaz, indica la capacidad de acceso a la Web y a sus contenidos por todas las personas, independientemente de la discapacidad (física, intelectual o técnica) que presenten o de las que se deriven del contexto de uso (tecnológicas o ambientales). Esta cualidad está íntimamente relacionada con la usabilidad.
- Un sitio web es accesible si las personas con discapacidad lo pueden utilizar con la misma efectividad, seguridad y protección que las personas sin discapacidad.

© JMA 2020. All rights reserved

Limitaciones

- Las limitaciones en la accesibilidad de los sitios Web pueden ser:
  - **Visuales**: En sus distintos grados, desde la baja visión a la ceguera total, además de problemas para distinguir colores (Daltonismo).
  - **Motrices**: Dificultad o la imposibilidad de usar las manos, incluidos temblores, lentitud muscular, etc, debido a enfermedades como el Parkinson, distrofia muscular, parálisis cerebral, amputaciones, entre otras.
  - **Auditivas**: Sordera o deficiencias auditivas.
  - **Cognitivas**: Dificultades de aprendizaje (dislexia, discalculia, …) o discapacidades cognitivas que afecten a la memoria, la atención, las habilidades lógicas, etc.
- A las personas con discapacidad podemos añadir el conjunto de personas de la "tercera edad", ya que las carencias y problemas de los medios físicos, así como muchas veces el contenido, hacen que estas personas se encuentren también en riesgo de infoexclusión.

© JMA 2020. All rights reserved

Problemas de accesibilidad

- **Manejo de terminales**: Los teléfonos, ordenadores, cajeros automáticos y televisión digital la mayoría de las veces no están diseñados y colocados, en el caso de los cajeros, prestando atención a las necesidades de las personas con discapacidad. La variedad de terminales es muy grande, lo que se debe buscar es seguir la tendencia a reducirlos y acceder a todos los servicios a través de unos pocos.
- **Interacción con las interfaces**: Los menús, barras de navegación y botones no suelen ser accesibles desde una variedad de terminales adaptados.
- **Acceso a los contenidos**: Los contenidos a los que se tiene acceso desde un mismo dispositivo son cada vez mayores y, este rápido crecimiento no suele atender las necesidades específicas de la discapacidad.

© JMA 2020. All rights reserved

Características de un sitio accesible

- **Transformable**: La información y los servicios deben ser accesibles para todos y deben poder ser utilizados con todos los dispositivos de navegación.
- **Comprensible**: Contenidos claros y simples.
- **Navegable**: Mecanismos sencillos de navegación.

© JMA 2020. All rights reserved

Ayudas técnicas

- Las ayudas técnicas, también llamadas tecnologías de apoyo, son los dispositivos empleados por las personas con discapacidad para prevenir, compensar, mitigar o neutralizar la discapacidad que poseen.
- Las siguientes son algunas de las tecnologías de apoyo que usan los usuarios discapacitados para navegar de la web:
  - Un programa lector de pantalla, que puede leer usando síntesis de voz, los elementos que se muestran en el monitor (de gran ayuda para los usuarios con dificultades de aprendizaje o lectura), o que puede leer todo lo que está pasando en el PC (utilizado por los usuarios ciegos y de visión reducida).
  - Líneas Braille, que consiste en dispositivo hardware que convierte el texto en caracteres Braille.
  - Un programa magnificador de pantalla que amplía lo que se muestra en el monitor de la computadora, haciéndolo más fácil de leer para los usuarios de visión reducida.

© JMA 2020. All rights reserved

Legislación

- Desde el año 2002, en España se han desarrollado varias leyes que definen los niveles de accesibilidad y fechas de cumplimiento:
  - Ley 34/2002 del 11 de julio, de servicios de la sociedad de la información y de comercio electrónico.
  - Ley 51/2003 del 2 de diciembre de Igualdad de Oportunidades, No Discriminación y Accesibilidad Universal con discapacidad (LIONDAU).
  - Real Decreto 366/2007 del 16 de marzo, de accesibilidad y no discriminación de las personas con discapacidad en sus relaciones con la Administración General del Estado.
  - Ley 27/2007, del 23 de octubre, por la que se reconocen las lenguas de signos españolas y se regulan los medios de apoyo a la comunicación oral de las personas sordas, con discapacidad auditiva y sordociegas.
  - Real Decreto 1494/2007, del 12 de noviembre, por el que se aprueba el Reglamento sobre las condiciones básicas para el acceso de las personas con discapacidad a la sociedad de la información.
  - Ley 49/2007, del 26 de diciembre, por la que se establece el régimen de infracciones y sanciones en materia de igualdad de oportunidades, no discriminación y accesibilidad universal de las personas con discapacidad.
  - Ley 56/2007, del 28 de diciembre, de Medidas de Impulso de la Sociedad de la Información que modifica la redacción de la disposición adicional quinta de la Ley 34/2002, del 11 de julio.
- Normativa UNE 139803:2012
  - En 2012 se actualizó para adoptar las WCAG 2.0 como base.

© JMA 2020. All rights reserved

Están obligadas

- Las Administraciones públicas
- Las empresas que presten servicios al público en general de especial trascendencia económica, las que agrupen a más de cien trabajadores o su volumen anual de operaciones, calculado conforme a lo establecido en la normativa del Impuesto sobre el Valor Añadido, exceda de 6.010.121,04 euros y que, en ambos casos, operen en los siguientes sectores económicos:
  - Servicios de comunicaciones electrónicas a consumidores
  - Servicios financieros destinados a consumidores (Servicios bancarios, de crédito o de pago, de inversión, de seguros privados, de corredor de seguros, Planes de pensiones)
  - Servicios de suministro de agua a consumidores
  - Servicios de suministro de gas al por menor
  - Servicios de suministro eléctrico a consumidores finales
  - Servicios de agencia de viajes
  - Servicios de transporte de viajeros por carretera, ferrocarril, por vía marítima, o por vía aérea
  - Actividades de comercio al por menor

© JMA 2020. All rights reserved

Beneficios

- Uno de los visitantes más fieles de nuestro sitio web es un usuario con discapacidad. Es ciego (no ve las imágenes, al menos de momento, ni los vídeos, ni admira la belleza de nuestras animaciones), es sordo, navega sin plugins instalados, sin applets y sin JavaScript activo, por lo tanto le es imposible seguir los enlaces que dependen de JavaScript.
- Jakob Nielsen le llama el “usuario ciego más rico del mundo” [Nielsen, 2012]. Este usuario es Googlebot, el robot de búsqueda de Google.
- La mitad de las visitas a tu sitio vienen de Google, y Google sólo ve lo que un ciego puede ver. Si tu sitio no es accesible, tendrás menos visitas. Fin de la historia.
- Esta es la razón por la cual muchas de las técnicas que aplicamos para hacer nuestra web más accesible repercuten directa y positivamente en su indexación y posicionamiento en los buscadores (SEO).

© JMA 2020. All rights reserved

Beneficios

- Aumenta el número de potenciales visitantes de la página web: esta es una razón muy importante para una empresa que pretenda captar nuevos clientes. Cuando una página web es accesible no presenta barreras que dificulten su acceso, independientemente de las condiciones del usuario y es más probable que se visualice correctamente en cualquier dispositivo con cualquier navegador.
- Disminuye los costes de desarrollo y mantenimiento: aunque inicialmente aprender a hacer una página web accesible supone un coste, una vez se tienen los conocimientos, el coste de desarrollar y mantener una página web accesible es menor que frente a una no accesible, dado que es menos propensa a contener errores y más sencilla de actualizar.
- Reduce el tiempo de carga de las páginas web y la carga del servidor web: al separar el contenido de la información sobre la presentación de una página web mediante CSS se logra reducir el tamaño de las páginas web y, por tanto, se reduce el tiempo de carga de las páginas web.
- Aumenta la usabilidad de la página web: esto también implica indirectamente, que la página podrá ser visualizada desde cualquier navegador.
- Demostramos que nos implicamos socialmente.
- Aumenta el capital humano de las comunidades de aprendizaje potenciando la inteligencia colectiva.

© JMA 2020. All rights reserved

Pautas de accesibilidad Web

- El máximo organismo dentro de la jerarquía de Internet que se encarga de promover la accesibilidad es el World Wide Web Consortium (W3C), en especial su grupo de trabajo Web Accessibility Initiative (WAI). En 1999 el WAI publicó la versión 1.0 de sus pautas de accesibilidad Web. Con el paso del tiempo se han convertido en un referente internacionalmente aceptado. En diciembre del 2008 las WCAG 2.0 fueron aprobadas como recomendación oficial.
- Estas pautas se dividen en tres bloques:
  - Pautas de Accesibilidad al Contenido en la Web (WCAG): Están dirigidas a los webmasters e indican cómo hacer que los contenidos del sitio web sean accesibles.
  - Pautas de Accesibilidad para Herramientas de Autor (ATAG): Están dirigidas a los desarrolladores del software que usan los webmasters, para que estos programas faciliten la creación de sitios accesibles.
  - Pautas de Accesibilidad para Agentes de Usuario (UAAG): Están dirigidas a los desarrolladores de Agentes de usuario (navegadores y similares), para que estos programas faciliten a todos los usuarios el acceso a los sitios Web.

© JMA 2020. All rights reserved

Guía breve para crear sitios webs accesibles

- Imágenes y animaciones: Use el atributo alt para describir la función de cada elemento visual.
- Mapas de imagen: Use el elemento map y texto para las zonas activas.
- Multimedia: Proporcione subtítulos y transcripción del sonido, y descripción del vídeo.
- Enlaces de hipertexto: Use texto que tenga sentido leído fuera de contexto. Por ejemplo, evite "pincha aquí".
- Organización de las páginas: Use encabezados, listas y estructura consistente. Use CSS para la maquetación donde sea posible.
- Figuras y diagramas: Descríbalos brevemente en la pagina o use el atributo longdesc.
- Scripts, applets y plug-ins: Ofrezca contenido alternativo si las funciones nuevas no son accesibles.
- Marcos: Use el elemento noframes y títulos con sentido.
- Tablas: Facilite la lectura línea a línea. Resuma.
- Revise su trabajo: Verifique.

© JMA 2020. All rights reserved

Pautas de Accesibilidad del Contenido en la Web (WCAG 1.0)

1. Proporcione alternativas equivalentes para el contenido visual y auditivo
2. No se base sólo en el color
3. Utilice marcadores y hojas de estilo y hágalo apropiadamente
4. Identifique el idioma usado
5. Cree tablas que se transformen correctamente
6. Asegúrese de que las páginas que incorporen nuevas tecnologías se transformen correctamente
7. Asegure al usuario el control sobre los cambios de los contenidos tempo-dependientes
8. Asegure la accesibilidad directa de las interfaces incrustadas
9. Diseñe para la independencia del dispositivo
10. Utilice soluciones provisionales
11. Utilice las tecnologías y pautas W3C
12. Proporcione información de contexto y orientación
13. Proporcione mecanismos claros de navegación
14. Asegúrese de que los documentos sean claros y simples

© JMA 2020. All rights reserved

Orientación de las WCAG 2.0

- Principios: En el nivel más alto se sitúan los cuatro principios que proporcionan los fundamentos de la accesibilidad web: perceptible, operable, comprensible y robusto.
- Pautas: Por debajo de los principios están las pautas. Las doce pautas proporcionan los objetivos básicos que los autores deben lograr con el fin de crear un contenido más accesible para los usuarios con distintas discapacidad. Estas pautas no son verificables, pero proporcionan el marco y los objetivos generales que ayudan a los autores a comprender los criterios de conformidad y a implementar mejor las técnicas.
- Criterios de Conformidad: Para cada pauta se proporcionan los criterios de conformidad verificables que permiten emplear las WCAG 2.0 en aquellas situaciones en las que existan requisitos y necesidad de evaluación de conformidad como: especificaciones de diseño, compras, regulación o acuerdos contractuales. Con el fin de cumplir con las necesidades de los diferentes grupos y situaciones, se definen tres niveles de conformidad: A (el más bajo), AA y AAA (el más alto).
- Técnicas suficientes y recomendables: Para cada una de las pautas y criterios de conformidad del propio documento de las WCAG 2.0, el grupo de trabajo ha documentado también una amplia variedad de técnicas. Las técnicas son informativas y se agrupan en dos categorías: aquellas que son suficientes para satisfacer los criterios de conformidad, y aquellas que son recomendables. Las técnicas recomendables van más allá de los requisitos de cada criterio de conformidad individual y permiten a los autores afrontar mejor las pautas. Algunas de las técnicas recomendables tratan sobre barreras de accesibilidad que no han sido cubiertas por los criterios de conformidad verificables. También se han documentado los errores frecuentes que son conocidos.

© JMA 2020. All rights reserved

Principios y pautas

http://www.sidar.org/traducciones/wcag20/es/

1. Perceptible
   - 1.1 Proporcionar alternativas textuales para todo contenido no textual de modo que se pueda convertir a otros formatos que las personas necesiten, tales como textos ampliados, braille, voz, símbolos o en un lenguaje más simple.
   - 1.2 Medios tempodependientes: proporcionar alternativas para los medios tempodependientes.
   - 1.3 Crear contenido que pueda presentarse de diferentes formas (por ejemplo, con una disposición más simple) sin perder información o estructura.
   - 1.4 Facilitar a los usuarios ver y oír el contenido, incluyendo la separación entre el primer plano y el fondo.

© JMA 2020. All rights reserved

Principios y pautas

2. Operable
   - 2.1 Proporcionar acceso a toda la funcionalidad mediante el teclado.
   - 2.2 Proporcionar a los usuarios el tiempo suficiente para leer y usar el contenido.
   - 2.3 No diseñar contenido de un modo que se sepa podría provocar ataques, espasmos o convulsiones.
   - 2.4 Proporcionar medios para ayudar a los usuarios a navegar, encontrar contenido y determinar dónde se encuentran.

3. Comprensible
   - 3.1 Hacer que los contenidos textuales resulten legibles y comprensibles.
   - 3.2 Hacer que las páginas web aparezcan y operen de manera predecible.
   - 3.3 Ayudar a los usuarios a evitar y corregir los errores.

4. Robusto
   - 4.1 Maximizar la compatibilidad con las aplicaciones de usuario actuales y futuras, incluyendo las ayudas técnicas.

© JMA 2020. All rights reserved

Conformidad

- Nivel de conformidad: Uno de los siguientes niveles de conformidad se satisface por completo.
  - Nivel A (el mínimo): la página web satisface todos los Criterios de Conformidad del Nivel A, o proporciona una versión alternativa conforme.
  - Nivel AA: la página web satisface todos los Criterios de Conformidad de los Niveles A y AA, o se proporciona una versión alternativa conforme al Nivel AA.
  - Nivel AAA: la página web satisface todos los Criterios de Conformidad de los Niveles A, AA y AAA, o proporciona una versión alternativa conforme al Nivel AAA.
- Páginas completas: La conformidad (y el nivel de conformidad) se aplica a páginas web completas, y no se puede alcanzar si se excluye una parte de la página.
- Procesos completos: Cuando una página web es parte de una serie de páginas web que presentan un proceso (es decir, una secuencia de pasos que es necesario completar para realizar una actividad), todas las páginas en ese proceso deben ser conformes con el nivel especificado o uno superior.

© JMA 2020. All rights reserved

Conformidad

- Uso de tecnologías exclusivamente según métodos que sean compatibles con la accesibilidad: Para satisfacer los criterios de conformidad sólo se depende de aquellos usos de las tecnologías que sean compatibles con la accesibilidad. Toda información o funcionalidad que se proporcione de una forma que no sea compatible con la accesibilidad debe estar disponible de una forma que sí sea compatible con la accesibilidad.
- Sin interferencia: Si las tecnologías se usan de una forma que no es compatible con la accesibilidad, o está usada de una forma que no cumple los requisitos de conformidad, no debe impedir a los usuarios acceder al contenido del resto de la página. Además, es necesario que la página web como un todo siga cumpliendo con los requisitos de conformidad en las siguientes circunstancias:
  - cuando cualquier tecnología de la que no se depende está activada en una aplicación de usuario,
  - cuando cualquier tecnología de la que no se depende está desactivada en una aplicación de usuario, y
  - cuando cualquier tecnología de la que no se depende no es soportada por una aplicación de usuario

© JMA 2020. All rights reserved

Conformidad

- Además, los siguientes criterios de conformidad se aplican a todo el contenido de la página, incluyendo el contenido del que, de todos modos, no se depende para alcanzar la conformidad, ya que su incumplimiento puede interferir con el uso de la página:
  - 1.4.2 - Control del audio,
  - 2.1.2 - Sin trampas para el foco del teclado,
  - 2.3.1 - Umbral de tres destellos o menos, y
  - 2.2.2 - Poner en pausa, detener, ocultar.

© JMA 2020. All rights reserved

Conformidad

PRINCIPIOS	NIVEL A	NIVEL AA	NIVEL AAA
PERCEPTIBILIDAD	❑ Textos alternativos ❑ Vídeos y audios pregrabados: Alternativas, subtítulos, controles... ❑ Independencia de la presentación ❑ Contrastes de colores	❑ Subtítulos en vídeos en directo ❑ Auto complementado ❑ Textos: ampliación y contrastes	❑ Lenguaje de señas ❑ Descripción y alternativas audio y multimedia ❑ Semántica etiquetas HTML ❑ Mayor contraste ❑ Eliminar ruido en audio ❑ Sin imágenes de textos
OPERABILIDAD	❑ Teclado, atajos y foco ❑ Control de temporizadores y movimiento ❑ Parpadeo de contenido ❑ Títulos y enlaces ❑ Orden de elementos y tabulación	❑ Ubicación del usuario en la web ❑ Títulos y enlaces descriptivos ❑ Diversos accesos a las páginas	❑ Interfaz de teclado ❑ Sesión de usuario ❑ Parpadeo del contenido ❑ Desactivar animaciones ❑ Organización de títulos y enlaces
COMPRENSIBILIDAD	❑ Cambio de idioma ❑ Foco y cambio de contextos ❑ Señalización de actuación y errores	❑ Cambiar idiomas de textos ❑ Mecanismos de navegación ❑ Errores y sugerencias ❑ Compromisos legales o transacciones financieras	❑ Definiciones para palabras inusuales, jergas y modismos ❑ Significado de abreviaturas ❑ Pronunciación ❑ Ayuda y prevención de errores
ROBUSTEZ	❑ Correcto marcado de etiquetas ❑ Identificadores y atributos no repetidos ❑ Interfaz de usuario	❑ Presentación de mensajes de estado	

© JMA 2020. All rights reserved

Declaraciones de conformidad

- La conformidad se aplica sólo a las páginas web. Sin embargo, la declaración de conformidad puede cubrir una sola página, una serie de páginas o múltiples páginas web relacionadas.
- Las declaraciones de conformidad no son obligatorias. Los autores pueden cumplir con los requisitos de las WCAG 2.0 sin realizar la declaración. Sin embargo, si se realiza la declaración, ésta debe contener la siguiente información:
  - Fecha de la declaración
  - Título de las pautas, versión y URI "Web Content Accessibility Guidelines 2.0 en http://www.w3.org/TR/2008/REC-WCAG20-20081211/"
  - Nivel de conformidad satisfecho: (Nivel A, AA o AAA)
  - Una breve descripción de las páginas web, como por ejemplo una lista de sus URI para las que se hace la declaración, incluyendo si los subdominios están incluidos en la declaración.
  - Una lista de las tecnologías de contenido web de las que se depende.

© JMA 2020. All rights reserved

Accessible Rich Internet Applications

- El W3C define WAI-ARIA como:
  - La forma para crear contenido Web y aplicaciones Web que sean accesibles para las personas con discapacidades.
- Una de las problemáticas más recientes de accesibilidad en la Web surgió con la llegada de Ajax, momento en el que los desarrolladores se "animaron" a usar esta tecnología motivados por sus posibilidades para la actualización dinámica del contenido y la **creación de diferentes controles a medida** (widgets). Todo esto con el fin de simular una GUI más parecida a las de las aplicaciones de escritorio, obteniendo como resultado aplicaciones web más ricas e interactivas, pero menos accesibles.
- La creación de widgets que combinan etiquetas HTML, CSS y JavaScript para definir nuevos comportamientos separan los elementos originales del estándar por lo que presentan problemas con las ayudas técnicas o tecnologías de apoyo como el lector de pantalla.

© JMA 2020. All rights reserved

ARIA

- ARIA proporciona un marco de trabajo complementario:
  - Estructuras más semánticas para las zonas funcionales.
  - Mejora de la navegación mediante el teclado.
  - Controles complejos (widgets) más accesibles.
  - Accesibilidad para el contenido actualizado de forma dinámica.
- Para ello ARIA cuenta con:
  - Roles: su misión es definir el papel que juegan los elemento dentro del documento web.
  - Estados y propiedades: determinan las características y los valores de cada elementos.
- Por tanto, ARIA no funciona como una tecnología restrictiva o exclusiva, sino que se trata de un complemento con el que podemos hacer accesibles las aplicaciones web enriquecidas.

© JMA 2020. All rights reserved

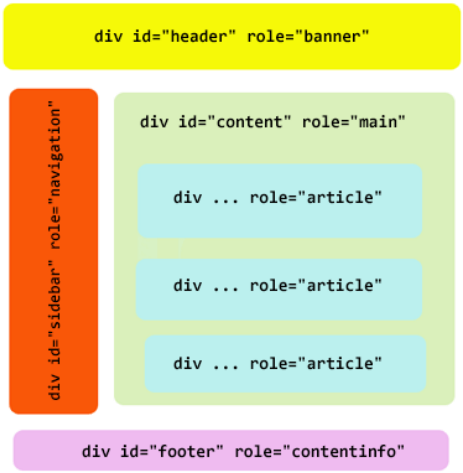
Estructura semántica de un documento

- Con ARIA podemos especificar los roles de las diferentes zonas funcionales de un documento web, haciendo que sea más semántico y accesible.
- Por ejemplo, actualmente estamos acostumbrados a usar un enlace para saltar al contenido principal (skip to content), mientras que ARIA nos permite, a través de los Document Landmarks, crear una estructura más semántica y accesible para que, entre otras cosas, las tecnologías asistidas puedan discernir donde se encuentra el contenido principal sin falta de recurrir a un enlace.
- Para crear una estructura semántica accesible con ARIA únicamente tenemos que especificar los roles de cada zona funcional por medio de la propiedad role:
  - application, banner, complementary, contentinfo, form, main, navigation, search

  <div id="navigation" role="navigation“>

© JMA 2020. All rights reserved

Roles



© JMA 2020. All rights reserved

Roles

Abstract Roles
- command
- composite
- input
- landmark
- range
- roletype
- section
- sectionhead
- select
- structure
- widget
- Window

Landmark Roles
- banner
- complementary
- contentinfo
- form
- main
- navigation
- region
- Search

Document Structure
- application
- article
- cell
- columnheader
- definition
- directory
- document
- feed
- figure
- group
- heading
- img
- list
- listitem
- math
- none
- note
- presentation
- row
- rowgroup
- rowheader
- separator
- table
- term
- toolbar
- Tooltip

Region Roles
- alert
- log
- marquee
- status
- Timer

Window Roles
- alertdialog
- Dialog

Widget Roles
- button
- checkbox
- gridcell
- link
- menuitem
- menuitemcheckbox
- menuitemradio
- option
- progressbar
- radio
- scrollbar
- searchbox
- separator
- slider
- spinbutton
- switch
- tab
- tabpanel
- textbox
- Treeitem

Widgets container Roles
- combobox
- grid
- listbox
- menu
- menubar
- radiogroup
- tablist
- tree
- treegrid

© JMA 2020. All rights reserved

Navegación mediante el teclado

- Por medio del teclado y a través del atributo tabindex podemos navegar entre los diferentes elementos de una web, pero sólo algunos elementos soportan este atributo y son capaces de recibir el foco: A, AREA, BUTTON, INPUT, OBJECT, SELECT, y TEXTAREA. Por tanto, un simple elemento DIV no puede ser accedido a través del teclado.
- Es justo aquí donde hace acto de presencia ARIA:
  - Haciendo que el atributo tabindex sea soportado por todos los elementos visibles de una web.
  - Permitiendo el valor “-1” en el atributo tabindex, lo que posibilita sacar un elemento del orden natural de navegación y del orden expresado por el índice de tabulación. Un elemento con el atributo tabindex="-1" únicamente podrá recibir el foco por medio de JavaScript (con el método focus()).
  - tabindex="0" permite que un elemento de marca o maquetación (div, p, li, ...) pueda recibir el foco (por defecto, no pueden) con lo que se podrá acceder con el tabulador hasta él.
  - Otros valores (enteros positivos) en tabindex definen un orden de tabulación explícito. No se recomienda su uso, salvo excepciones, porque puede crear confusión al alterar el flujo habitual de tabulación.

© JMA 2020. All rights reserved

Estados y Propiedades

- Los estados y las propiedades proporcionan información específica sobre un objeto y ambos forman parte de la definición de la naturaleza de los roles. Los roles son fijos, los estados y las propiedades cambian.
- La diferencia importante es que los valores de las propiedades (como aria-labelledby) a menudo tienen menos probabilidades de cambiar a lo largo del ciclo de vida de la aplicación que los valores de los estados (como aria-checked) que pueden cambiar con frecuencia debido a la interacción del usuario. WAI-ARIA los diferencia conceptualmente, aunque tiene poca importancia para la mayoría de los autores de contenido web, que se refieren tanto a unos como a otros simplemente como "atributos".
- Los estados y las propiedades se establecen como atributos de las etiquetas prefijados por "aria-".

```
<input id="startTime" type="text" aria-errormessage="msgID" value="" aria-invalid="false">
<span id="msgID" aria-live="off" style="visibility:hidden"> Invalid time:  the time must be between 9:00 AM and 5:00 PM" </span>

$('#startTime').attr('aria-invalid', 'true');
```

© JMA 2020. All rights reserved

Accesibilidad en los controles (widgets)

- HTML fue concebido para compartir documentos en la web y no para crear aplicaciones, razón por la cual no ofrece controles (widgets) avanzados.
- El diseño y desarrollo de un control de este tipo puede tener el siguiente aspecto:

```
<div id="slider-bg" title="level">
    <div id="slider-handler"><img src="handler.gif" /></div>
</div>
```

- Problema de la accesibilidad resuelto con ARIA:

```
<p id="slider-description">Puede usar las teclas derecha/izquierda para cambiar el nivel.</p>
<span id="slider-label">Nivel:</span>
<div id="slider-rail">
    <button id="slider-handler" role="slider" aria-labelledby="slider-label"
        aria-describedby="slider-description" aria-valuemin="1" aria-valuemax="3"
        aria-valuenow="2"></button>
</div>
```

© JMA 2020. All rights reserved

Accesibilidad en actualizaciones dinámicas de contenido

- Otro de los grandes problemas de la accesibilidad en las aplicaciones web enriquecidas, recae sobre la actualización dinámica del contenido (o parte del contenido) que se realiza por medio de Ajax y en “segundo plano”.
- ARIA denomina “regiones activas” a los elementos/zonas que pueden presentar estos cambios, y cuenta con la propiedad aria-live con la que indicar el valor de “intrusismo” (off, polite, assertive o rude) sobre la actividad actual del usuario.

```
<label for="user-name">
    Nombre (requerido)
    <input name="user-name" id="user-name" type="text" size="20"
        maxlength="20" aria-required="true" />
    <span id="count" aria-live="polite"></span> pendientes.
</label>
```

© JMA 2020. All rights reserved

Pruebas de Accesibilidad

- Las pruebas de Accesibilidad, como el resto de las pruebas de interfaces de usuario con el diseño de entrada y salida, se basan en la experiencia para descubrir los casos de prueba.
- Las reglas de diseño pueden establecerse en varios documentos, pero generalmente se describen en:
  - Guías de estilo, que muchas veces contienen pautas o reglas para toda la organización, relativas a cuestiones como el uso del color, las fuentes, la disposición de la pantalla, etc.
  - Especificaciones para el diseño de la lista o pantalla correspondiente.
  - Normativas aplicables
- Las pruebas se apoyaran en herramientas de análisis estático para eliminar todos los casos de pruebas automatizable, pero hay una serie de principios y pautas (como la 3, Comprensible) que actualmente no se pueden automatizar.
- La técnica, es dichos casos, se basa en listas de control (checklist), una serie de cuestionarios que contienen las situaciones más habituales de error. La identificación de los casos de prueba se realiza recorriendo cada una de las cuestiones de cada lista, estudiando si es o no aplicable. En caso de ser aplicable necesitas diseñar un caso de prueba.

© JMA 2020. All rights reserved

Buenas practicas

- Listas de verificación
  - Lista de verificación WCAG de Wuhcag
  - Lista de verificación WCAG de WebAIM
  - Lista de verificación del A11yProject
- HTML semántico
  - El HTML semántico es la base de la accesibilidad en una aplicación web.
  - Haciendo uso de los diversos elementos HTML para reforzar el significado de la información en nuestros sitios web a menudo nos dará accesibilidad de forma gratuita.

© JMA 2020. All rights reserved

Buenas practicas

- Configurando el idioma
  - Indique el idioma humano de los textos de la página, ya que el software del lector de pantalla lo utiliza para seleccionar la configuración de voz correcta:
    - WebAIM - Documento de lenguaje
- Configuración del título del documento
  - Configure el <title> del documento para que describa correctamente el contenido de la página actual, ya que esto garantiza que el usuario esté al tanto del contexto de la página actual:
    - WCAG - Comprendiendo el requisito del título del documento
- Contraste de color
  - Asegúrese de que todo el texto legible en su sitio web tenga el contraste de color suficiente para que los usuarios con poca visión puedan leerlo al máximo:
    - WCAG - Comprender el requisito de contraste de color
    - Todo sobre el contraste de color y por qué debes repensarlo
    - A11yProject - ¿Qué es el contraste de color?
  - Se puede calcular una paleta de colores accesible completa con Colorable.

© JMA 2020. All rights reserved

Formularios accesibles

- Etiquetado
  - Todos los controles de formulario HTML, como <input> y <textarea>, deben ser etiquetados de forma accesible. Necesitamos proporcionar etiquetas descriptivas que también estén expuestas a los lectores de pantalla.

```
<label for="namedInput">Name:</label>
<input id="namedInput" type="text" name="name"/>
```

  - Los siguientes recursos nos muestran cómo hacer esto:
    - El W3C nos muestra cómo etiquetar elementos
    - WebAIM nos muestra cómo etiquetar elementos
    - El Grupo Paciello explica los nombres accesibles

© JMA 2020. All rights reserved

Formularios accesibles

- Notificando errores al usuario
  - Las situaciones de error deben ser entendidas por todos los usuarios. El siguiente enlace también nos muestra cómo exponer textos de error a lectores de pantalla:
    - El W3C demuestra notificaciones de usuario
    - WebAIM analiza la validación de formularios
- Control de foco
  - Asegúrese de que su aplicación web pueda ser operada completamente solo con el teclado:
    - WebAIM habla sobre accesibilidad de teclado
- Eventos de ratón y puntero
  - Asegúrate que también se puede acceder a todas las funciones expuestas a través de un evento de mouse o puntero utilizando solo el teclado. Dependiendo solo del dispositivo puntero llevará a muchos casos donde Los usuarios de teclado no pueden usar tu aplicación.

© JMA 2020. All rights reserved

Herramientas de Pruebas de GUI

- Hay herramientas disponibles, especialmente para aplicaciones web, que pueden realizar todo tipo de comprobaciones. Ejemplos de estos controles son:
  - ¿Los gráficos y animaciones cuentan con una alternativa (un cuadro de texto) para proporcionar la misma información en caso de que los gráficos, animaciones, etc. no funcionen? Este puede ser el caso si utilizas un navegador diferente, no tienes una tarjeta de vídeo o tienes una discapacidad visual.
- ¿El tamaño de los gráficos es demasiado grande, lo que hace que el sitio sea lento?
- ¿Cada página contiene un enlace para regresar a la página anterior y/o un enlace para continuar a la página siguiente?
- ¿Quizás los cuadros de texto son demasiado largos en un campo de desplazamiento?
- ¿Son todos los hipervínculos (todavía) válidos?

© JMA 2020. All rights reserved

Herramientas de análisis estático Web

- Adecuación a estándares
  - HTML
    - https://validator.w3.org/
  - CSS
    - http://jigsaw.w3.org/css-validator/
- Accesibilidad
  - WAI: Web Accessibility Evaluation Tools List
    - https://www.w3.org/WAI/ER/tools/
  - WAVE Web Accessibility Evaluation Tools
    - https://wave.webaim.org/
  - Total Validator
    - https://www.totalvalidator.com/
- Lighthouse en Chrome DevTools (panel "Audits")
  - https://github.com/GoogleChrome/lighthouse

© JMA 2020. All rights reserved